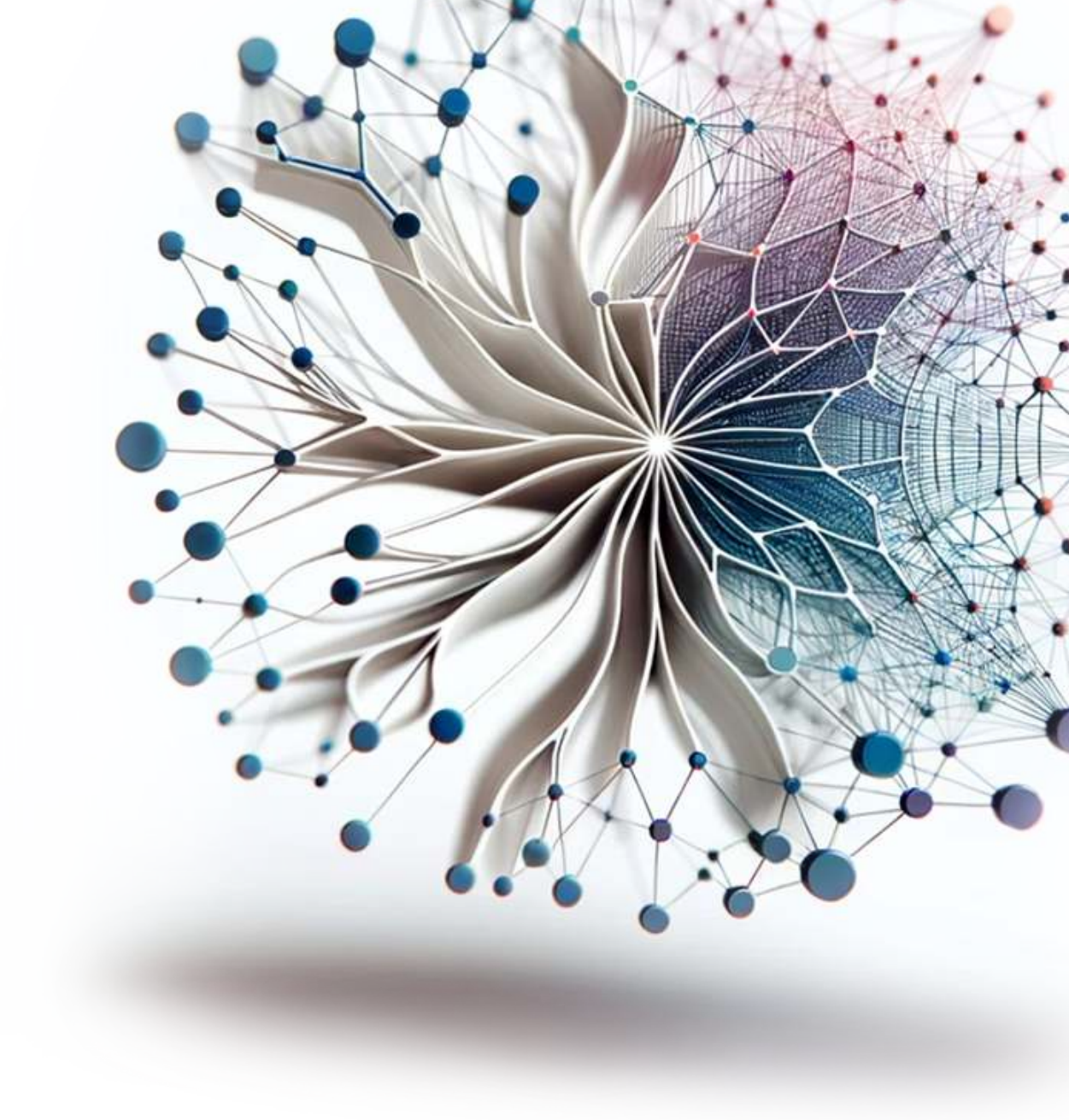


Introduction to

Machine Learning Systems

Vijay
Janapa Reddi

Introduction to Machine Learning Systems

Introduction to Machine Learning Systems

Vijay Janapa Reddi

The MIT Press
Cambridge, Massachusetts
London, England

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers. This book was prepared and formatted in Quarto by Željko Hrček. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-05888-9

EU Authorised Representative: Easy Access System Europe, Mustamäe tee 50, 10621 Tallinn, Estonia | Email: gpsr.requests@easproject.com

In loving memory of my father,
whose passion for learning and unwavering commitment to quality
challenge me daily to strive for excellence.

And to my mother,
who taught me, by instinct more than instruction,
that knowledge—like everything else worth having—
only matters if you give it away.

Table of Contents

Author's Note

The world is rushing to build AI systems. It is not yet engineering them. That gap is what we mean by AI engineering. Who designs the training infrastructure? Who builds serving systems that scale? Who optimizes models to run on a phone or a sensor? Who architects the accelerators those models execute on? That work is AI engineering: the discipline of building efficient, reliable, safe, and robust intelligent systems that operate in the real world, not models in isolation. Yet the work itself is still not recognized as a discipline.

The consequences are visible in practice. By most industry estimates, the vast majority of AI projects never reach production or fail to deliver the value they promised. The failures are not exotic: a model degrades silently because no one built monitoring for distribution shift; a system that worked in the lab fails at the edge because latency requirements were never communicated to the team selecting architectures; a deployment pipeline breaks because the team retrained a model without versioning the data and cannot reproduce last week's results. These are not research problems. They are engineering problems, and they recur because the field lacks the shared principles, vocabulary, and training that a discipline provides.

That missing discipline is also reflected in fragmentation. The knowledge to build ML systems exists, but it is scattered across silos. One course teaches compilers. Another covers hardware architecture. A third focuses on data pipelines. A fourth introduces the mathematics of optimization. Each is valuable on its own, but none of them teaches the *system*, and the result is engineers who understand individual components but not how they fit together. Nobody learns how a computer works by studying the processor in one course, the memory hierarchy in another, and the interconnect in a third, without ever building a complete machine. A computer is an integrated machine; understanding it means understanding where bottlenecks form between components, where trade-offs hide, and where one design decision constrains every other. An ML system is no different. Data pipelines, model architectures, software frameworks, and accelerator hardware constrain each other and fail together. The engineer who understands only one piece will build systems that break at the seams.

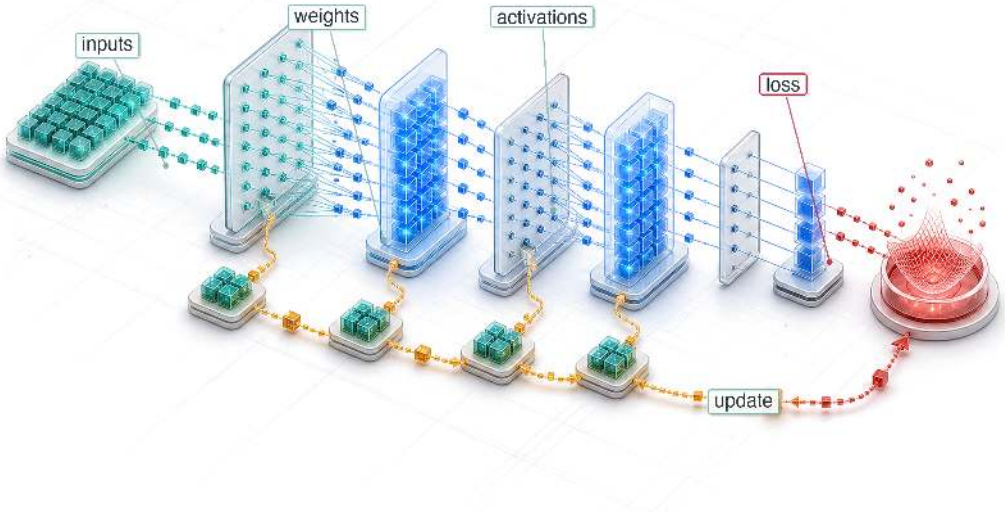
This book addresses that fragmentation by providing the missing integration: not a survey of tools, not a collection of recipes, but the enduring principles that govern ML systems regardless of which framework or accelerator is fashionable this year. We do not teach operating systems by starting with distributed operating systems or embedded operating systems; we teach the fundamentals (scheduling, memory management, concurrency) and then specialization follows. That philosophy guides this book. Whether you are building a model that runs on a microcontroller or training one across a thousand GPUs, the principles are the same: memory bandwidth limits data movement, arithmetic intensity governs compute utilization, and the interplay between data, models, and hardware determines what is practical to build. The scale changes; the principles do not.

The existence of such principles is precisely what makes a discipline possible. Software engineering and computer engineering each emerged when practitioners recognized that building reliable systems required its own body of knowledge, distinct from the science that produced the underlying ideas. AI engineering is at that same inflection point. The best textbooks I encountered as a student, in both traditions, did not teach me subjects; they taught me how to think. They took vast, messy landscapes and revealed the structure underneath. That is what I have tried to do here.

— Vijay Janapa Reddi

1

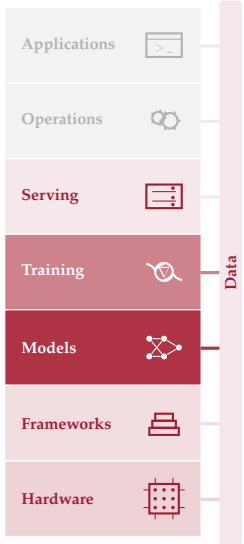
Neural Computation



Purpose

Why does understanding a neural network's math matter more than reading its code?

Neural networks reduce to a small set of mathematical operations. Matrix multiplications dominate compute. Activation functions, the nonlinear gates between layers, introduce nonlinearity. Gradient computations, the error-sensitivity calculations that drive parameter updates, enable learning. These operations are the workload that every layer of the system stack must execute, and each carries concrete physical consequences: a matrix multiplication's dimensions determine how many operations and bytes must move; an activation function's complexity determines how much extra work each layer adds; the number of parameters determines whether a model fits in memory at all. When something goes wrong, inspecting the code reveals little because it simply says "multiply these matrices." The bug is not in the logic but in the math itself: a misconfigured learning rate, the update step size, can make gradients explode; an activation can saturate, stop changing with its input, and silently block learning; a memory footprint can fit during development but exhaust the target machine in production. This is why the mathematical primitives come first, before architectures, frameworks, or training systems. Every subsequent chapter builds on these operations: architectures compose them into larger models, frameworks execute them efficiently, training systems repeat them billions of times, and compression techniques approximate them to fit tighter constraints. An engineer who understands the primitives can look at any new architecture and immediately reason about its compute profile, its memory demands, and its hardware compatibility, because they understand the atoms it is made of. In D·A·M terms, these mathematical primitives form the atomic interface between algorithm and machine, translating abstract logic into the physical constraints of memory and arithmetic.



Learning Objectives

- Explain why learned arithmetic replaced rule-based logic for high-dimensional pattern recognition
- Calculate parameter counts, multiply-accumulate operations, activation storage, and memory traffic for multilayer networks
- Compare activation functions by nonlinearity, gradient behavior, and hardware execution cost
- Apply cross-entropy loss and gradient updates to reason about supervised learning dynamics
- Derive forward and backward propagation as matrix operations over batches
- Analyze training versus inference workloads using compute, memory, and deployment constraints
- Evaluate an end-to-end inference pipeline from preprocessing through decision thresholds and USPS-style deployment

1.1 From Logic to Arithmetic

A compiled dataset is clean, versioned, and ready for consumption, but examples do not learn on their own. The next system boundary is the model, where data becomes learned behavior through matrix multiplications, activation functions, and gradient updates. A model that runs correctly on one machine and crashes on another is not necessarily suffering from a hardware bug. Its layer dimensions may exceed the memory available for intermediate activations, the per-layer outputs that training must keep for the backward pass. The failure comes from the mathematics inside the model, not the code around it.

The silicon contract says that every model architecture makes a computational bargain with the hardware it runs on. On the Algorithm axis, the architecture's mathematical operators set the terms of that bargain: they determine how much memory the model consumes, how long each computation takes, and how much energy the system expends. To honor the contract, a systems engineer must understand those operators.

The operators that follow are not abstract theory but a specification for computational workloads. Neural computation represents a qualitative shift in how we process information: instead of executing a sequence of explicit logical instructions (if-then-else), we execute a massive sequence of continuous mathematical transformations (multiply-add-accumulate). This shift from *Logic* to *Arithmetic* changes everything for the systems engineer, creating dense tensor workloads whose bottleneck depends on arithmetic intensity, batch size, reuse, and the hardware roofline (the performance envelope set by a chip's peak compute and memory bandwidth). The "bug" in such a system is rarely a syntax error; it is numerical instability, a gradient that shrinks toward zero, or an activation function that stops changing with its input. Concretely, recognizing a single handwritten digit in the running MNIST network requires 109,184 MACs—not one of which is a logical branch.

Arithmetic without branches is not arithmetic without risk: a number that exceeds the range its format can represent fails silently, and the consequences scale with the system that depends on it. A canonical example comes from outside machine learning entirely.

War Story 1.1: The overflow that became guidance (1996)

Context: The European Space Agency's Ariane 5 Flight 501 reused inertial reference software from Ariane 4, including numeric conversions whose assumptions matched the older vehicle's flight profile (European Space Agency 1996).

Failure mode: Ariane 5's faster horizontal motion drove a value outside the range expected by a protected conversion. The resulting exception shut down inertial guidance data instead

of being treated as a recoverable numerical fault. About 40 seconds after the flight sequence began, the launcher veered off course, broke up, and exploded (European Space Agency 1996).

Systems lesson: Numerical ranges, exception handling, and reuse assumptions are part of the systems contract. A computation can be syntactically correct and still be invalid for the physical regime in which it runs. ML systems hit this whenever a numerical format's representable range fails to match the magnitudes flowing through it: a low-precision multiplication that overflows produces an Inf or NaN that propagates silently through the rest of the computation, and a conversion that worked at one scale of activations can fail at another exactly as Ariane 5's reused conversion did.

This arithmetic-first style of computation, where failure hides in numerical regimes rather than in control flow, is exactly what defines the dominant paradigm of modern machine learning.

Definition 1.1: Deep learning

Deep Learning is the computational paradigm that learns hierarchical feature representations directly from raw data by composing layers of nonlinear transformations, trading increased computation (O) for the ability to generalize without manual feature engineering.

1. **Significance**: Depth is the mechanism that converts compute into capability. Each additional layer increases O proportionally, but the representational capacity grows combinatorially: a k-layer network can compose k stages of learned abstraction, where each stage reuses the features below it. Within the iron law, deep learning shifts the binding constraint from the algorithm axis (hand-designed features that limit accuracy) to the machine axis (compute and memory to train and store the hierarchy).
2. **Distinction**: Unlike shallow learning (for example, logistic regression, support vector machines), which learns a single input-to-output mapping and requires hand-crafted features for complex domains, deep learning learns a hierarchy of increasingly abstract representations that can be transferred across tasks, amortizing the compute investment across downstream applications.
3. **Common pitfall**: A frequent misconception is that deep learning is "just a big neural network." Scaling a shallow model wider does not replicate what depth provides: compositional feature hierarchies that reuse lower-level representations. The depth, not the parameter count alone, is what makes deep learning a distinct computational paradigm.

The development starts from deep learning as a computational workload: the primitive operations every network reduces to, how those primitives compose into network structure, what the learning process and inference pipeline demand of the hardware, and how the whole maps onto the D·A·M taxonomy, grounded in a single handwritten-digit recognizer that makes each cost concrete. The landmark *Nature* review by LeCun, Bengio, and Hinton[1] (LeCun et al. 2015) formalized this paradigm.

Classical machine learning required human experts to design feature extractors for each new problem, a labor-intensive process that encoded domain knowledge into handcrafted representations. Deep learning eliminates this bottleneck by learning representations directly from raw data through hierarchical layers of nonlinear transformations. To see where neural networks fit in the broader landscape, figure 1.1 traces seven decades of this progression: as the field narrowed from symbolic artificial intelligence to statistical machine learning to deep learning, each era nested inside the prior, so deep learning is a subset of machine learning, which is itself a subset of artificial intelligence.

This paradigm shift creates an engineering problem with no precedent in traditional software. When conventional software fails, an error message points to a line of code. When deep learning fails, the symptoms are subtler: gradient instabilities[2] that silently prevent learning, numerical precision errors that corrupt model weights over thousands of iterations, or memory access patterns in tensor operations[3] that leave GPUs idle for most of each training step. These are not algorithmic bugs that

1 **LeCun, Bengio, and Hinton**: Recipients of the 2018 ACM Turing Award, their individual contributions (convolutional networks from LeCun, probabilistic sequence models from Bengio, and backpropagation training from Hinton) directly shaped the three operations that dominate modern accelerator workloads: local spatial filtering, sequence modeling, and gradient computation.

2 **Gradient Instabilities**: In a 20-layer sigmoid network, gradient magnitude after backpropagation is approximately 0.25^{20}, or 9.1×10^{-13}—effectively zero, making learning a mathematical impossibility without architectural intervention. These failures are invisible in standard logs (loss simply plateaus or becomes not a number (NaN)), making them among the hardest bugs to diagnose. Rectified linear unit (ReLU) activations (gradient of one for positive inputs) and residual connections (direct gradient highways that bypass layers) were the two architectural breakthroughs that made deep networks tractable; **?@sec-network-architectures-evolution-dense-spatial-processing-2761** treats the residual-connection depth solution in detail.

3 **Tensor Operations**: The logical structure of a tensor (for example, a 4D image batch) often requires nonsequential memory access patterns to retrieve elements from its flat, 1D physical storage. A concrete example is changing an image tensor from channel-first order to channel-last order before execution. A typical ImageNet input shape (224×224×3) is about 151 KB, so transposing it between formats requires about 301 KB of read-plus-write memory traffic per image, a pure memory operation with no arithmetic benefit. On tight latency budgets, this layout mismatch can consume a visible fraction of the end-to-end inference budget before the model does any useful math.

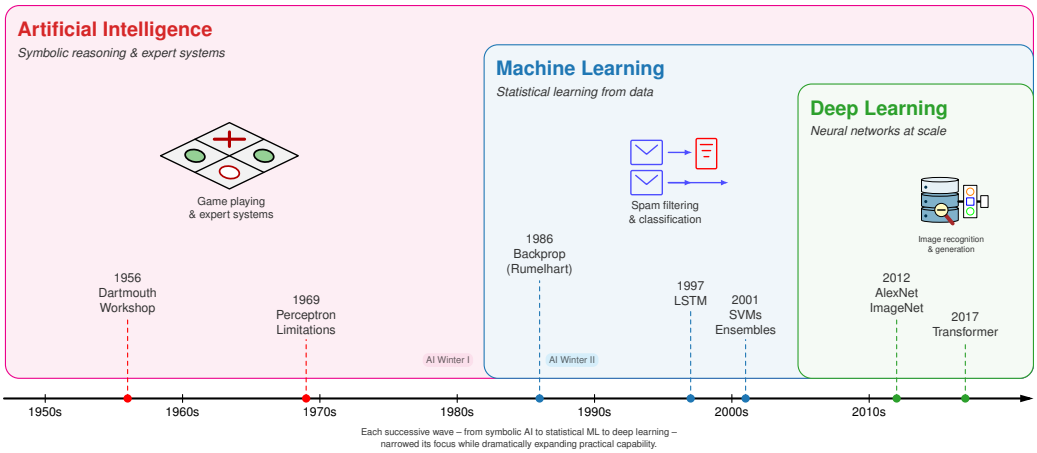


Figure 1.1: AI Hierarchy and Timeline: A timeline from the 1950s to the 2010s, overlaid with nested era-bands for artificial intelligence, machine learning, and deep learning. Each successive wave narrows in scope while nesting inside the prior, so deep learning falls within machine learning, which falls within artificial intelligence. Dated milestones, from the 1956 Dartmouth Workshop through the 2017 transformer, mark the transition between eras.

a debugger can catch. They are systems problems that require understanding the mathematical machinery underneath.

Diagnosing and solving such problems requires mathematical literacy that spans the full neural computation stack. The arc begins with learning paradigms, tracing how they evolved from explicit rules to handcrafted features to learned representations and establishing *why* deep learning demands qualitatively different system infrastructure than classical machine learning. Neural network fundamentals (neurons, layers, activation functions, and tensor operations) then receive treatment as both mathematical operations and computational workloads, with particular attention to the memory access patterns and arithmetic intensity that determine hardware utilization.

The learning process then takes center stage: the forward pass that produces predictions, the backpropagation algorithm that computes gradients, the loss functions that define optimization objectives, and the optimization algorithms that navigate loss landscapes. Each connects directly to system engineering decisions: matrix multiplication illuminates memory bandwidth requirements (the memory wall explored in **?@sec-hardware-acceleration**), gradient computation explains numerical precision constraints, and optimization dynamics inform resource allocation. The inference pipeline shifts the engineering concerns from throughput to latency and from training stability to deployment efficiency. A historical case study (USPS digit recognition) grounds these concepts in a real deployment, and the D·A·M taxonomy (Data, Algorithm, Machine) closes the arc by explaining why deep learning systems succeed only when all three components align.

The common thread is that each mathematical choice creates a workload profile. We make that thread concrete by following a single MNIST digit through three computational paradigms and quantifying how each step changes the system's work.

1.2 Computing with Patterns

The shift from logic to arithmetic reshapes how we encode real-world patterns in a form a computer can process. To make this evolution concrete, we track a single task across all three paradigms: classifying a handwritten digit from a 28×28 pixel image from the MNIST dataset (the same input used throughout this chapter). The computational profile changes as representation strategies evolve.

1.2.1 From explicit logic to learned patterns

Rule-based programming requires developers to explicitly define rules that tell computers how to process inputs and produce outputs. Consider a simple game like Breakout[4]. The program needs explicit rules for every interaction: when the ball hits a brick, the code must specify that the brick

4 **Breakout (DQN)**: Atari's 1976 arcade game became an AI milestone when DeepMind's DQN learned to play it from raw pixels alone (2015), requiring no programmed rules. The systems implication: DQN preprocessed each Atari frame to 84×84 pixels and stacked 4 frames as input, selecting a new action every 4 frames (about 15 decisions per second on a 60 Hz game). This real-time inference loop pushed GPU utilization beyond what labeled-example training typically required and foreshadowed the latency constraints of production inference pipelines.

should be removed and the ball's direction should be reversed (figure 1.2). While this approach works effectively for games with clear physics and limited states, it hits a wall when dealing with the messy, unstructured data of the real world.

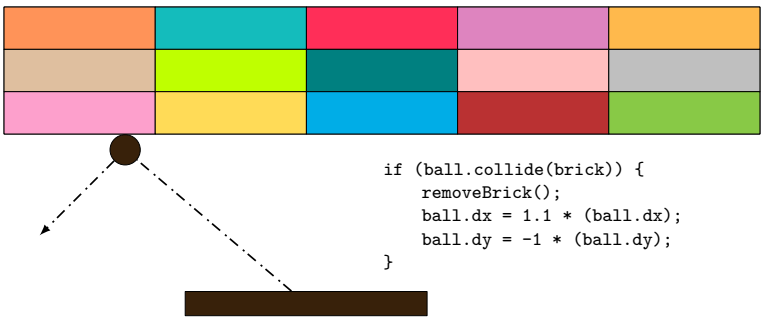


Figure 1.2: Breakout Collision Rules: The game program uses explicit if-then rules for collision detection, specifying ball direction reversal and brick removal upon contact. While effective for a game with clear physics and limited states, this approach illustrates how rule-based systems must anticipate every possible scenario.

Beyond individual applications, this rule-based paradigm extends to all traditional programming. The data flow in figure 1.3 makes the constraint explicit: the program takes both rules for processing and input data to produce outputs. Early artificial intelligence research explored whether this approach could scale to solve complex problems by encoding sufficient rules to capture intelligent behavior.

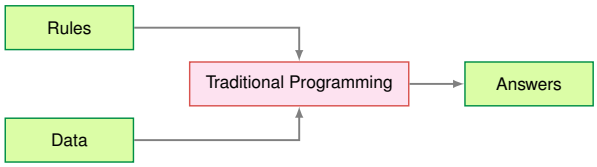


Figure 1.3: Traditional Programming Flow: Rules and data serve as inputs to a traditional program, which produces answers as output. This input-output pattern formed the basis for early AI systems but lacks the adaptability needed for complex pattern recognition tasks.

Despite their apparent simplicity, rule-based limitations surface quickly with complex real-world tasks. Recognizing human activities illustrates the challenge. Classifying movement below 6 km/h as walking seems straightforward until real-world complexity intrudes. Speed variations, transitions between activities, and boundary cases each demand additional rules, creating unwieldy decision trees (figure 1.4). Computer vision tasks compound these difficulties. Detecting cats requires rules about ears, whiskers, and body shapes while accounting for viewing angles, lighting, occlusions, and natural variations. Early systems achieved success only in controlled environments with well-defined constraints.

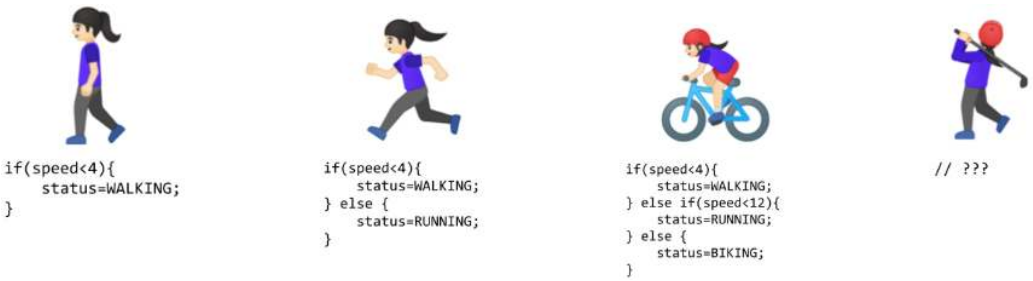


Figure 1.4: Activity Classification Decision Tree: A rule-based decision tree classifies human activity by branching on speed thresholds, with values below 6 km/h mapped to walking, 6 to 20 km/h to running, and above 20 km/h to biking. Real-world edge cases and transitions between activities demand increasingly complex branching logic.

Recognizing these limitations, the knowledge engineering approach that characterized AI research in the 1970s and 1980s attempted to systematize rule creation. Expert systems[5] encoded domain knowledge as explicit rules, showing promise in specific domains with well-defined parameters but struggling with tasks humans perform naturally: object recognition, speech understanding, and natural language interpretation. These failures highlighted a deeper challenge: many aspects of intelligent behavior rely on implicit knowledge that resists explicit rule-based representation.

Consider classifying our 28 by 28 digit with explicit rules: compare pixel intensities against thresholds, check stroke patterns in specific regions, branch on the results. The entire computation is roughly 100 comparisons over 784 bytes of pixel data—sequential, predictable, and comfortably within any CPU's L1 cache. No special hardware needed. That simplicity is exactly what disappears as we move toward learned representations.

1.2.2 The feature engineering bottleneck

The failures of rule-based systems suggested an alternative: rather than encoding human knowledge as explicit rules, let the system discover patterns from data. Machine learning offered this direction—instead of writing rules for every situation, researchers wrote programs that identified patterns in examples. The success of these methods, however, still depended heavily on human insight to define which patterns to look for, a process known as feature engineering.

Feature engineering transformed raw data into representations that expose patterns to learning algorithms. The Histogram of Oriented Gradients (HOG)[6] (Dalal and Triggs 2005) method exemplifies this approach, identifying edges where brightness changes sharply, dividing images into cells, and measuring edge orientations within each cell (figure 1.5). This transforms raw pixels into shape descriptors robust to lighting variations and small positional changes.

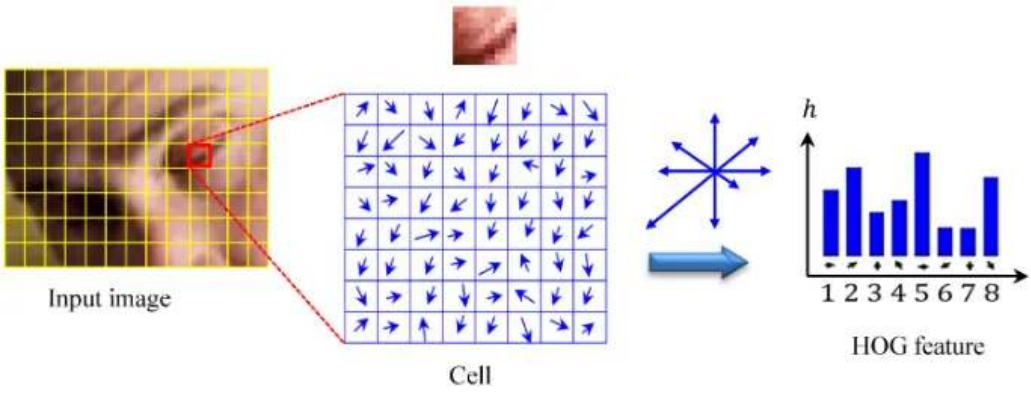


Figure 1.5: HOG Method: Identifies edges in images to create a histogram of gradients, transforming pixel values into shape descriptors that are invariant to lighting changes.

Complementary methods like SIFT[7] (Lowe 1999) (Scale-Invariant Feature Transform) and Gabor filters[8] captured different visual patterns. SIFT detected keypoints stable across scale and orientation changes, while Gabor filters identified textures and frequencies. Each encoded domain expertise about visual pattern recognition.

These engineering efforts enabled advances in computer vision during the 2000s. Systems could now recognize objects with some robustness to real-world variations, leading to applications in face detection, pedestrian detection, and object recognition. Despite these successes, the approach had limitations. Experts needed to carefully design feature extractors for each new problem, and the resulting features might miss important patterns that were not anticipated in their design. The bottleneck remained: human expertise could not scale to the complexity and diversity of real-world visual patterns.

Return to the same 28 by 28 digit. HOG divides the image into a 7 by 7 grid of 4 by 4 cells, computes gradient magnitudes and orientations at each pixel, bins them into 9 orientation histograms per cell, and produces a 441-element feature vector. A linear classifier (SVM) then performs ten dot products over that vector. Total: roughly 8,000 operations and about 2 KB of working memory—about 80×

5 | **Expert Systems**: These systems convert human expertise into explicit `IF-THEN` rules. This 'knowledge engineering' approach fails for tasks like object recognition, as the text notes, because the required knowledge is implicit and resists articulation. Even in a successful system (DEC's XCON), the maintenance of 10,000+ hand-authored rules revealed an unsustainable scaling cost that motivated the shift to learned representations.

6 | **Histogram of Oriented Gradients (HOG)**: An influential handcrafted descriptor for pedestrian detection before deep learning (Dalal and Triggs 2005). HOG computes gradient orientations in fixed 8×8 pixel cells, a rigid spatial decomposition that requires expert tuning per domain. The systems contrast with deep learning is instructive: HOG's fixed computation graph runs efficiently on CPUs with predictable latency, while learned features demand GPU parallelism but generalize across domains without redesign.

7 | **Scale-Invariant Feature Transform (SIFT)**: SIFT encodes this "domain expertise" in a rigid, four-stage algorithm that identifies keypoints invariant to scale and rotation. This hand-engineering meant the number of keypoints varied unpredictably per image, from hundreds to thousands. This variable-sized output is mechanically incompatible with the fixed-size tensors that modern hardware accelerators demand.

8 | **Gabor Filters**: Originally developed for localized time-frequency analysis (Gabor 1946), these filters detect edges and textures at specific orientations and frequencies. A typical bank contains many hand-designed filters across orientations and frequencies. Deep convolutional layers instead learn small kernels from data; early CNN filters often become edge-, color-, and texture-sensitive detectors, shifting feature design from manual filter banks to data-driven optimization (Krizhevsky et al. 2012; LeCun et al. 2015).

more compute than the rule-based approach, but still structured, predictable, and well-served by CPU vector units using single instruction, multiple data (SIMD). Resource demands scale linearly with image count, not with model complexity.

1.2.3 Automatic pattern discovery

The limitations of handcrafted features motivate a more radical approach: rather than encoding features by hand, the system discovers its own. Neural networks embody exactly this shift—rather than following explicit rules or relying on human-designed feature extractors, the system learns representations directly from raw data.

Deep learning inverts the traditional programming relationship entirely. Traditional programming, as we saw earlier, required both rules and data as inputs to produce answers. Machine learning reverses this: examples (data) and their correct answers become the inputs, and the system discovers the underlying rules automatically. Figure 1.6 makes this inversion tangible by showing rules as the output rather than the input. This shift eliminates the need for humans to specify what patterns are important.

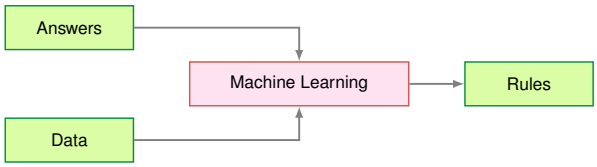


Figure 1.6: Data-Driven Rule Discovery: The flow diagram inverts the traditional programming pattern: data and answers serve as inputs to the machine learning process, which produces learned rules as output. This inversion eliminates the need for manually specified rules and enables automated feature extraction from raw inputs.

The system discovers patterns from examples through this automated process. When shown millions of images of cats, it learns to identify increasingly complex visual patterns, from simple edges to combinations that constitute cat-like features. This parallels how biological visual systems operate, building understanding from basic visual elements to complex objects.

The gradual layering of patterns reveals *why* neural network depth matters. Deeper networks can express exponentially more functions with only polynomially more parameters, a compositionality advantage we formalize in section 1.3.1 with a concrete MNIST example.

Deep learning exhibits predictable scaling: unlike traditional approaches where performance plateaus, these models continue improving with additional data (recognizing more variations) and computation (discovering subtler patterns). The scalability drove dramatic performance gains. In the ImageNet competition, traditional methods achieved approximately 25.8 percent top-5 error in 2011. AlexNet[9] reduced this to 15.3 percent in 2012. By 2015, ResNet achieved 3.6 percent top-5 error, surpassing estimated human performance of approximately 5.1 percent.

Figure 1.7 previews this scaling behavior through three distinct regimes. The underlying mechanisms (training error, overfitting, gradient-based learning) are developed in subsequent sections; here we establish the shape of the phenomenon. The Classical Regime is where traditional statistical intuitions hold, the Interpolation Threshold is where the model perfectly fits training data, and the Modern Regime is where massive overparameterization paradoxically improves generalization. The axes are normalized to emphasize shape rather than a specific dataset.

The counterintuitive shape matters because test error, the error on held-out examples rather than the training examples the model sees directly, initially follows the expected U-curve, then decreases again when the model has more parameters than the simplest theory expects. This scaling behavior resolves the central paradox of deep learning. Classical statistical theory predicted that models should be sized to match data complexity: too small and they underfit, too large and they overfit by memorizing noise. The bias-variance trade-off[10] suggested that massive models would inevitably fail on new data. Instead, double descent (Belkin et al. 2019) shows that larger models, trained with sufficient data and regularization, can sometimes find smoother solutions that generalize better than smaller ones. This overparameterization effect means that scale can be an engineering lever, not that scale is automatically safe: the data distribution, training procedure, and regularization still determine whether the extra capacity helps or memorizes. Overfitting and the regularization

9 **AlexNet's Memory Split**: Krizhevsky's team split AlexNet across two NVIDIA GTX 580s not by architectural preference but by physical constraint: each card had only 3 GB of VRAM, and the full model required more memory than a single card could provide. The workaround is the important systems lesson at this point in the book: model math can exceed a single device's memory budget, forcing engineers to divide work across machines. General parallelism strategies grow from the same constraint.

10 **Bias-Variance Trade-off**: In overparameterized networks (parameter count » training samples), the classical bias-variance trade-off breaks down: test error decreases again after the interpolation threshold, the Double Descent phenomenon. The systems consequence is that model size cannot be treated as a monotonic overfitting risk once the interpolation threshold is crossed. Larger models can become useful engineering options, provided data coverage, optimization, and regularization keep added capacity from memorizing noise.

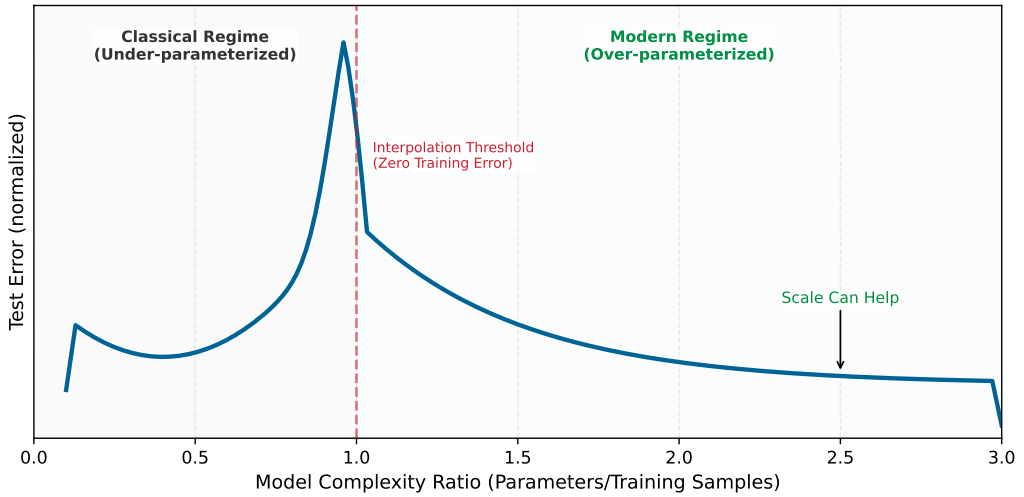


Figure 1.7: The Double Descent Phenomenon: Why modern deep learning defies classical statistics. In the *Classical Regime* (left), increasing model complexity eventually leads to overfitting (the "U" curve). Past the *Interpolation Threshold* (middle), test error drops again in the *Modern Regime* (right). Axes are normalized and the curve is illustrative.

techniques that control it receive formal treatment in section 1.4.5.4; this preview needs only the shape of the curve.

Neural network performance follows empirical scaling relationships with direct systems consequences. The durable anchor is that frontier model sizes and training compute budgets have grown by orders of magnitude over the past decade (section 1.2.7 quantifies the trajectory), and that growth shifted the binding constraint from arithmetic throughput to data movement: at scale, memory bandwidth and storage capacity set the ceiling, not raw FLOP/s. **?@sec-model-training** develops the quantitative scaling formulations, including how model size, data, and compute trade off against one another; **?@sec-model-compression** explores the practical responses.

Learning directly from raw data reshapes AI system construction. Eliminating manual feature engineering introduces new demands: infrastructure to handle massive datasets, high-throughput hardware to process that data, and specialized accelerators to perform mathematical calculations efficiently. These computational requirements have driven the development of specialized chips optimized for neural network operations. Empirical evidence confirms this pattern across domains: the success of deep learning in computer vision, speech recognition, game playing, and natural language understanding has established it as the dominant paradigm in artificial intelligence.

Return to the same 28 by 28 digit, now processed by even a modest three-layer neural network ($784 \to 128 \to 64 \to 10$). The forward pass alone requires 109,184 MACs, 1,091.8× more than the rule-based approach. The 109,386 parameters consume 438 KB in FP32, exceeding most L1 caches and forcing memory traffic between cache levels on every inference. Training multiplies the cost further: each image must be processed *forward*, then *backward* (computing gradients for all 109,386 parameters), then updated, at roughly three times the forward cost per image, repeated over 60,000 training images for multiple epochs, meaning full passes through the training set. The computation is no longer sequential; it is dominated by dense matrix multiplications that leave a standard CPU mostly idle. This is the systems explosion that drives everything that follows.

The scaling advantage comes with computational costs that raise a practical question about when engineers should invest in neural networks vs. simpler alternatives.

> Systems Perspective 1.1: When to use neural networks
>
> Not every problem benefits from deep learning. Neural networks justify their computational cost when datasets exceed roughly 10,000 labeled examples, inputs carry hundreds of raw

features, data exhibits spatial, sequential, or hierarchical structure that architecture can exploit, and nonlinear relationships dominate the signal (table 1.1). Under the opposite regime—small samples, low-dimensional tabular data, linear relationships, or sub-millisecond latency budgets—classical methods like tree-based gradient boosting or logistic regression often match neural-network accuracy at a fraction of the compute (table 1.2).

Table 1.1: Neural-Network Candidates: Conditions under which deep learning is likely to outperform classical methods.

Condition	Threshold	Rationale
Dataset size	> 10,000 labeled examples	Below this, simpler models often match or exceed NN performance
Input dimensionality	> 100 raw features	NNs excel at automatic feature learning from high-dimensional data
Data has structure	Spatial, sequential, or hierarchical patterns	Architecture can encode inductive bias
Accuracy requirement	Need clear improvement over baseline	Late-stage gains often require disproportionate extra compute
Problem complexity	Nonlinear relationships dominate	Linear models handle linear relationships more efficiently

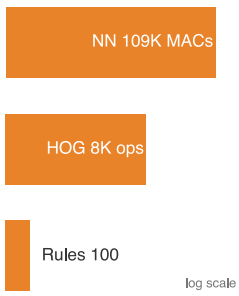
Table 1.2: Classical-Method Candidates: Conditions under which simpler models are likely to match or beat a neural network.

Condition	Better Alternative	Typical Outcome
< 1,000 samples	Logistic regression, Random Forest	10 ms training vs. hours; similar accuracy
Tabular data, < 100 features	Gradient Boosting (XGBoost, LightGBM)	Often matches NN accuracy with 100× less compute
Linear relationships	Linear/Ridge regression	Interpretable, fast, often better generalization
Real-time constraint < 0.1 ms	Rule-based system	Deterministic latency, no model loading overhead
Explainability required	Decision trees, linear models	Regulatory compliance, debugging clarity

Systems insight: Before building a neural network, train a logistic regression or gradient boosting model in < 1 hour. If it achieves > 90 percent of the target accuracy, the neural network's additional complexity may not be justified. The USPS system (section 1.6) succeeded partly because the problem genuinely required hierarchical feature learning that simpler methods could not provide.

1.2.4 Computational infrastructure requirements

The MNIST running example traced a single digit from ~100 comparisons (rule-based) through ~8,000 operations (HOG) to 109,184 MACs (neural network): a 1,091.8× escalation in computation, with a corresponding shift from predictable sequential access to bandwidth-hungry parallel matrix operations. Table 1.3 generalizes this pattern across every systems dimension.



Learned features buy accuracy by spending far more arithmetic.

Table 1.3: System Resource Evolution: Programming paradigms shift system demands from sequential computation to structured parallelism with feature engineering, and finally to massive matrix operations and complex memory hierarchies in deep learning. Deep learning reshapes system requirements compared to traditional programming and classical machine learning, impacting both computation and memory access patterns.

System Aspect	Traditional Programming	ML with Features	Deep Learning
Computation	Sequential, predictable paths	Structured parallel ops	Massive matrix parallelism
Memory Access	Small, predictable patterns	Medium, batch-oriented	Large, complex hierarchical

System Aspect	Traditional Programming	ML with Features	Deep Learning
Data Movement	Simple input/output flows	Structured batch processing	Intensive cross-system movement
Hardware Needs	CPU-centric	CPU with vector units	Specialized accelerators
Resource Scaling	Fixed requirements	Linear with data size	Formula-driven by width, batch, and state

The comparison matters because each step pushes the bottleneck outward: from branchy CPU control, to batch-oriented feature pipelines, to memory-fed matrix parallelism. This shift explains *why* conventional CPUs, designed for sequential processing, perform poorly for neural network computations.

The shift toward parallelism creates new bottlenecks that differ qualitatively from those in sequential computing. The central challenge is the memory wall[11]: while computational capacity can be increased by adding more processing units, memory bandwidth to feed those units does not scale as favorably. Matrix multiplication, the core neural network operation, is often limited by memory bandwidth rather than raw computational capability[12]—adding more processing units does not proportionally improve performance. Hardware responses to this challenge are examined in **?@sec-hardware-acceleration-understanding-ai-memory-wall-3ea9**, while **?@sec-appdx-machine-foundations-memory-hierarchy-2278** details the formal memory hierarchy with quantitative latency comparisons.

The deeper constraint is energy, not speed. Moving data from main memory to processing units can consume far more energy than the arithmetic itself (Horowitz 2014). This energy hierarchy explains why neural network accelerators focus on maximizing data reuse: keeping frequently accessed weights in fast local storage and carefully scheduling operations to minimize data movement. GPUs address this through both higher memory bandwidth and massive parallelism, but the underlying physics remains unchanged: data movement dominates computation cost, driving the adoption of specialized hardware architectures from data center GPUs to TinyML accelerators.

The memory-computation trade-off manifests differently across the cloud-to-edge spectrum introduced in **?@sec-ml-systems**. Cloud servers can afford more memory and power to maximize throughput, while mobile devices must carefully optimize to operate within strict power budgets. Training systems prioritize computational throughput even at higher energy costs, while inference systems emphasize energy efficiency. These different constraints drive different optimization strategies across the ML systems spectrum, ranging from memory-rich cloud deployments to heavily optimized TinyML implementations.

These single-machine constraints compound when scaling across multiple machines: dense layers scale with adjacent dimensions, batches scale activation storage and throughput demand, and training adds backward-pass, activation, and optimizer-state multipliers. Model-compression, hardware-acceleration, and training-system techniques all respond to this pressure by reducing the work, raising the useful machine rate, or changing where state lives.

The infrastructure demands traced earlier (massive parallelism, memory walls, energy-dominated data movement) arise from how neural networks compute: weights that change during training, many simple units operating simultaneously, layers that compose low-level features into high-level concepts, and data reuse that minimizes energy-intensive movement. These properties manifest concretely in the fundamental building block of neural computation: the artificial neuron[13]. Just as understanding a single transistor reveals how complex processors work, understanding the artificial neuron reveals how million-parameter networks operate.

1.2.5 The artificial neuron as a computing primitive

The basic unit of neural computation, the **artificial neuron** (or node), serves as a simplified mathematical abstraction of nervous activity (McCulloch and Pitts 1943). Later digital neural-network implementations adapted this abstraction into a standardized processing unit. This building block enables complex networks to emerge from simple components working together. Compare the biological and artificial neurons side by side in figure 1.8 to see how this computational model distills biological complexity into a simpler computational form.

11 | **Memory Wall**: Fast on-chip storage responds in nanoseconds, while larger off-chip memory is orders of magnitude farther away in latency and energy. Neural network weights rarely fit in the smallest caches (even our MNIST model is hundreds of KB in FP32), forcing repeated memory fetches that can leave compute units idle. This bandwidth bottleneck, not arithmetic capacity alone, is why accelerators invest die area in HBM and on-chip SRAM.

12 | **Memory-Bound Operations**: Matrix multiplication's arithmetic intensity (FLOP/byte loaded) determines whether a layer is compute bound or memory bound. Layers that fall below the hardware's roofline crossover point finish their arithmetic before the next tile of weights arrives from memory. The result: effective hardware utilization can drop sharply, and adding more compute units yields little speedup until memory bandwidth or data reuse improves.

13 | **Neuron**: McCulloch and Pitts (1943) introduced a mathematical threshold model of nervous activity: inputs combine and an all-or-none output fires when a threshold is met. That logical neuron is an origin point for the artificial-neuron abstraction and for networks built from many simple units. Learned weights, deep hierarchies, and modern fused multiply-add (FMA) accelerator datapaths are later developments that turn this abstraction into the matrix-heavy workloads studied in this chapter.

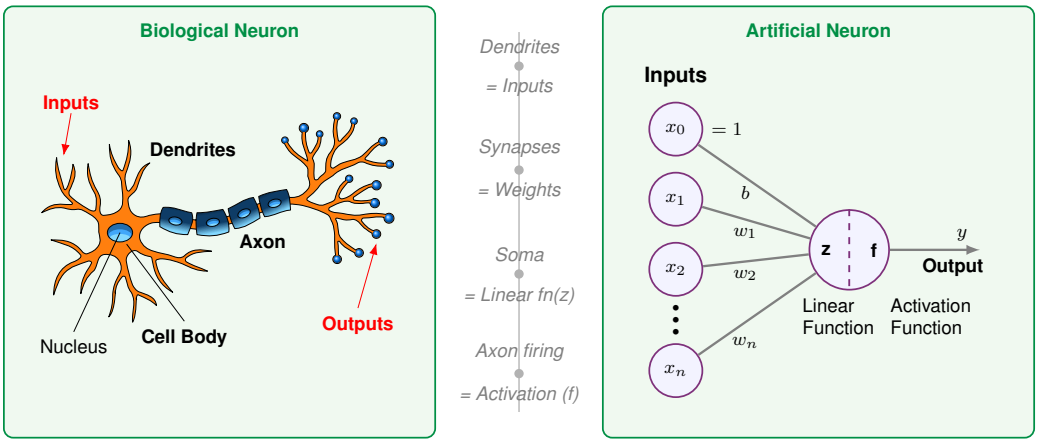


Figure 1.8: Biological-to-Artificial Neuron Mapping: Side-by-side comparison showing how biological neuron structures map to artificial neuron components. Dendrites correspond to inputs, synapses to weights, the cell body to the summation function, and the axon to the activation output. This mapping established the "Compute-Aggregate-Activate" pattern central to neural network design.

The mapping in figure 1.8 traces a signal through four stages, each translating a biological structure into a mathematical operation. Table 1.4 formalizes these correspondences.

The right panel of figure 1.8 traces this signal through four stages:

1. **Input Reception** (Dendrites $\rightarrow x_1, x_2, \ldots, x_n$): The neuron receives a vector of input features $\mathbf{x}$. In a system like MNIST digit recognition, these represent individual pixel intensities—the digital equivalent of signals arriving at a biological neuron's dendrites.
2. **Weighted Modulation** (Synapses $\rightarrow w_1, w_2, \ldots, w_n$): Each input is multiplied by a learnable weight w_i, just as synaptic strengths modulate biological signals. These weights act as "gain" controls, determining how much influence each feature has on the final decision. A bias term b (shown as the top input $x_0 = 1$ in figure 1.8) shifts the activation threshold. This is where the model's "knowledge" is stored.
3. **Signal Aggregation** (Cell Body $\rightarrow$ Linear Function z): The neuron integrates the weighted signals, producing a single scalar value $z = \sum(x_i \cdot w_i) + b$. This mirrors how a biological cell body sums incoming electrochemical signals to determine whether the neuron has received enough evidence for a particular pattern.
4. **Nonlinear Activation** (Axon $\rightarrow$ Activation Function f): The aggregated signal passes through an activation function $f(z)$, producing the output y. This mirrors the axon's all-or-nothing firing decision: the nonlinearity determines whether the neuron "fires" a signal to the next layer. Unlike the biological case, f can produce graded outputs (for example, ReLU passes positive values through, zeroes negatives), but the principle is the same—thresholding followed by propagation.

Table 1.4: Neuron Structure and Function: Each biological structure maps to a computational operation in the artificial neuron. Dendrites become input vectors, synaptic strengths become learnable weights, the cell body becomes a linear aggregation function z, and the axon's firing behavior becomes the nonlinear activation function f that produces the output y.

Biological Structure	Artificial Component	Mathematical Operation	Engineering Role
Dendrites (receive signals)	**Input Vector**	$\mathbf{x} = [x_1, \ldots, x_n]$	Data ingestion from sensors or prior layers
Synapses (modulate strength)	**Weight Vector**	$\mathbf{w} = [w_1, \ldots, w_n]$	Learnable parameters encoding importance
Cell Body (integrates signals)	**Linear Function** z	$z = \sum(x_i \cdot w_i) + b$	Linear integration of feature signals
Axon (fires output)	**Activation Function** f	$a = f(z)$	Nonlinear thresholding and signal propagation

From a systems engineering perspective, this translation reveals *why* neural networks have such demanding computational requirements. Each "simple" neuron requires N **multiply-accumulate (MAC)**[14] operations and $2N+2$ memory accesses (loading N inputs and N weights, plus the bias and output). When replicated millions of times across a network, these primitives create the massive arithmetic and bandwidth demands that define modern AI infrastructure.

The transition from individual neurons to integrated systems requires navigating the central trade-off between representational capacity and computational cost. While silicon transistors operate at gigahertz frequencies, millions of times faster than biological chemical signaling, the sheer volume of operations in deep networks creates unique bottlenecks.

Replicating intelligent behavior in silicon confronts three interrelated system-level constraints. The memory wall becomes acute as models grow to billions of parameters, making data movement the primary bottleneck rather than raw computation. Concurrency clashes with dependency: many operations inside a layer can run in parallel for throughput, but the sequential nature of deep networks (layer $\ell+1$ depends on layer ℓ) creates fundamental latency limits. Precision also trades against power: digital systems achieve high accuracy through precise 32-bit or 64-bit math, but each bit increases the energy cost of every operation, driving the search for minimum viable precision explored in **?@sec-model-compression**.

Addressing these constraints requires two complementary strategies. An architectural inductive bias is a built-in structural assumption about the data: convolutional networks assume nearby pixels matter together in images, while recurrent networks assume order matters in sequences. Encoding those assumptions directly into the network design reduces the search space the optimizer must navigate (Mitchell 1980). Computational scaling compensates for remaining complexity through brute-force optimization on massive hardware arrays. Modern AI engineering sits at the intersection of these two paths: clever architectures shrink the problem, and massive scale solves what remains.

1.2.6 Hardware and software requirements

Translating neural concepts to silicon carries a physical cost. Feature extraction becomes weighted linear sums, thresholding becomes nonlinear activation functions, and pattern interaction becomes fully connected layers, all implemented as matrix operations that modern hardware must execute efficiently. A single matrix multiplication in code translates to millions of transistors switching at high frequency, generating heat and consuming significant power. Each neural network operation creates a specific hardware demand: activation functions require fast nonlinear units, weight operations require high-bandwidth memory access, parallel computation requires specialized processors, and learning algorithms require gradient computation hardware. These demands interact: the sheer volume of weight parameters creates a storage problem, the need to move those weights to processing units creates a bandwidth problem, and the learning process compounds both by requiring space for gradients and optimizer state alongside the weights themselves.

A key difference from traditional computing is that neural network "memory" is *distributed* across all weights rather than *stored at specific addresses*. Every prediction requires reading a significant portion of the model's parameters, and every training step requires coordinating weight updates across the entire network. This creates a fundamental tension between storage capacity and access bandwidth that biological neural systems avoid (synapses both store and process locally). The human brain operates on approximately twenty watts (Raichle and Gusnard 2002); artificial neural networks demand orders of magnitude more energy, primarily because of this data movement overhead. This energy gap drives the specialized hardware architectures covered in **?@sec-hardware-acceleration** and the optimization strategies explored in **?@sec-model-compression**.

These hardware demands did not emerge overnight. The tension between algorithmic ambition and available silicon has shaped the entire trajectory of neural network research, from the earliest perceptrons to today's trillion-parameter models.

1.2.7 Evolution of neural network computing

The **Perceptron**[15] (Rosenblatt 1958) exposed the hardware-algorithm bargain from the start: the neuron mathematics was simple, but the available machines lacked the processing power and memory capacity needed for complex networks. Deep learning evolved through that co-evolution, with the neuron abstraction remaining recognizable while the silicon able to execute it transformed.

14 **MAC (Multiply-Accumulate)**: The atomic operation of neural computation: $a \leftarrow a + (b \times c)$. Hardware data sheets often report fused multiply-add throughput in FLOPs because one multiply and one add count as two floating-point operations. On that convention, the reported FLOP/s rate is twice the MAC/s rate when a MAC is counted as one multiply-accumulate. Every layer size and batch size decision ultimately reduces to how many MACs fit within the latency and power budget.

15 **Perceptron**: A machine built to execute a learning algorithm, directly linking hardware and software from the start. Its single-layer architecture was fundamentally constrained to linearly separable problems, a limitation Minsky and Papert later proved was algorithmic, not just computational. This early failure demonstrated that without sufficient model depth (that is, layers), even custom-built hardware with 400 photocell inputs was insufficient for complex tasks.

The **backpropagation**[16] algorithm was applied to neural networks by Paul Werbos in his 1974 PhD thesis (Werbos 1974), building on Seppo Linnainmaa's 1970 work on automatic differentiation (Linnainmaa 1970), and was later popularized by Rumelhart, Hinton, and Williams (Rumelhart et al. 1986). Their publication demonstrated the algorithm's practical effectiveness and brought it to widespread attention in the machine learning community, triggering renewed interest in neural networks. This chapter returns to the algorithm in section 1.4.4; **?@sec-model-training-backpropagation-mechanics-0b64** develops its systems-level implementation. Despite this breakthrough, the computational demands far exceeded available hardware capabilities. Training even modest networks could take weeks, making experimentation and practical applications challenging. This mismatch between algorithmic requirements and hardware capabilities contributed to a period of reduced interest in neural networks.

The historical trajectory demonstrates a recurring systems engineering lesson: an algorithm is only as effective as the hardware available to execute it. The decades-long gap between the mathematical formulation of backpropagation[17] and its widespread adoption was a latency in infrastructure, not a failure of theory. Efficient ML systems engineering requires co-designing algorithms and silicon together. The deep learning revolution was sparked by the convergence of data availability, algorithmic maturity, and the parallel processing power of GPUs, not by a new mathematical discovery alone.

The term itself gained prominence in the 2010s, coinciding with advances in computational power and data accessibility. The scale of this computational explosion is difficult to grasp without visualization. Figure 1.9 plots seven decades of AI training compute on a logarithmic scale, revealing two fitted regimes: total training compute, measured in floating-point operations (FLOPs), followed a comparatively slow pre-2010 trend, then the post-2012 deep-learning frontier accelerated sharply. Large-scale models after 2015 sit orders of magnitude above the pre-2010 trajectory, showing that modern progress reinvests hardware and algorithmic gains into much larger training runs.

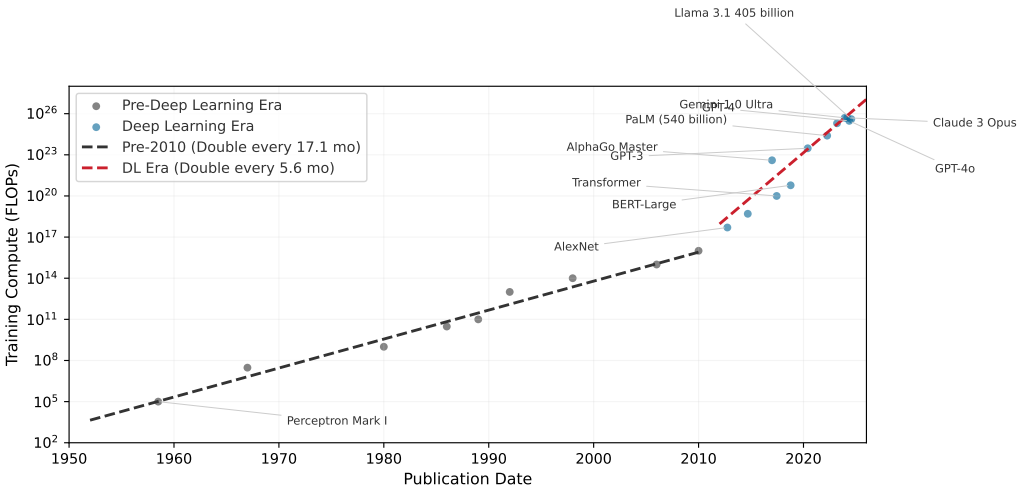


Figure 1.9: Computational Growth: Log-scale scatter plot showing total training compute in FLOPs across seven decades of AI models. Fitted trend lines show a slower pre-2010 regime followed by a much faster deep-learning frontier after 2012. Large-scale models after 2015 sit orders of magnitude above the pre-2010 trend, addressing the historical bottleneck of training complex neural networks.

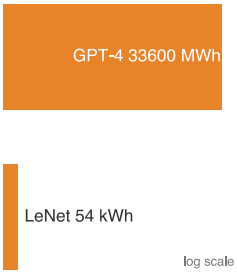
Beyond raw compute, this exponential growth carries an energy cost that systems engineers cannot ignore. Training LeNet-1 in 1989 consumed roughly 54 kWh, about a few days of household electricity. A GPT-4-scale training run, using public GPU-day estimates and a data center-overhead factor, lands around 33,600 MWh—enough to power roughly 3,200 US homes for a year[18]. The energy cost of AI has moved from negligible to industrial, forcing engineers to treat energy efficiency (J per operation) as a primary design constraint alongside achieved FLOP/s. The quantitative energy analysis, including Horowitz's data-movement-dominates-compute numbers and the full energy

16 **Backpropagation**: Short for "backward propagation of errors," the algorithm solves the credit assignment problem by determining which of millions of weights caused a given error, using the chain rule. Werbos applied it to neural networks in 1974 (Werbos 1974), and the 1986 Rumelhart, Hinton, and Williams publication demonstrated practical effectiveness (Rumelhart et al. 1986). The systems cost: backprop requires storing forward-pass activations and additional training state, creating the several-times-higher memory footprint quantified later in the chapter.

17 **Algorithm-Hardware Adoption Lag**: Backpropagation was available in neural-network form by 1974 (Werbos 1974) but not widely adopted until after the 1986 Rumelhart, Hinton, and Williams demonstration (Rumelhart et al. 1986), a gap explained partly by insufficient compute: training a meaningful network required hardware that did not exist. The pattern recurs more softly with attention, a later sequence-modeling mechanism that scores how strongly one element should use information from another: attention mechanisms were introduced for neural machine translation in 2014, and transformers made them central in 2017; GPUs were sufficient for the original Transformer experiments, while later TPU- and GPU-scale infrastructure enabled much larger deployments. The implication is that apparently "failed" algorithms may simply be hardware-premature. When evaluating today's computationally intractable techniques, the right diagnostic is not whether the algorithm works on current hardware but what hardware regime would make it work.

hierarchy, appears in **?@sec-hardware-acceleration** where it can be connected to concrete hardware architectures.

Table 1.5 grounds these trends in concrete systems, showing how parameters, compute, and hardware co-evolved across four decades of neural network development. Three quantitative patterns emerge from this historical data. The plotted post-2012 training-compute frontier doubles on the order of months, while broader summaries that smooth across model families and account for reporting uncertainty show similarly rapid annual growth. Separately, the compute required to achieve a fixed benchmark has improved substantially due to algorithmic and systems advances. Training *costs* grow more slowly than *raw compute* because hardware utilization, reduced precision, and software efficiency also improve. Frontier model training costs have nonetheless moved from workstation-scale budgets into industrial-scale investments. These patterns have direct implications for systems engineering: compute scaling determines infrastructure investment timelines, algorithmic efficiency justifies continuous architecture research, and the cost-compute gap shapes build-vs.-buy decisions for ML teams.



Neural-network history is a scale explosion in training energy.

18 **Training Energy Scale**: This estimate uses 10.5 MWh/year as a representative US household electricity budget and treats GPT-4's public GPU-day estimate as A100-equivalent accelerator time with data center overhead. The point is order of magnitude, not an audited utility bill: a single frontier training run now rivals a small industrial facility's energy budget, making J-per-operation a first-order design constraint alongside achieved FLOP/s.

Table 1.5: Historical Performance: Four decades of neural network evolution showing the co-scaling of model parameters, training compute, and hardware infrastructure. Training FLOPs increased by approximately $10^{13}\times$ from LeNet-1 to GPT-4, while parameters grew by $10^{8}\times$. Uncertainty notes: Earlier systems (LeNet, AlexNet) have well-documented specifications; recent closed models (GPT-4) have only external estimates (OpenAI has not officially confirmed GPT-4's architecture or parameter count; the ~1.8T estimate for a mixture-of-experts (MoE) design, where only a subset of parameters activates per input, is based on public reporting and analysis). "OoM" = order of magnitude uncertainty.

Year	System	Params	Train FLOPs	Hardware	Train Time	Error/Task
1989	LeNet-1	~10K	10^{11}–10^{12}	Sun-4/260 workstation	3 days	1% (USPS)
1998	LeNet-5	$60\text{K} \pm 1\text{K}$	$10^{14} \pm 1$ OoM	SGI Origin 2000 (200 MHz)	2–3 days	0.95% (MNIST)
2012	AlexNet	~60M	5×10^{17}	2× GTX 580 GPUs	5–6 days	15.3% (ImageNet)
2015	ResNet-152	~60M	$10^{19} \pm 0.5$ OoM	8× Tesla K80 GPUs	~3 weeks	3.6% (ImageNet)
2020	GPT-3	175B (exact)	3×10^{23}	~10K V100 GPUs	weeks	N/A (language)
2023	GPT-4	~1.8T (MoE, est.)	10^{24}–10^{25}	10–25K A100s (est.)	months	N/A (language)

Parallel advances across three dimensions drove these evolutionary trends: data availability, algorithmic innovations, and computing infrastructure. Follow the arrows in figure 1.10 to see this reinforcing cycle in motion: faster computing infrastructure enabled processing larger datasets, larger datasets drove algorithmic innovations, and better algorithms demanded more sophisticated computing systems. This reinforcing cycle continues to drive progress today.

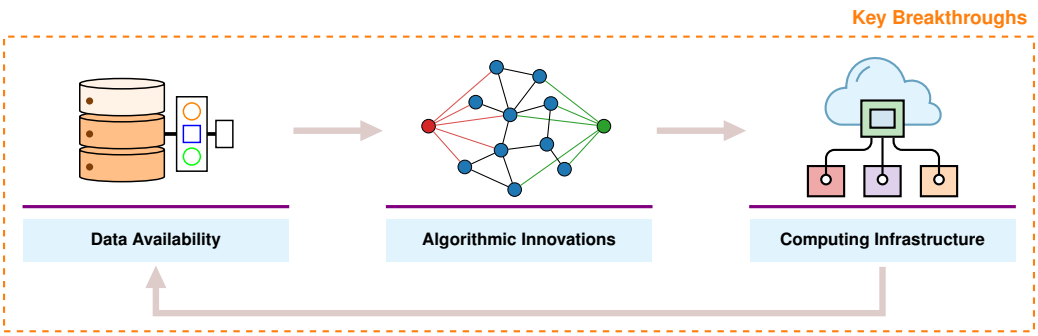


Figure 1.10: Deep Learning Virtuous Cycle: Three mutually reinforcing factors, data availability, algorithmic innovations, and computing infrastructure, form a self-reinforcing loop where breakthroughs in one area create opportunities in the others.

Data availability supplied the volume of examples that learned representations require. The rise of the internet and digital devices created vast new sources of training data: image sharing platforms provided millions of labeled images, digital text collections enabled language processing at scale,

and sensor networks generated continuous streams of real-world data. This abundance provided the raw material neural networks needed to learn complex patterns effectively.

Algorithmic innovations turned that volume into trainable systems. New methods for initializing networks and controlling learning rates made training more stable. Techniques for preventing overfitting allowed models to generalize better to new data. Researchers discovered that neural network performance scaled predictably with model size, computation, and data quantity, leading to increasingly ambitious architectures.

The resulting workloads created demand for higher-throughput computing infrastructure, which evolved in response. On the hardware side, GPUs provided the parallel processing capabilities needed for efficient neural network computation, and specialized AI accelerators like TPUs[19] (Jouppi et al. 2023) pushed performance further. High-bandwidth memory systems and fast interconnects addressed data movement challenges. Software advances matched the hardware evolution: frameworks and libraries simplified building and training networks, distributed computing systems enabled training at scale, and tools for optimizing model deployment reduced the gap between research and production.

The convergence of data availability, algorithmic innovation, and computational infrastructure created the foundation for modern deep learning. The following checkpoint consolidates this arc before the chapter returns to the computational operations that drive it.

> **Checkpoint 1.1: Understanding deep learning's emergence**
>
> Before proceeding to the mathematical foundations, verify your understanding of why deep learning emerged:
>
> - ☐ **Rule-based limits**: Can you explain why rule-based programming fails for tasks like image recognition?
> - ☐ **Classical ML vs. deep learning**: Do you understand the difference between classical ML (feature engineering) and deep learning (automatic feature learning)?
> - ☐ **Three enabling factors**: Can you describe the three factors that converged to enable modern deep learning (data, algorithms, infrastructure)?
> - ☐ **Hardware demand**: Do you understand why deep learning requires specialized hardware compared to traditional programming?
>
> If any of these concepts remain unclear, review the relevant sections before continuing. The mathematical details that follow build directly on this conceptual foundation.

The historical trajectory from Perceptrons through AI winters to the GPU-driven revolution reveals a recurring pattern: algorithms outpace hardware, creating latency between discovery and adoption, until infrastructure catches up and triggers an explosion of capability. This pattern continues today as frontier models push against memory walls and energy budgets. Understanding the mathematical operations that create these pressures is essential for navigating the next cycle—which requires examining the computational primitives themselves.

1.3 Neural Network Fundamentals

The question now is *why* the computational demands are so extreme. A GPU processes neural networks faster than a CPU not because of raw clock speed but because of the specific mathematical operations neural networks perform. Training requires more memory than inference not because of software overhead but because the chain rule demands storing every intermediate result. Understanding these operations reveals how simple arithmetic on individual neurons compounds into the infrastructure requirements that shaped modern AI.

The concepts here apply to all neural networks, from simple classifiers to large language models. While architectures evolve and new paradigms emerge, these fundamentals remain constant: weighted sums, nonlinear activations, gradient-based learning. Mastering these operations and their computational characteristics enables reasoning about any neural network's resource requirements.

[19] **Tensor Processing Unit (TPU)**: Google's custom accelerator, first deployed internally in 2015, was optimized specifically for the matrix multiplications that dominate neural network workloads. The TPU v1 achieved 92 TOPS (vendor-reported INT8 tera-operations/s) for inference at 75 W, a power-efficiency point that general-purpose GPUs of the era could not match. The name "Tensor Processing Unit" reflects the design decision to sacrifice general-purpose flexibility for maximum throughput on the operation neural networks need most.

1.3.1 Why depth matters: The power of hierarchical representations

A single-layer network attempting to classify handwritten digits must map raw pixels directly to labels, essentially memorizing every variation of every digit. A network with three layers solves the same problem with far fewer parameters by decomposing it hierarchically. The question is *why* depth provides such dramatic representational advantages, and the answer grounds all the mathematical development that follows.

Deep networks succeed because they use **compositionality**: complex patterns decompose into simpler patterns that themselves decompose further. In image recognition, pixels combine into edges, edges into textures, textures into parts, and parts into objects. This hierarchical decomposition reflects the structure of the world itself and explains why "deep" learning earns its name.

Consider recognizing the digit "seven" in our MNIST example. A single-layer network would need to directly map all 784 pixel values to a decision, essentially memorizing every variation of how people write "seven." A deep network instead decomposes the digit hierarchically, each layer building on the previous:

- **Layer 1** learns simple edge detectors: vertical lines, horizontal lines, diagonal strokes
- **Layer 2** combines edges into shapes: the horizontal top stroke of a "seven," the diagonal downstroke
- **Layer 3** combines shapes into complete digit patterns

Each layer builds on the previous, exponentially expanding representational capacity with only linear parameter growth. This hierarchy enables efficiency that shallow networks cannot match. The same edge detectors learned for "seven" also detect edges in "one," "four," and every other digit. This **parameter reuse** means a deep network with 100K parameters can represent patterns that would require millions of parameters in a shallow network attempting direct pixel-to-label mapping. However, the choice between adding layers and widening existing ones is not symmetric: depth and width contribute to representational capacity through very different mechanisms, with consequences that compound rather than cancel.

> Systems Perspective 1.2: The depth vs. width trade-off
>
> The theoretical power of depth comes from the **exponential advantage**: for certain function classes, a network with N_L layers can represent functions that would require exponentially more neurons in a single-layer network (Telgarsky 2016). Composing nonlinear layers enables exponentially more complex decision boundaries with only linearly more parameters.
>
> However, depth introduces three engineering challenges with each additional layer:
>
> - Adds sequential dependencies (layer $\ell + 1$ waits for layer ℓ), limiting parallelism
> - Increases gradient path length, risking vanishing/exploding gradients
> - Requires storing intermediate activations for backpropagation
>
> Modern architectures balance depth (representational power) against width (parallelism). A network with ten layers of 100 neurons has the same 1,000 total hidden neurons as one with two layers of 500 neurons, but fundamentally different computational characteristics. The deeper network can represent more complex functions; the wider network can compute all neurons in a layer simultaneously.

Biological visual systems employ the same hierarchical decomposition, and the specific architectures examined in **?@sec-network-architectures** formalize different ways to encode this hierarchical structure for images, sequences, and other data types. Depth explains why layered representations are useful; the remaining mechanics explain how the hierarchy is implemented. The following sections develop those mechanics: how neurons compute, how layers connect, and how information flows from input to output.

1.3.2 Network architecture fundamentals

A neural network's architecture determines how information flows from input to output. Modern networks can be enormously complex, but they all build on a few organizational principles that shape both implementation and the computational infrastructure they demand.

To ground these concepts in a concrete example, we use handwritten digit recognition throughout this section, specifically the task of classifying images from the MNIST dataset (LeCun et al. 1998). This seemingly simple task reveals all the core principles of neural networks while providing intuition for more complex applications.

Example 1.1: Running example: MNIST digit recognition

Objective: Given a 28×28 pixel grayscale image of a handwritten digit, classify it as one of the 10 digits (0–9).

Input representation: Each image contains 784 pixels (28×28), with values ranging from 0 (white) to 255 (black). We normalize these to the range [0,1] by dividing by 255. When fed to a neural network, these 784 values form our input vector $\mathbf{x} \in \mathbb{R}^{784}$.

Output representation: The network produces 10 values, one for each possible digit. These values represent the network's confidence that the input image contains each digit. The digit with the highest confidence becomes the prediction.

Rationale: MNIST is small enough to understand completely (784 inputs, ~100K parameters for a simple network) yet large enough to be realistic. The task is intuitive: everyone understands what "recognize a handwritten seven" means, making it ideal for learning neural network principles that scale to much larger problems.

Architecture preview: A typical MNIST classifier might use: 784 input neurons (one per pixel) $\rightarrow$ 128 hidden neurons $\rightarrow$ 64 hidden neurons $\rightarrow$ 10 output neurons (one per digit class). As we develop concepts, we will reference this specific architecture.

Systems insight: MNIST is useful because its data representation, model size, and output semantics are all small enough to inspect directly. The same representation-to-computation path scales to larger networks, where the dimensions rather than the logic create the systems challenge.

Each architectural choice, from how neurons are connected to how layers are organized, creates specific computational patterns that must be efficiently mapped to hardware. This mapping between network architecture and computational requirements is essential for building scalable ML systems.

1.3.2.1 The perceptron's weighted sum

The computational machinery within each layer is the artificial neuron, or perceptron, whose signal path (inputs, weights, bias, aggregation, activation) section 1.2.5 traced through its biological origins. What remains is to formalize that path mathematically, because the formal version is what hardware executes millions of times per prediction. In the MNIST network, a single first-layer neuron must combine all 784 pixel intensities into one output that signals whether its learned pattern, perhaps a vertical edge shared by "one" and "seven," is present. Follow the signal path through figure 1.11 to see how weighted inputs combine with activation functions to produce a decision: each input x_i multiplies by its corresponding weight w_{ij}, the products sum with a bias term, and the activation function produces the final output.

Each input x_i has a corresponding weight w_{ij}, and the perceptron multiplies each input by its matching weight. The intermediate output, z, is computed as the weighted sum of inputs in equation 1.1:

$$z = \sum (x_i \cdot w_{ij}) \tag{1.1}$$

In plain terms, each input feature is scaled by how important it is (its weight), and the results are summed into a single score. This is the dot product of two vectors—the fundamental operation that hardware accelerators are designed to execute at maximum throughput, and the reason neural network performance is measured in multiply-accumulate (MAC) operations per second.

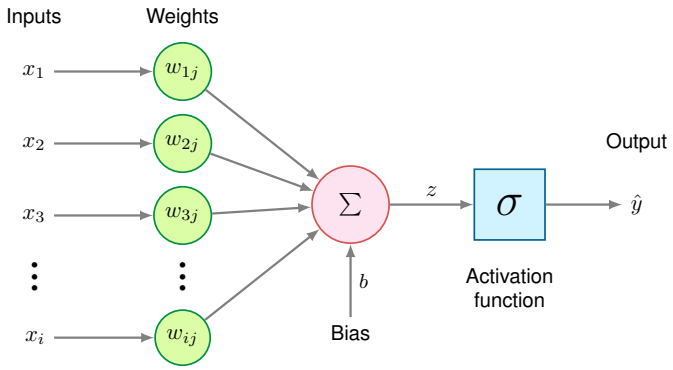


Figure 1.11: Perceptron Architecture: The fundamental computational unit of neural networks, showing inputs multiplied by weights, summed with bias, and passed through an activation function to produce output.

A bias term b shifts the linear output up or down, giving the model additional flexibility to fit the data. Thus, the intermediate linear combination computed by the perceptron including the bias becomes equation 1.2:

$$z = \sum(x_i \cdot w_{ij}) + b \tag{1.2}$$

With this weighted sum in hand, a perceptron can serve either regression or classification: regression uses the activated output $\hat{y}$ directly, while classification thresholds it—above the threshold, one class; below it, another.

The per-neuron cost is the one established earlier: N multiply-accumulate operations and $2N + 2$ memory accesses. What the formalization adds is the layer view. Perceptrons work in concert, each layer's output feeding the next, and a layer of M neurons repeats the weighted sum M times, so the layer's total cost is $M \times N$ MACs—exactly the matrix multiplication $\mathbf{xW}$ that hardware must execute, and the form in which the rest of this chapter counts work.

1.3.2.2 Nonlinear activation functions

Activation functions are where expressiveness, gradient flow, and hardware cost meet. They convert linear weighted sums into nonlinear outputs; without them, multiple linear layers would collapse into a single linear transformation, severely limiting the network's expressive power. Figure 1.12 compares three commonly used element-wise activation functions and one vector-level function (softmax), each with mathematical characteristics that shape both learning behavior and execution cost.

The choice of activation function affects both learning effectiveness and computational efficiency, and the history of that choice reveals why systems constraints shape algorithmic design. ReLU ($\max(0, x)$) became the default hidden-layer activation for many feed-forward networks, and it remains a useful baseline for good reason: it is computationally trivial (a single comparison), its gradient never vanishes for positive inputs, and it introduces natural sparsity. ReLU's dominance, however, only makes sense against the backdrop of what came before. The earliest networks used sigmoid and tanh activations, whose smooth S-curves seemed mathematically elegant but created a systems nightmare: gradients that shrank exponentially through deep layers, killing learning before it could begin. Understanding *why* sigmoid and tanh fail in deep networks is essential for understanding *why* ReLU succeeded and what its own limitations imply for later architectures.

Sigmoid The sigmoid function[20] maps any input value to a bounded range between 0 and 1, as defined in equation 1.3:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1.3}$$

The S-shaped curve produces outputs interpretable as probabilities, making sigmoid particularly useful for binary classification tasks. For large positive inputs, the function approaches one; for large negative inputs, it approaches 0. The smooth, continuous nature of sigmoid makes it differentiable everywhere, which is necessary for gradient-based learning.

20 **Sigmoid**: From Greek *sigma* + *eidos* ("sigma-shaped"), referring to the S-curve that maps inputs to the bounded (0, 1) range. The mapping requires a floating-point exponential (e^{-x}), which is substantially more expensive than ReLU's comparator-style operation. This arithmetic penalty scales with every neuron in every layer, making sigmoid's replacement by ReLU as much a hardware efficiency decision as a gradient stability one.

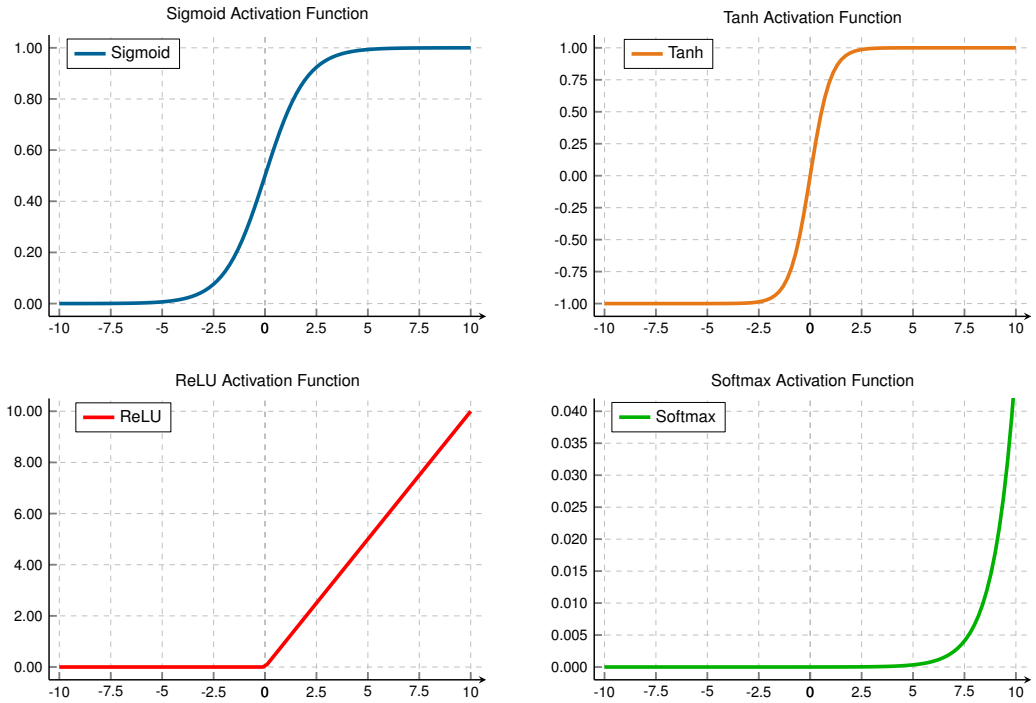


Figure 1.12: Common Activation Functions: Four nonlinear activation functions plotted with their output ranges. Sigmoid maps inputs to $(0, 1)$ with smooth gradients, tanh provides zero-centered outputs in $(-1, 1)$, ReLU introduces sparsity by outputting zero for negative inputs, and softmax (bottom-right) shows one component of a 3-element vector, illustrating how a single logit's probability varies as it changes relative to the other elements. The softmax panel uses a magnified y-axis (roughly 0 to 0.04) that is not comparable to the other three panels: it plots a single output component, not softmax's full vector-level behavior, where all K outputs are interdependent and sum to 1.

Sigmoid has a significant limitation: for inputs with large absolute values (far from zero), the gradient becomes extremely small, a phenomenon called the **vanishing gradient problem**[21]. During backpropagation, these small gradients multiply together across layers, causing gradients in early layers to become exponentially tiny. This effectively prevents learning in deep networks, as weight updates become negligible.

Sigmoid outputs are not zero-centered (all outputs are positive). This asymmetry can cause inefficient weight updates during optimization, as gradients for weights connected to sigmoid units will all have the same sign.

Tanh The hyperbolic tangent function[22] addresses sigmoid's zero-centering limitation by mapping inputs to the range $(-1, 1)$, as defined in equation 1.4:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{1.4}$$

Tanh produces an S-shaped curve similar to sigmoid but centered at zero: negative inputs map to negative outputs and positive inputs to positive outputs. This symmetry balances gradient flow during training, often yielding faster convergence than sigmoid.

Like sigmoid, tanh is smooth and differentiable everywhere, and it still suffers from the vanishing gradient problem for inputs with large magnitudes. When the function saturates (approaches -1 or 1), gradients shrink toward zero. Despite this limitation, tanh's zero-centered outputs make it preferable to sigmoid for hidden layers in many architectures, particularly in recurrent neural networks where maintaining balanced activations across time steps is important.

Both sigmoid and tanh share a critical limitation: gradient saturation at extreme input values. The search for an activation function that avoids this problem while remaining computationally efficient led to one of deep learning's most important innovations.

21 **Vanishing Gradient Problem**: The chain rule's multiplication of gradients across layers causes this failure mode when using activations like sigmoid, whose derivative is always less than 1. With sigmoid's maximum derivative of 0.25, the gradient in a 10-layer network shrinks by a factor of nearly one million ($0.25^{10} \approx 10^{-6}$), preventing weights in early layers from updating.

22 **Tanh (Hyperbolic Tangent)**: By centering its output range on zero, `tanh` allows weight gradients to be both positive and negative, fixing the all-positive update bias that slows sigmoid-based training. Its computational cost is similar to sigmoid because it also depends on exponentials, but the zero-centered output makes optimization better behaved than sigmoid in hidden layers.

ReLU The Rectified Linear Unit (ReLU)[23] function was known for decades before deep learning, but Nair and Hinton (2010) demonstrated that ReLU enabled more effective training of deep networks. Combined with GPU computing, dropout[24], and other innovations, ReLU helped enable the AlexNet breakthrough in 2012 (Krizhevsky et al. 2012). The ReLU function is defined in equation 1.5:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{1.5}$$

ReLU's characteristic shape—a straight line for positive inputs and zero for negative inputs—provides three advantages that explain its dominance. First, gradient flow remains intact for positive inputs: ReLU's gradient is one, allowing gradients to propagate without the saturation that plagues sigmoid and tanh in deep architectures. Second, ReLU introduces natural sparsity by zeroing negative activations, so part of the activation vector can become inactive for any given input. Third, computational efficiency improves because ReLU is computed with a simple comparison, `output = (input > 0) ? input : 0`, rather than an exponential.

ReLU is not without drawbacks. The **dying ReLU problem**—neurons that permanently output zero and cease learning—occurs when neurons become stuck in the inactive state. If a neuron's weights evolve during training such that the preactivation $z = \mathbf{w}^T\mathbf{x} + b$ is consistently negative across all training examples, the neuron outputs zero for every input. Since ReLU's gradient is also zero for negative inputs, no gradient flows back through this neuron during backpropagation: the weights cannot update, and the neuron remains dead. This can happen with large learning rates that push weights into unfavorable regions. From a systems perspective, dead neurons represent wasted capacity: parameters that consume memory and compute during inference but contribute nothing to the output. Careful initialization (He et al. 2015), moderate learning rates, and architectural choices (leaky ReLU variants or batch normalization[25] (Ioffe and Szegedy 2015)) help mitigate this issue.

Softmax Unlike element-wise activation functions that operate independently on each value, softmax[26] is a *vector-level* function: it considers all values simultaneously to produce a probability distribution. In simple classifiers, softmax is commonly used in the output layer rather than as an element-wise hidden activation; it also normalizes learned importance-score vectors in attention architectures. The softmax function is defined in equation 1.6:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1.6}$$

For a vector of K values (often called logits[27]), softmax transforms them into K probabilities that sum to 1. One component of the softmax output appears in figure 1.12 (bottom-right); in practice, softmax processes entire vectors where each element's output depends on all input values.

In multi-class classifiers, softmax is usually used in the output layer. By converting arbitrary real-valued logits into probabilities, softmax enables the network to express confidence across multiple classes. The class with the highest probability becomes the predicted class. The exponential function ensures that larger logits receive disproportionately higher probabilities, creating clear distinctions between classes when the network is confident.

The mathematical relationship between input logits and output probabilities is differentiable, allowing gradients to flow back through softmax during training. When combined with cross-entropy loss (discussed in section 1.4.3), softmax produces particularly clean gradient expressions that guide learning effectively. Beyond their mathematical properties, the choice of activation functions has direct consequences for hardware efficiency.

> Systems Perspective 1.3: Activation functions and hardware
>
> Beyond its mathematical benefits like avoiding vanishing gradients, ReLU's hardware efficiency explains its widespread adoption. Computing $\max(0, x)$ requires a single comparison operation, while sigmoid and tanh require computing exponentials—operations that are much more expensive in both time and energy. The transistor-tax calculation below models this

23 **ReLU (Rectified Linear Unit)**: Unlike the costly exponential operations in prior activation functions, ReLU's `max(0, x)` operation maps to a simple comparison and selection. This efficiency, together with better gradient flow for positive activations, helped make the deep architectures of the AlexNet era computationally tractable (Nair and Hinton 2010; Krizhevsky et al. 2012).

24 **Dropout**: Randomly deactivating neurons during training forces a network to learn redundant representations, a regularization technique used in the AlexNet-era shift toward deep vision models (Srivastava et al. 2014; Krizhevsky et al. 2012). This creates a systems-level divergence between the computational graphs for training (stochastic) and inference (deterministic). Failing to switch from the training to the inference graph is a common bug that can silently degrade accuracy.

25 **Batch Normalization Systems Cost**: BatchNorm adds two learned parameters per feature (scale γ and shift β) and behaves differently during training and inference: training uses live mean and variance from the mini-batch, while inference uses frozen running statistics. By keeping activation distributions better scaled, it can reduce dead ReLUs, but it also makes the layer depend on batch statistics. Small batches can produce noisy mean and variance estimates, so a mathematical stabilizer becomes a systems choice about batch size, activation memory, and training-serving parity. Later architectures and large-scale training settings introduce further variants of the same batch-statistics trade-off.

gap as a logic-cost ratio, and **?@sec-hardware-acceleration** covers the broader benchmark and accelerator-implementation implications.

1.3.2.3 The transistor tax: Logic unit cost

The activation decision has a second cost beyond gradient behavior: silicon area. In computer architecture, we measure the **Logic Unit Cost** in terms of transistor count and energy per operation.

A ReLU unit is computationally trivial: it consists of a single comparator and a multiplexer, requiring approximately 50 transistors. In contrast, a Sigmoid or Tanh unit requires computing an exponential—a complex transcendental function that hardware must approximate using lookup tables or iterative Taylor expansions. A high-precision exponential unit can consume over 2,500 transistors.

We call this disparity **The Transistor Tax**: selecting Sigmoid over ReLU increases the silicon "price" of an activation by 50×. For a systems engineer, this means ReLU is a density optimization that allows hardware architects to pack orders of magnitude more neurons into the same power and area budget. This physical efficiency is the primary reason the deep learning era shifted away from the "biologically plausible" Sigmoid toward the "silicon-efficient" ReLU.

These nonlinear transformations convert the linear input sum into a nonlinear output, giving us the complete perceptron computation in equation 1.7:

$$\hat{y} = \sigma(z) = \sigma\left(\sum(x_i \cdot w_{ij}) + b\right) \tag{1.7}$$

This nonlinearity is what makes deep networks expressive. Without it, stacking multiple layers would be pointless—a composition of linear functions is still linear. Compare the two panels in figure 1.13 to see this principle in action: the left panel exposes a linear decision boundary that fails to separate the two classes (no amount of linear layers would help), while the right panel reveals how nonlinear activation functions enable the network to learn a curved boundary that correctly classifies the data.

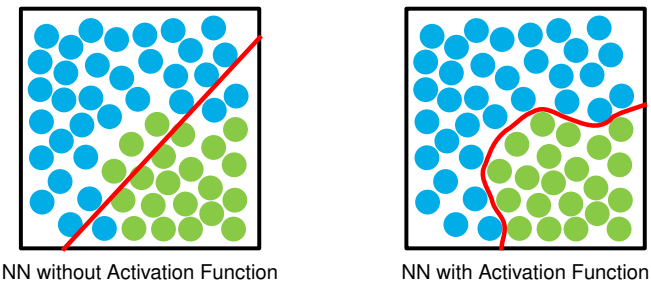
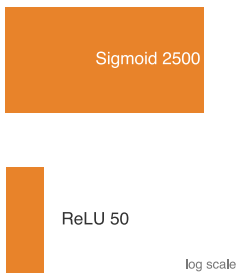


Figure 1.13: Linear vs. Nonlinear Decision Boundaries: Two scatter plots compare classification with and without activation functions. Without activation, a straight line fails to separate the two classes. With a nonlinear activation function applied, the network produces a curved decision boundary that correctly separates the points.

The **universal approximation theorem**[28] establishes that neural networks with activation functions can approximate arbitrary functions. This theoretical foundation, combined with the computational and optimization characteristics of specific activation functions like ReLU and sigmoid, explains neural networks' practical effectiveness in complex tasks. The theorem, however, states a pure existence result under the assumption of unlimited width; it says nothing about the accelerator executing the computation. Physical ML systems impose hard limits on both dimensions of the width-versus-depth trade-off: a single hidden layer wide enough to approximate a complex function requires parameter storage and activation memory that can exhaust accelerator VRAM entirely, and a network too deep to fit its activations in memory stalls the backward pass on the same constraint. The engineering problem is therefore not whether a network of sufficient width or depth *exists* but whether it fits within the memory and bandwidth envelope of the target hardware.

[26] **Softmax**: The name reflects its role as a "soft" or differentiable version of `argmax`, a function that must evaluate an entire vector to find its maximum value. This vector-wise operation is not used as a drop-in replacement for element-wise nonlinearities such as ReLU, but it is central whenever a model must normalize a vector of scores, including classification heads and attention layers. Critically, its use of exponentiation creates a numerical stability hazard: inputs greater than ~88 will overflow standard 32-bit floats, a common source of silent `NaN` failures in production.

[27] **Logits (Log-Odds Units)**: Short for "log-odds units," these raw scores preserve the relative ordering of class evidence before softmax normalizes them into probabilities. Because `argmax` over logits and `argmax` over softmax probabilities always select the same class, optimized inference pipelines skip the softmax computation entirely when only the top prediction is needed, saving K exponentiations per sample.



ReLU's comparator logic is far cheaper in silicon than a sigmoid exponential.

1.3.2.4 Layers and connections

Individual neurons compute weighted sums, apply bias terms, and pass results through activation functions. The power of neural networks, however, comes from organizing these neurons into layers. A layer is a collection of neurons that process information in parallel. Each neuron in a layer operates independently on the same input but with its own set of weights and bias, allowing the layer to learn different features from the same input data.

In a typical neural network, three layer types form the hierarchy:

1. **Input Layer**: Receives the raw data features
2. **Hidden Layers**: Process and transform the data through multiple stages
3. **Output Layer**: Produces the final prediction or decision

Follow the data flow in figure 1.14 from left to right: data enters at the input layer, passes through multiple hidden layers that progressively extract more abstract features, and emerges at the output layer as a prediction. Each successive layer transforms the representation, building increasingly complex features—a hierarchical processing pipeline that gives deep neural networks their ability to learn complex patterns.

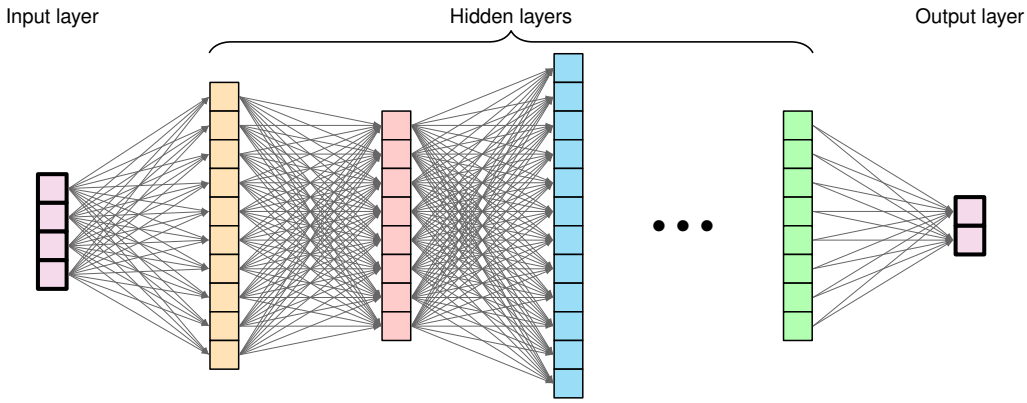


Figure 1.14: Layered Network Architecture: Deep neural networks transform data through successive layers, enabling the extraction of increasingly complex features and patterns. Each layer applies nonlinear transformations to the outputs of the previous layer, ultimately mapping raw inputs to desired outputs.

As data flows through the network, it is transformed at each layer to extract meaningful patterns. The weighted summation and activation process we established for individual neurons scales up: each layer applies these operations in parallel across all its neurons, with outputs from one layer becoming inputs to the next. This creates a hierarchical pipeline where simple features detected in early layers combine into increasingly complex patterns in deeper layers—enabling neural networks to learn sophisticated representations from raw data.

1.3.3 Parameters and connections

The learnable parameters[29] of neural networks, weights and biases, determine how information flows through the network and how transformations are applied to input data. Their organization directly impacts both learning capacity and computational requirements.

1.3.3.1 Weight matrices

Weights determine how strongly inputs influence neuron outputs. In larger networks, these organize into matrices for efficient computation across layers. In a layer with n input features and m neurons, the weights form a matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$, where each column represents the weights for a single neuron. This organization allows the network to process multiple inputs simultaneously, an essential feature for handling real-world data efficiently.

Recall that for a single neuron, we computed $z = \sum_{i=1}^{n}(x_i \cdot w_{ij}) + b$. When we have a layer of m neurons, we could compute each neuron's output separately, but matrix operations provide a much

28 **Universal Approximation Theorem**: The theorem guarantees a single hidden layer can approximate any function, but it is nonconstructive—it does not say how to find the weights. The critical flaw for practical effectiveness is that the required number of neurons in this layer can grow exponentially, making the network untrainable. This is why depth matters: deep networks trade this exponential width for polynomial depth, achieving the same approximation with exponentially fewer parameters.

29 **Parameter Memory Cost**: Parameter count is a misleading proxy for memory importance. Some layers add small learned scale or shift parameters that barely affect byte budgets but can strongly affect model behavior when training choices change. Conversely, the bulk of parameters in dense weight matrices require extra state during training: gradients plus optimizer bookkeeping beyond the stored weights themselves. A model that fits in memory for inference may therefore require several times more memory for training, a cost quantified later in the training memory budget.

more efficient approach. Rather than computing each neuron individually, matrix multiplication enables us to compute all m outputs simultaneously, as shown in equation 1.8:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} \tag{1.8}$$

This single equation computes every neuron's output in one operation: the input vector $\mathbf{x}$ multiplied by the weight matrix $\mathbf{W}$ produces all m preactivation values simultaneously, and the bias vector $\mathbf{b}$ shifts each one. From a systems perspective, this is the operation that dominates neural network runtime—a matrix-vector multiply whose dimensions (n inputs $\times$ m neurons) determine whether the layer is compute bound or memory bound on the target hardware.

This matrix organization is more than mathematical convenience; it reflects how modern neural networks are implemented for efficiency. Each weight w_{ij} represents the strength of the connection between input feature i and neuron j in the layer.

In the simplest and most common case, each neuron in a layer is connected to every neuron in the previous layer, forming what we call a "dense" or "fully-connected" layer. This pattern means that each neuron has the opportunity to learn from all available features from the previous layer. Fully-connected layers establish foundational principles, but alternative connectivity patterns (explored in **?@sec-network-architectures**) can dramatically improve efficiency for structured data by restricting connections based on problem characteristics.

Figure 1.15 makes the dense pattern explicit by laying out a small three-layer network with every connection weight labeled. Every input connects to every hidden neuron, and every hidden neuron connects to every output. Each labeled edge represents one learnable weight, making visible the total parameter count and, consequently, why matrix multiplication dominates neural network computation: the weight matrix dimensions directly determine both the layer's storage requirements and its arithmetic cost. The numerical values shown are actual computed activations, demonstrating how inputs transform through the network. For a network with layers of sizes (n_1, n_2, n_3), the weight matrices are $\mathbf{W}^{(1)} \in \mathbb{R}^{n_1 \times n_2}$ between the first and second layers and $\mathbf{W}^{(2)} \in \mathbb{R}^{n_2 \times n_3}$ between the second and third layers.

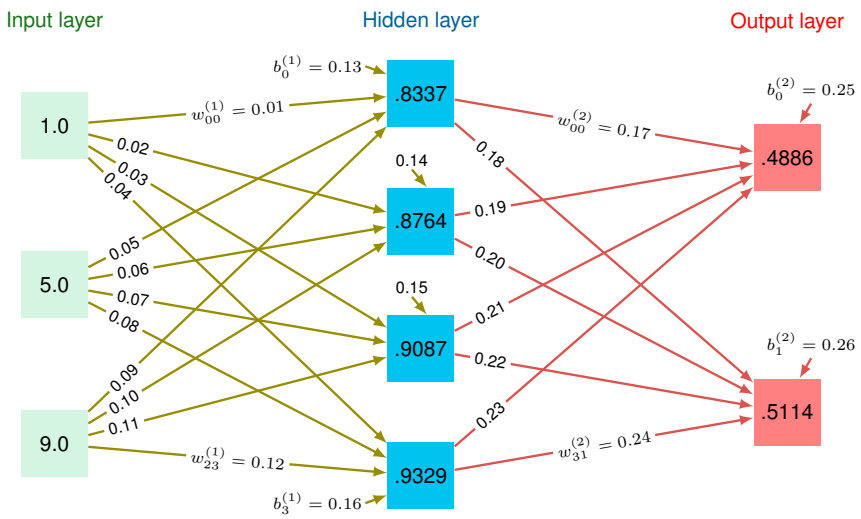


Figure 1.15: Fully-Connected Layers: A three-layer network with dense connections between layers, where each neuron integrates information from all neurons in the preceding layer. Weight matrices between layers determine connection strengths, with labeled values shown on each edge alongside computed activation values at each node.

1.3.3.2 Bias terms

Each neuron in a layer also has an associated bias term. While weights determine the relative importance of inputs, biases allow neurons to shift their activation functions. This shifting is important for learning, as it gives the network flexibility to fit more complex patterns.

For a layer with m neurons, the bias terms form a vector $\mathbf{b} \in \mathbb{R}^m$. When we compute the layer's output, this bias vector is added to the weighted sum of inputs (the same form as equation 1.8):

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$$

The bias terms[30] effectively allow each neuron to have a different "threshold" for activation, making the network more expressive.

The organization of weights and biases across a neural network follows a systematic pattern. For a network with N_L layers, three components per layer define the learnable state:

- A weight matrix $\mathbf{W}^{(\ell)}$ for each layer ℓ
- A bias vector $\mathbf{b}^{(\ell)}$ for each layer ℓ
- Activation functions $f^{(\ell)}$ for each layer ℓ

This gives us the complete layer computation in equation 1.9:

$$\mathbf{a}^{(\ell)} = f^{(\ell)}(\mathbf{z}^{(\ell)}) = f^{(\ell)}(\mathbf{a}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \tag{1.9}$$

Where $\mathbf{a}^{(\ell)}$ (written as $\mathbf{A}^{(\ell)}$ for batches) represents the layer's activation output. We adopt the row-vector convention throughout: each sample is a row, and the weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ maps from the previous layer's width to the current layer's width. With this equation in place, the core architecture concepts are complete enough to proceed.

30 **Bias Terms**: Biases add one parameter per neuron (vs. n weights per neuron), typically comprising 1–5 percent of total parameters. Despite this small fraction, removing biases can degrade accuracy by 1–3 percent on classification tasks because the network loses the ability to shift decision boundaries independently of input magnitude. Some modern architectures (notably those using batch normalization) omit biases entirely, since normalization layers subsume their function.

> **Checkpoint 1.2: Neural network architecture fundamentals**
>
> Before proceeding to network topology and training, verify your understanding of the foundational concepts we have covered:
>
> Use the MNIST architecture $784 \to 128 \to 64 \to 10$ as the running check.
>
> - ☐ **Neuron equation**: Can you write the neuron equation, including weighted sum, bias, and activation function?
> - ☐ **ReLU vs. sigmoid**: Can you explain why ReLU is computationally cheaper than sigmoid while still providing the nonlinearity a multilayer network needs?
> - ☐ **Layers to matrices**: Can you map input, hidden, and output layers to the corresponding weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n \times m}$?
> - ☐ **Parameter count**: Can you calculate the total parameter count, including both weights and biases?
> - ☐ **Width, depth, cost**: Can you explain how layer width, depth, and connectivity change memory footprint and arithmetic cost?
>
> If any of these feel unclear, review the earlier sections on Neural Network Fundamentals, Neurons and Activations, or Weights and Biases before continuing. The upcoming sections on training and optimization build directly on these foundations.

1.3.4 Architecture design

Architecture design determines how individual neurons, activation functions, and weight matrices connect to solve problems that no single layer can handle. That design has direct systems consequences, because every topological choice (adding a layer, widening a hidden dimension, changing connectivity) multiplies the parameter count and, with it, memory footprint and arithmetic cost.

Network topology describes how individual neurons organize into layers and connect to form complete neural networks. Building intuition begins with a simple problem that became famous in AI history[31].

31 **XOR Problem**: The inability of a single layer of neurons to solve the XOR function is the canonical example of why network topology is a computational necessity. Minsky and Papert's 1969 proof demonstrated that this simple function is not linearly separable, forcing a topological shift from a single layer to one with a hidden layer. This proves that some problems *cannot* be solved by making a layer wider, but require depth, with a minimum of three total neurons needed to learn XOR.

Example 1.2: Building intuition: The XOR problem

Consider a network learning the XOR function, a classic problem that requires nonlinearity. With inputs x_1 and x_2 that can be 0 or 1, XOR outputs 1 when inputs differ and 0 when they are the same.

Setup: two inputs → two hidden neurons → one output

Example: For inputs $(1, 0)$:

- Hidden neuron one: $h_1 = \text{ReLU}(1 \cdot w_{11} + 0 \cdot w_{12} + b_1)$
- Hidden neuron two: $h_2 = \text{ReLU}(1 \cdot w_{21} + 0 \cdot w_{22} + b_2)$
- Output: $y = \text{sigmoid}(h_1 \cdot w_{31} + h_2 \cdot w_{32} + b_3)$

Systems insight: Hidden layers are not just extra capacity; they change the class of functions the system can represent. XOR is the small example that makes depth a structural requirement rather than a decorative design choice (Minsky and Papert 1969).

The XOR example established the canonical three-layer architecture, but real-world networks require systematic consideration of design constraints and computational scale. Recognizing handwritten digits using the MNIST (LeCun et al. 1998) dataset illustrates how problem structure determines network dimensions while hidden layer configuration remains an important design decision.

1.3.4.1 Feedforward network architecture

Applying the three-layer architecture to MNIST reveals how data characteristics and task requirements constrain network design. Compare the two panels in figure 1.16 to see this architecture from both perspectives: panel (a) presents a 28×28 pixel grayscale image of a handwritten digit connected to the hidden and output layers, while panel (b) reveals how the 2D image flattens into a 784-dimensional vector. Flattening is necessary because fully-connected layers expect a fixed-size one-dimensional input: matrix multiplication requires a vector, not a grid. The cost of this transformation is that all spatial structure in the original image is discarded, which motivates convolutional architectures (explored in **?@sec-network-architectures**) that preserve spatial locality.

The input layer's width is directly determined by our data format. For a 28 by 28 pixel image, each pixel becomes an input feature, requiring 784 input neurons. Equivalently, 28 rows and 28 columns flatten into 784 values. We can think of this either as a 2D grid of pixels or as a flattened vector, where each value represents the intensity of one pixel.

The output layer's structure is determined by our task requirements. For digit classification, we use 10 output neurons, one for each possible digit (0–9). When presented with an image, the network produces a value for each output neuron, where higher values indicate greater confidence that the image represents that particular digit.

Between these fixed input and output layers, we have flexibility in designing the hidden layer topology. The choice of hidden layer structure, including the number of layers to use and their respective widths, represents one of the key design decisions in neural networks. Additional layers increase the network's depth, allowing it to learn more abstract features through successive transformations. The width of each layer provides capacity for learning different features at each level of abstraction.

Widening hidden layers from 100 to 1000 neurons increases parameters from ~89.6K to ~1.8M (~358 KB vs. ~7 MB in FP32) while improving MNIST accuracy only marginally (98.5 percent vs. 99.5 percent)—a trade-off that matters when deployment targets mobile memory budgets.

1.3.4.2 Layer connectivity design patterns

The preceding fully connected architecture maximizes flexibility by connecting every neuron to every neuron in the next layer, but connectivity itself is an engineering decision. Dense, sparse, and skip patterns trade learning flexibility against parameter count, locality, and gradient flow.

Dense connectivity represents the standard pattern where each neuron connects to every neuron in the subsequent layer. In our MNIST example, connecting our 784-dimensional input layer to a

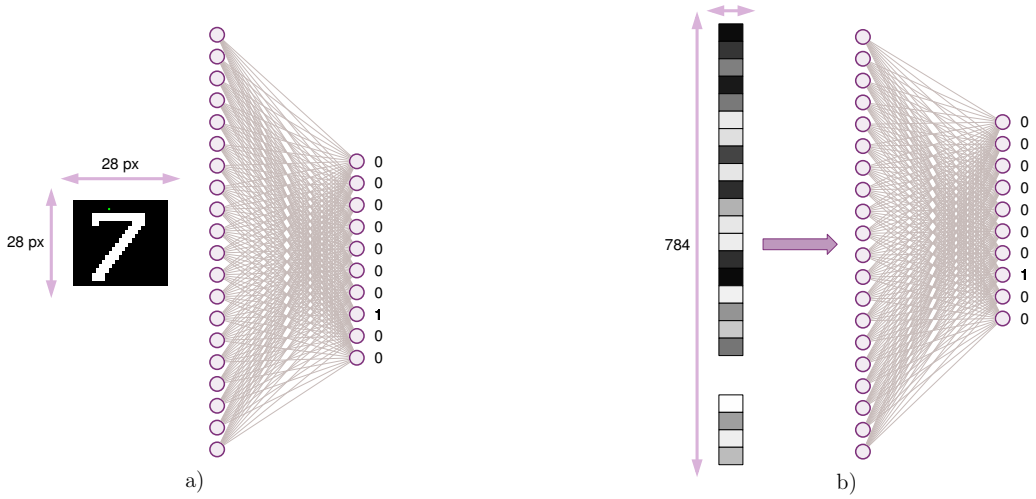


Figure 1.16: MNIST Network Topology: Two panels show the network architecture for digit recognition. Panel (a) displays a 28×28 pixel image of a digit connected through hidden layers to 10 output nodes. Panel (b) shows the same architecture with the input image flattened into a 784-element vector, illustrating how spatial data enters the network.

hidden layer of 128 neurons requires 100,352 weight parameters (784×128). This full connectivity enables the network to learn arbitrary relationships between inputs and outputs, but the number of parameters scales quadratically with layer width.

Sparse connectivity patterns introduce purposeful restrictions in how neurons connect between layers. Rather than maintaining all possible connections, neurons connect to only a subset of neurons in the adjacent layer. This approach draws inspiration from biological neural systems, where neurons typically form connections with a limited number of other neurons. In visual processing tasks like our MNIST example, neurons might connect only to inputs representing nearby pixels, reflecting the local nature of visual features.

As networks grow deeper, the path from input to output becomes longer, potentially complicating the learning process. Skip connections address this by adding direct paths between nonadjacent layers. These connections provide alternative routes for information flow, supplementing the standard layer-by-layer progression. In our digit recognition example, skip connections might allow later layers to reference both high-level patterns and the original pixel values directly.

These connection patterns have direct hardware consequences as well as theoretical ones. Dense connections maximize learning flexibility and map naturally onto the GEMM kernels that tensor cores and memory hierarchies execute at maximum bandwidth. Sparse connections reduce theoretical parameter counts and FLOPs, but a sparse weight matrix does not automatically execute faster than a dense one on standard hardware—without structured sparsity support (such as 2:4 patterns that hardware can exploit through compressed index formats), the arithmetic reduction does not translate to proportional reductions in execution time. Skip connections help maintain effective information flow in deeper networks while preserving the gradient paths that make very deep architectures trainable.

1.3.4.3 Model size and computational complexity

How parameters (weights and biases) are arranged determines both learning capacity and computational cost—this is the model's side of the silicon contract (**?@sec-introduction-iron-law-ml-systems-c32a**): the parameter count, their numerical precision, and the operations they require collectively define the computational bargain the model strikes with hardware. While topology defines the network's structure, parameter initialization and organization directly affect learning dynamics and final performance.

Worked example: Training vs. inference memory Earlier we showed that a single forward pass through the 784 → 128 → 64 → 10 network costs 109,184 MACs. The memory footprint during

training tells a different story. We compute the footprint for this network at batch size 32 in 32-bit (4-byte) floating-point precision, then contrast it with inference requirements, accounting for parameters, activations, gradients, and optimizer state in turn.

The first contribution is the model parameters. Table 1.6 tallies the weights and biases layer by layer:

Table 1.6: MNIST parameter memory by layer: Weight and bias counts for the 784-128-64-10 network. Layer 1 (Input -> Hidden1) dominates the parameter budget because it multiplies the 784-dimensional input by 128 hidden units; deeper layers shrink rapidly as the representation compresses toward the 10-class output.

Layer	Weights	Biases	Total Parameters
Input → Hidden1	$784 \times 128 = 100{,}352$	128	100,480
Hidden1 → Hidden2	$128 \times 64 = 8{,}192$	64	8,256
Hidden2 → Output	$64 \times 10 = 640$	10	650
Total			**109,386 parameters**

In total, 109,386 parameters parameters at 4 bytes each occupy 437.5 KB.

Activations are the next contribution. Table 1.7 records each layer's activation tensor and its memory cost at batch size 32:

Table 1.7: MNIST activation memory at batch size 32: Per-layer activation tensor shapes, value counts, and 32-bit float memory cost during a single forward pass. Input activations dominate the activation table at 100.4 KB; for this small MNIST MLP, the total activation footprint remains below the parameter footprint, while larger batches and deeper networks make activations a central training-memory concern.

Layer	Activation Shape	Values	Memory
Input	32×784	25,088	100.4 KB
Hidden1	32×128	4,096	16.4 KB
Hidden2	32×64	2,048	8.2 KB
Output	32×10	320	1.3 KB
Total		31,552	**126 KB**

Training adds two memory contributions that inference never pays. Gradients occupy the same space as the parameters they update, here 437.5 KB, and Adam's optimizer state stores momentum and velocity at twice the parameter size, another 875.1 KB. Table 1.8 summarizes the per-component memory footprint for training vs. inference.

Table 1.8: MNIST training vs. inference memory: Per-component memory footprint (parameters, activations, gradients, optimizer state) for training vs. inference on the toy MNIST MLP, exposing why training memory dominates and scales with both batch size and parameter count.

Component	Training	Inference
Parameters	437.5 KB	437.5 KB
Activations	126 KB	4 KB
Gradients	437.5 KB	—
Optimizer state	875.1 KB	—
Total	**~1.9 MB**	**~441 KB**

The systems lesson is one of scale: training at batch size 32 requires 4.3× more memory than single-sample inference in this example. For larger models, this ratio increases further because gradient and optimizer storage scale with parameter count, while activations scale with batch size and layer widths.

Parameter count grows with network width and depth. For our MNIST example, consider a network with a 784-dimensional input layer, hidden layers of 128 and 64 neurons, and a 10-neuron output layer ($784 \to 128 \to 64 \to 10$). The first layer requires 100,352 weights and 128 biases, the second layer 8,192 weights and 64 biases, and the output layer 640 weights and 10 biases, totaling 109,386 parameters. Each must be stored in memory and updated during learning.

The preceding memory requirements seem modest for our small MNIST classifier. Scaling to production-sized models transforms these requirements dramatically, changing the hardware regime.

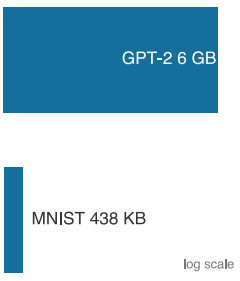
Napkin Math 1.1: The memory explosion

Problem: How does model scale affect the data (D_{vol}) term of the iron law? Compare our MNIST running example against the GPT-2 lighthouse.

- **MNIST (running example)**: 109,386 parameters at 4 bytes each occupy approximately 438 KB. This entire model fits inside the L2 cache of a modern processor.
- **GPT-2 (lighthouse)**: 1,500,000,000 parameters at 4 bytes each occupy approximately 6 GB. This requires dedicated GPU VRAM and high-speed memory bandwidth.

Systems insight: Moving from ~109.4K to 1.5B parameters is a 13,712.9× jump. The increase represents a phase change in engineering, not merely "more parameters." MNIST is a cache-resident arithmetic problem; GPT-2 is a data movement problem.

The preceding memory calculations are precise but slow. In practice, systems engineers work at two levels of fidelity: exact budgets for design documents and order-of-magnitude estimates for early feasibility gates. The exact budget we just computed confirmed that MNIST fits comfortably in cache while GPT-2 requires dedicated accelerator memory. A quick mental estimate should reach the same conclusion in seconds, not minutes, and flag any model that cannot physically fit on the target hardware before a single line of profiling code runs.



MNIST is cache-scale; GPT-2 is VRAM-scale.

Napkin Math 1.2: Quick estimation for ML engineers

Detailed calculations are essential for design documents, but experienced engineers also develop rapid mental estimation skills. These "napkin math" shortcuts enable quick feasibility checks before committing to detailed analysis.

Memory Estimation

- **Parameters → bytes**: Multiply by four (FP32), two (FP16 or BF16, a 16-bit format with FP32-like exponent range), or one (INT8)
- **FC layer parameters**: Input × Output (plus Output biases, usually negligible)
- **Training memory**: ~3–4× inference memory (gradients + optimizer state)
- **Adam optimizer overhead**: 2× parameter memory (momentum + velocity)
- **Max batch size**: $(\text{GPU VRAM} - \text{Model Size})/\text{Activations per sample}$

Compute Estimation

- **FC layer FLOPs**: $2 \times d_{\text{in}} \times d_{\text{out}} \times B$ (multiply-add = 2 ops)
- **MACs to FLOPs**: Multiply by 2
- **GPU utilization**: achieved $\text{FLOP/s}/R_{\text{peak}}$ (typically 30–70 percent for training)

Table 1.9 distills three of the most common feasibility questions into one-line formulas an engineer can apply before reaching for a profiler.

Table 1.9: Napkin-Math Feasibility Checks: Three rapid mental-estimation rules for GPU fit, epoch time, and bound regime, distilled into one-line formulas an engineer can apply before running a profiler.

Question	Quick Estimate
"Will this model fit in GPU memory?"	Parameters × 4 bytes × 4 (training) < VRAM
"How long per epoch on MNIST?"	$60\text{K} \times \text{FLOPs/image}/R_{\text{peak}}$
"Is this compute bound or memory bound?"	If $B \times d_{\text{layer}} < 1000$, likely memory bound

Example: *"Can I train a 100M parameter model on a 16 GB GPU?"*
Mental math: 100M parameters at 4 bytes each, with 4 training-memory copies, require 1.6 GB for the model. That leaves about 14.4 GB for activations and batch data. Answer: Yes, comfortably—batch size is the main constraint.

Feasibility math tells the engineer whether a model *fits*; it says nothing about whether it will *learn*. That depends on how the parameters those bytes hold are first set. Parameter initialization is critical to network behavior. Setting all parameters to zero would cause neurons in a layer to behave identically, preventing diverse feature learning. Instead, weights are typically initialized randomly, often using specific strategies like Xavier/Glorot initialization[32] (Glorot and Bengio 2010) or He initialization (He et al. 2015), while biases often start at small constant values or zeros. The scale of these initial values matters: values that are too large or too small lead to poor learning dynamics.

The distribution of parameters affects information flow through layers. In digit recognition, if weights are too small, important input details might not propagate to later layers. If too large, the network might amplify noise. Biases help adjust the activation threshold of each neuron, enabling the network to learn optimal decision boundaries.

Different architectures impose specific constraints on parameter organization. Some share weights across network regions to encode position-invariant pattern recognition; others restrict certain weights to zero, implementing sparse connectivity patterns.

Network architecture, neurons, and parameters are now in place, but a central question remains: the mechanism by which these randomly initialized parameters become useful. A randomly wired network produces outputs no better than chance. Understanding the architecture of a neural network answers *what* the model computes; understanding training answers *how* the model learns. The training process transforms a randomly initialized network into one that captures meaningful patterns in data, and the mechanics of that transformation reveal fundamental systems constraints. Training demands far more memory than inference, gradient computation dominates energy budgets, and batch size is ultimately a hardware decision. The learning process addresses these constraints as networks systematically adjust their weights based on feedback from training data, transforming 109,386 parameters from random numbers into a functioning digit classifier.

[32] **Xavier/Glorot Initialization**: Weight variance must scale as $1/n$ (where n is layer width) to prevent activations from vanishing or exploding across layers (Glorot and Bengio 2010). Before this insight, training failures from poor initialization were routinely misdiagnosed as hardware bugs or insufficient compute. The fix costs zero additional FLOPs; it is purely a matter of setting the right random distribution at startup.

1.4 Learning Process

Our MNIST network currently holds 109,386 parameters initialized randomly—numbers that encode no knowledge at all. The transformation of these random values into a digit classifier achieving over 95 percent accuracy relies on four operations repeated millions of times: forward propagation computes a prediction, a loss function measures the error, backpropagation assigns blame to each weight, and an optimizer adjusts those weights to reduce the error.

1.4.1 Supervised learning from labeled examples

A randomly initialized network classifies digits no better than random guessing among ten classes (about 10 percent accuracy). Transforming it into a 95 percent-accurate classifier requires supervised learning: showing the network labeled examples and adjusting its weights based on the errors it makes. Consider our MNIST digit recognition task: we have a dataset of 60,000 training images, each a 28×28 pixel grayscale image paired with its correct digit label. The network must learn the relationship between these images and their corresponding digits through an iterative process of prediction and weight adjustment. Ensuring the quality and integrity of training data is essential to model success, as established in **?@sec-data-engineering**.

The relationship between inputs and outputs drives the training methodology. Training operates as a loop where each iteration processes a subset of training examples called a batch[33]. For each batch, the network performs four operations: forward computation through the network layers generates predictions, a loss function evaluates prediction accuracy, weight adjustments are computed based on prediction errors, and network weights are updated to improve future predictions.

The iterative approach can be expressed mathematically. Given an input image x and its true label y, the network computes its prediction according to equation 1.10:

$$\hat{y} = f(x;\theta) \tag{1.10}$$

[33] **Batch Processing**: Batching serves dual purposes: larger batches provide more stable gradient estimates by averaging noise across examples and better saturate parallel hardware, since GPUs process thirty-two inputs with nearly the same latency as one because matrix multiplication parallelizes across the batch dimension. The trade-off: each doubling of batch size roughly doubles activation memory, making batch size ultimately a hardware-memory decision rather than a purely statistical one.

This equation encapsulates the entire forward pass: the network f takes an input x (say, a 28×28 digit image) and, using its current parameters θ (all the weights and biases we examined earlier), produces a prediction $\hat{y}$ (a vector of ten probabilities, one per digit). The semicolon notation $f(x;\theta)$ distinguishes the input x, which changes with every example, from θ, which remains fixed during inference but evolves during training. The network's error is measured by a loss function[34] $\mathcal{L}$, as shown in equation 1.11:

$$\text{loss} = \mathcal{L}(\hat{y}, y) \tag{1.11}$$

The error measurement drives the adjustment of network parameters through backpropagation, examined in section 1.4.4.

In practice, training operates on batches of examples rather than individual inputs. For the MNIST dataset, each training iteration might process 32, 64, or 128 images simultaneously for reasons we formalize in section 1.4.5.2. The training cycle continues until the network achieves sufficient accuracy or reaches a predetermined number of iterations. Throughout this process, the loss function serves as a guide, its minimization indicating improved performance. Establishing proper metrics and evaluation protocols is essential for assessing training effectiveness, as discussed in **?@sec-benchmarking**.

[34] **Loss Function**: Formalized by Abraham Wald in statistical decision theory as the "cost" of an incorrect decision, $\mathcal{L}$ quantifies the gap between prediction $\hat{y}$ and ground truth y. The choice of loss function shapes the optimization geometry: it determines the gradient landscape that backpropagation must navigate. A loss with flat regions near incorrect predictions produces weak gradients that stall learning, while a loss with steep gradients near the decision boundary accelerates convergence where it matters most—a systems consequence explored in the cross-entropy discussion below.

1.4.2 Forward pass computation

An MNIST image becomes ten class scores by moving through weighted layers, nonlinear activations, and a final comparison against the correct digit. That computation is **forward propagation**: input data flows through the network's layers to generate predictions. Figure 1.17 traces the complete process. Inputs enter from the left, pass through weighted connections to hidden layers, generate a prediction that is compared against the true value, and produce a loss score that drives parameter updates through the optimizer. This process underlies both inference and training.

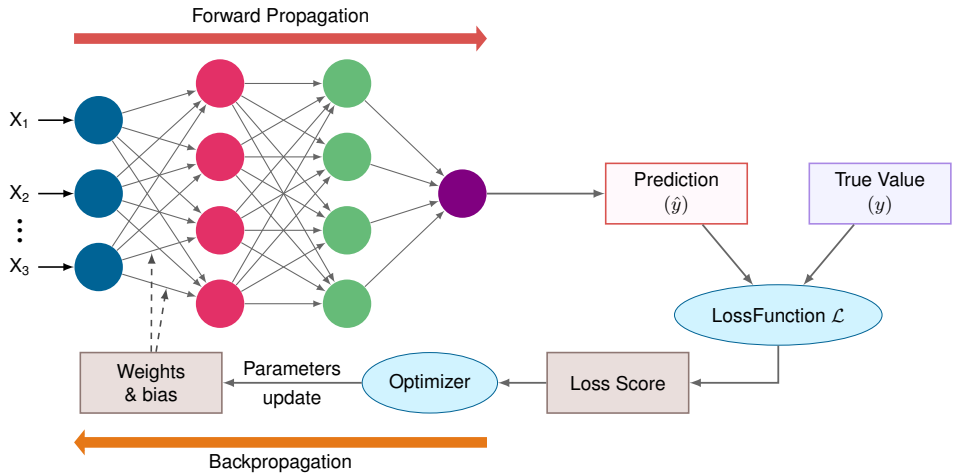


Figure 1.17: Training Loop Architecture: Complete neural network training flow showing forward propagation through layers to generate prediction, comparison with true value via loss function, and backward propagation of gradients through optimizer to update weights and biases.

The bidirectional flow of data moving forward through the layers (the red arrow in figure 1.17) and gradients flowing backward to update weights (the orange arrow) is the heartbeat of neural network training. The figure reveals a critical asymmetry: forward propagation produces a single output, but backward propagation must compute gradients for every weight in the network. *This asymmetry explains why training requires storing all intermediate activations—each layer's gradient computation depends on what that layer received during the forward pass.* Before the mathematical details, pause to consolidate this core mechanism.

When an image of a handwritten digit enters our network, it undergoes a series of transformations through the layers. Each transformation combines the weighted inputs with learned patterns to

progressively extract relevant features. For the 784-128-64-10 digit classifier, a 28×28 pixel image is processed through multiple layers to ultimately produce probabilities for each possible digit (0–9).

Checkpoint 1.3: Gradient flow

The forward pass is only half the story.

Data vs. Signal

- ☐ **Forward pass**: Can you trace how input data becomes predictions layer by layer?
- ☐ **Backward pass**: Can you trace how the error signal moves backward to update weights?

Dependencies

- ☐ **Memory**: Why must we store the forward pass activations?

The process begins with the input layer, where each pixel's grayscale value becomes an input feature. For MNIST, this means 784 input values ($28\times28 = 784$), each normalized between 0 and 1. These values then propagate forward through the hidden layers, where each neuron combines its inputs according to its learned weights and applies a nonlinear activation function.

Each forward pass through our MNIST network (784-128-64-10) requires substantial matrix operations. The first layer alone performs 100,352 MACs per sample. When processing multiple samples in a batch, these operations multiply accordingly, requiring careful management of memory bandwidth and computational resources. Specialized hardware like GPUs executes these operations efficiently through parallel processing.

1.4.2.1 Individual layer processing

The forward computation through a neural network proceeds systematically, with each layer transforming its inputs into increasingly abstract representations. The digit classifier illustrates this: its transformation process occurs in distinct stages. At each layer, the computation involves two key steps: a linear transformation of inputs followed by a nonlinear activation. The linear transformation applies the same weighted sum operation we saw earlier, but now using notation that tracks which layer we are in, as shown in equation 1.12:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \tag{1.12}$$

Here, $\mathbf{A}^{(\ell-1)}$ contains the activations from the previous layer (the outputs after applying activation functions), $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1}\times n_\ell}$ is the weight matrix for layer ℓ, and $\mathbf{b}^{(\ell)}$ is the bias vector (broadcast across the batch). The superscript (ℓ) keeps track of which layer each parameter belongs to. This row-vector convention matches the single-sample equation from earlier: each row of $\mathbf{A}$ is one sample, and right-multiplying by $\mathbf{W}$ transforms it to the next layer's width.

Following this linear transformation, each layer applies a nonlinear activation function f (we now write f or $f^{(\ell)}$ for a generic activation function at layer ℓ; earlier, σ referred specifically to the sigmoid function), as expressed in equation 1.13:

$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)}) \tag{1.13}$$

This process repeats at each layer, creating a chain of transformations:

$$\text{Input} \to \text{Linear Transform} \to \text{Activation} \to \text{Linear Transform} \to \text{Activation} \to \cdots \to \text{Output}$$

Returning to digit recognition, the pixel values first undergo a transformation by the first hidden layer's weights, converting the 784-dimensional input into an intermediate representation. Each subsequent layer further transforms this representation, ultimately producing a 10-dimensional output vector representing the network's confidence in each possible digit.

1.4.2.2 Matrix multiplication formulation

The complete forward propagation process can be expressed as a composition of functions, each representing a layer's transformation. Formalizing this mathematically builds on the MNIST example.

For a network with N_L layers, we can express the full forward computation as equation 1.14:

$$\mathbf{A}^{(N_L)} = f^{(N_L)}\Big(\cdots f^{(2)}\Big(f^{(1)}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\Big)\cdots \mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}\Big) \tag{1.14}$$

This composition reveals that forward propagation is, at its core, a chain of matrix multiplications interleaved with nonlinear activations. Understanding *why* matrix multiplication dominates AI computation requires examining the arithmetic intensity of each operation.

MatMul

Matrix multiplication is over 90 percent of forward-pass FLOPs.

Systems Perspective 1.4: Why matrix multiplication dominates AI

Not all operations are created equal. Systems engineers distinguish between *compute-bound* operations (dense math) and *memory-bound* operations (simple math). Table 1.10 contrasts matrix multiplication against element-wise ReLU on these dimensions:

Table 1.10: Arithmetic intensity of matrix multiply vs. element-wise ReLU: Dense $N \times N$ matrix multiplication performs $2N^3$ operations while moving about $3N^2 s$ bytes for s bytes per value, yielding intensity that grows linearly with N and saturates GPU/TPU compute. Element-wise activations stay at $1/(2s)$ FLOP/byte regardless of size (0.125 FLOP/byte for FP32), leaving accelerators starved for arithmetic and explaining why GEMM-heavy layers dominate modern architectures.

Operation	Operation Count (O)	Data Movement (I/O)	Intensity (FLOP/byte)	Hardware Fit
Matrix Mul $(N \times N)$	$2N^3$	$3N^2 s$ bytes	$\approx 2N/(3s)$ (High)	**GPU/TPU**
Element-wise (ReLU)	N^2	$2N^2 s$ bytes	$1/(2s)$ (Low)	**CPU/Vector**

Modern AI accelerators (GPUs) have massive compute arrays but limited memory bandwidth. They only achieve peak performance on high-intensity operations like matrix multiplication where data is reused many times. This is why "fully connected" and "convolutional" layers are preferred over complex, custom element-wise logic.

The mathematical expression **xW** is implemented in hardware as a **General Matrix Multiply (GEMM)** kernel, the most optimized routine in all of computing, accounting for over 90 percent of the floating-point operations in most neural networks. To achieve peak performance, engineers use techniques like blocking and tiling to ensure data fits perfectly into L1/L2 caches and remains there as long as possible (data reuse). This hardware-software co-design principle, designing model architectures to use large, dense matrix multiplications that specialized dense-matrix hardware can execute efficiently, is what makes modern deep learning physically possible. **?@sec-appdx-algorithm-foundations-general-matrix-multiply-gemm-b55d** provides the detailed treatment of GEMM arithmetic intensity, sparse matrix formats, and the computational complexity of common layer types needed to optimize these operations in practice.

The nested expression unfolds layer by layer, each step consuming the previous layer's activations as its input:

1. First layer:

$$\mathbf{Z}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$
$$\mathbf{A}^{(1)} = f^{(1)}(\mathbf{Z}^{(1)})$$

2. Hidden layers $(\ell = 2, \ldots, N_L - 1)$:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$
$$\mathbf{A}^{(\ell)} = f^{(\ell)}(\mathbf{Z}^{(\ell)})$$

3. Output layer:

$$\mathbf{Z}^{(N_L)} = \mathbf{A}^{(N_L-1)}\mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}$$
$$\mathbf{A}^{(N_L)} = f^{(N_L)}(\mathbf{Z}^{(N_L)})$$

In our MNIST example, the operation dimensions for a batch of B images are as follows:

- Input $\mathbf{X}$: $B \times 784$
- First layer weights $\mathbf{W}^{(1)}$: $784 \times n_1$
- Hidden layer weights $\mathbf{W}^{(\ell)}$: $n_{\ell-1} \times n_\ell$
- Output layer weights $\mathbf{W}^{(N_L)}$: $n_{N_L-1} \times 10$

1.4.2.3 Step-by-step computation sequence

Understanding how these mathematical operations translate into actual computation requires examining the forward propagation process for a batch of MNIST images. This process illustrates how data transforms from raw pixel values to digit predictions.

Consider a batch of 32 images entering our network. Each image starts as a 28×28 grid of pixel values, which we flatten into a 784-dimensional vector. For the entire batch, this gives us an input matrix $\mathbf{X}$ of size 32×784, where each row represents one image. The values are typically normalized to lie between 0 and 1.

The transformation at each layer proceeds as a shape-changing sequence; the shapes determine both kernel choice and activation storage. The network first takes the input matrix $\mathbf{X}$ (32×784) and transforms it using the first layer's weights. If the first hidden layer has 128 neurons, $\mathbf{W}^{(1)}$ is a 784×128 matrix, so the computation $\mathbf{X}\mathbf{W}^{(1)}$ produces a 32×128 matrix. Each element in this matrix then has its corresponding bias added and passes through an activation function. For example, with a ReLU activation, any negative values become zero while positive values remain unchanged. This nonlinear transformation enables the network to learn complex patterns in the data. The final layer transforms its inputs into a 32×10 matrix, where each row contains 10 values corresponding to the network's confidence scores for each possible digit. Let z_j denote the raw score (logit) for digit j and let z_k range over all 10 digits. Often, these scores are converted to probabilities using the softmax function in equation 1.6:

$$p(\text{digit } j) = \frac{e^{z_j}}{\sum_{k=1}^{10} e^{z_k}}$$

For each image in the batch, this produces a probability distribution over the possible digits. The digit with the highest probability represents the network's prediction. A layer-by-layer operation count makes the computational cost explicit.

Example 1.3: Counting operations in forward pass

Problem: What is the total arithmetic operation count (O) for one forward pass through the MNIST network (784 → 128 → 64 → 10) with batch size 32?

Background: A matrix multiplication of dimensions $(M \times K) \times (K \times N)$ requires $2 \times M \times K \times N$ FLOPs (one multiply and one add per output element, summed over K terms). Bias addition adds $M \times N$ floating-point additions. ReLU activation adds $M \times N$ comparisons, so the table reports total operation-equivalent work rather than pure FLOPs for activation rows.

Solution: Table 1.11 breaks down the operation count layer by layer. The total comes to ~7 MOp, or 7 MOp ÷ 32 = ~219 KOp per image.

Systems insight:

1. **Layer 1 dominates**: The first layer accounts for 91.9 percent of all operations because it processes the largest input (784 dimensions). This is why dimensionality reduction in early layers is so impactful.
2. **Compute vs. memory**: At 219 KOp per image and ~441 KB memory, this network has a matmul-dominated arithmetic intensity of ~0.5 FLOP/byte—firmly in the memory-

bound regime for most hardware (**?@sec-appdx-machine-foundations-roofline-model-2529** shows how arithmetic intensity determines whether a workload is memory bound or compute bound). A modern GPU achieving ten TFLOP/s would process each image in ~22 nanoseconds of pure compute, but memory latency typically dominates actual inference time.

3. **Scaling intuition**: Doubling the hidden layer widths (784 → 256 → 128 → 10) increases the operation count by about 2.1× to about 15 MOp. This comes from recomputing each layer: layers 1 and 3 double, layer 2 quadruples, so the total grows by about 2.15× rather than four times.

Table 1.11: MNIST Forward Pass Operation Count by Layer: Operation-equivalent breakdown for a single forward pass through the 784-128-64-10 network at batch size 32. Matrix multiplication rows are FLOPs; bias, ReLU, and softmax rows are elementwise operations included to show their negligible contribution. Layer 1's matrix multiply dominates because it multiplies the 784-dimensional input against the widest weight matrix; bias and activation operations contribute under 1 percent of the total.

Layer	Operation	Dimensions	Operations
Layer 1	MatMul	$(32\times784) \times (784\times128)$	$2 \times 32 \times 784 \times 128 = 6{,}422{,}528$
Layer 1	Bias + ReLU	32×128	$2\times4{,}096 = 8{,}192$
Layer 2	MatMul	$(32\times128) \times (128\times64)$	$2 \times 32 \times 128 \times 64 = 524{,}288$
Layer 2	Bias + ReLU	32×64	$2\times2{,}048 = 4{,}096$
Layer 3	MatMul	$(32\times64) \times (64\times10)$	$2 \times 32 \times 64 \times 10 = 40{,}960$
Layer 3	Bias + Softmax	32×10	~640 (simplified)
Total			**~7 MOp**

1.4.2.4 Implementation and optimization considerations

Forward propagation is easy to state mathematically, but its implementation is constrained by activation storage, batch size, memory layout, and hardware fit.

Memory management plays a central role during forward propagation. Each layer's activations must be stored for the backward pass during training. For our MNIST example (784-128-64-10) with a batch size of 32, the activation storage requirements are:

- Input layer: $32\times784 = 25{,}088$ values
- First hidden layer: $32\times128 = 4{,}096$ values
- Second hidden layer: $32\times64 = 2{,}048$ values
- Output layer: $32\times10 = 320$ values

This produces a total of 31,552 values that must be maintained in memory for each batch during training, consistent with the worked example in section 1.3.4.3. The memory requirements scale linearly with batch size and become substantial for larger networks.

Batch processing introduces important trade-offs. Larger batches enable more efficient matrix operations and better hardware utilization but require more memory. For example, doubling the batch size to 64 would double the memory requirements for activations. This relationship between batch size, memory usage, and computational efficiency guides the choice of batch size in practice.

The organization of computations also affects performance. Matrix operations can be optimized through careful memory layout and specialized libraries. The choice of activation functions affects both the network's learning capabilities and computational efficiency, as some functions (like ReLU) require less computation than others (like tanh or sigmoid).

The computational characteristics of neural networks favor parallel processing architectures. While traditional CPUs can execute these operations, GPUs designed for parallel computation can be substantially faster for large dense matrix operations. Specialized AI accelerators achieve even better efficiency through reduced precision arithmetic, specialized memory architectures, and dataflow optimizations tailored for neural network computation patterns.

Energy consumption also varies by orders of magnitude across hardware platforms. CPUs offer flexibility but consume more energy per operation. GPUs provide high throughput at higher power consumption. Specialized edge accelerators optimize for energy efficiency, achieving the same computations with orders of magnitude less power, which is important for mobile and embedded deployments. This energy disparity stems from the memory hierarchy constraints where data movement dominates computation costs. These considerations recur throughout subsequent chapters, particularly in **?@sec-network-architectures** where architecture-specific optimizations introduce additional trade-offs.

Forward propagation transforms inputs into predictions, but a prediction alone is useless for learning. The training loop requires a way to measure *how wrong* that prediction is in a form that guides weight adjustments. Loss functions fill this role: they translate the gap between prediction and reality into a single number that optimization can minimize.

1.4.3 Loss functions

The forward propagation process described earlier suffices for **inference**, using a pretrained model to make predictions. To train a model, however, we need a way to measure how well those predictions match reality. Loss functions quantify these errors, serving as the feedback mechanism that guides learning. They convert the abstract goal of "making good predictions" into a concrete optimization problem.

Continuing with our MNIST digit recognition example: when the network processes a handwritten digit image, it outputs ten numbers representing its confidence in each possible digit (0–9). The loss function measures how far these predictions deviate from the true answer. If an image displays a "seven", the network should exhibit high confidence for digit "seven" and low confidence for all other digits. The loss function penalizes deviations from this target, with higher loss values signaling that the network needs significant improvement.

1.4.3.1 Error measurement fundamentals

A loss function measures how far the network's predictions are from the correct answers. This difference is expressed as a single number: lower loss means more accurate predictions, while higher loss indicates the network needs improvement. During training, the loss function guides weight adjustments. In recognizing handwritten digits, for example, the loss penalizes predictions that assign low confidence to the correct digit.

Mathematically, a loss function $\mathcal{L}$ takes two inputs: the network's predictions $\hat{y}$ and the true values y. For a single training example in digit classification, the loss measures the discrepancy between prediction and truth. When training with batches of data, we typically compute the average loss across all examples in the batch, as shown in equation 1.15:

$$\mathcal{L}_{\text{batch}} = \frac{1}{B}\sum_{i=1}^{B}\mathcal{L}(\hat{y}_i, y_i) \tag{1.15}$$

where B is the batch size and $(\hat{y}_i, y_i)$ represents the prediction and truth for the i-th example. Averaging over the batch serves two purposes: it makes the loss independent of batch size (so the same learning rate works whether $B = 32$ or $B = 256$), and the summation across examples maps naturally to parallel hardware—each example's loss can be computed independently before a single reduction step combines them.

The choice of loss function depends on the type of task. For digit classification, the loss function must handle probability distributions over multiple classes, provide meaningful gradients that guide learning, penalize wrong predictions in proportion to their severity, and scale efficiently with batch processing. Cross-entropy loss satisfies all four requirements.

1.4.3.2 Cross-entropy and classification loss functions

For classification tasks like MNIST digit recognition, **cross-entropy loss**[35] is a common way to compare predicted probability distributions with true class labels. The information-theoretic idea of entropy traces to Shannon (Shannon 1948); in supervised classification, cross-entropy penalizes low probability assigned to the correct class.

[35] **Cross-Entropy Loss**: This function measures the mismatch between the predicted probability distribution and the true one-hot encoded label. It penalizes confident but incorrect predictions logarithmically, creating strong gradients when the model assigns little probability to the correct class.

[36] **One-Hot Encoding**: Representing K classes as K-dimensional binary vectors where exactly one element is 1. This encoding is sparse by construction: for MNIST's 10 classes, 90 percent of each label vector is zeros. At scale, this sparsity becomes a systems concern. Encoding 100,000 classes (as in large-vocabulary language models) produces label vectors that waste memory and bandwidth, motivating alternatives like label smoothing and sampled softmax that trade exact one-hot targets for compute efficiency.

For a single digit image, our network outputs a probability distribution over the 10 possible digits. We represent the true label as a one-hot vector[36] where all entries are 0 except for a one at the correct digit's position. For instance, if the true digit is "seven", the label would be $y = [0,0,0,0,0,0,0,1,0,0]$.

The cross-entropy loss for this example is defined in equation 1.16:

$$\mathcal{L}(\hat{y}, y) = -\sum_{j=1}^{10} y_j \log(\hat{y}_j) \tag{1.16}$$

where $\hat{y}_j$ represents the network's predicted probability for digit j. Given our one-hot encoding, this simplifies to equation 1.17:

$$\mathcal{L}(\hat{y}, y) = -\log(\hat{y}_c) \tag{1.17}$$

where c is the index of the correct class. The loss therefore depends only on the predicted probability for the correct digit; the network is penalized based on how confident it is in the right answer.

For example, if our network predicts the following probabilities for an image of "seven":

```
Predicted: [0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.8, 0.0, 0.1]
True: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
```

The loss would be $-\log(0.8)$, which is approximately 0.223. If the network were more confident and predicted 0.9 for the correct digit, the loss would decrease to approximately 0.105.

1.4.3.3 Batch loss calculation methods

The practical computation of loss involves considerations for both numerical stability and batch processing. When working with batches of data, we compute the average loss across all examples in the batch.

For a batch of B examples, the cross-entropy loss becomes equation 1.18:

$$\mathcal{L}_{\text{batch}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{10} y_{ij} \log(\hat{y}_{ij}) \tag{1.18}$$

Here, i indexes examples in the batch, j indexes the ten output classes, y_{ij} is the one-hot target indicator for class j on example i, and $\hat{y}_{ij}$ is the predicted probability for that class. Averaging over B keeps the loss scale comparable as batch size changes.

Computing this loss efficiently requires careful consideration of numerical precision. The dangerous case is not an ordinary small probability, but a probability that an unstable softmax rounds to zero. If the correct-class probability becomes 0, then $\log(0)$ becomes $-\infty$ and the loss can turn into a not a number (NaN) during later arithmetic.

Two implementation safeguards prevent numerical instability in the loss computation:

1. Add a small epsilon to prevent taking log of zero, as in equation 1.19:

$$\mathcal{L} = -\log(\hat{y} + \epsilon) \tag{1.19}$$

Here, ϵ is a tiny positive constant that prevents $\log(0)$; it changes only numerically saturated probabilities, not ordinary confident predictions.

2. Apply the log-sum-exp trick for numerical stability (**?@sec-appdx-data-foundations-logits-numerical-stability-13e2** explains why this is necessary and how it works), as shown in equation 1.20:

$$\text{softmax}(z_i) = \frac{\exp(z_i - \max(z))}{\sum_j \exp(z_j - \max(z))} \tag{1.20}$$

Subtracting the same $\max(z)$ from every logit leaves the softmax probabilities unchanged because the common factor cancels between numerator and denominator. The numerical effect is decisive: the largest shifted logit becomes zero, so the largest exponential is $\exp(0) = 1$ rather than a potentially overflowing value.

With a batch size of 32 and 10 output classes, each training step processes 32 sets of 10 probabilities, computes a loss value for each of the 32 examples, and averages those values into a single batch loss.

1.4.3.4 Impact on learning dynamics

Loss functions influence training in ways that explain key implementation decisions. During each training iteration, the loss value serves multiple purposes. As a performance metric, it quantifies current network accuracy. As an optimization target, its gradients guide weight updates toward better predictions. As a convergence signal, its trend over time indicates whether training is progressing, stalling, or diverging.

For our MNIST classifier, monitoring the loss during training reveals the network's learning trajectory. A typical pattern begins with high loss (~ 2.3, equivalent to random guessing among ten classes), followed by rapid decrease in early iterations as the network discovers the most salient features. Progress then slows to gradual improvement as the network fine-tunes its predictions for harder cases, eventually stabilizing at a lower loss (~ 0.1, indicating confident correct predictions).

The loss function's gradients with respect to the network's outputs provide the initial error signal that drives backpropagation. For cross-entropy loss, these gradients have a particularly simple form: the difference between predicted and true probabilities. This mathematical property makes cross-entropy loss especially suitable for classification tasks, as it provides strong gradients even when predictions are far from the target.

The choice of loss function also influences other training decisions. Larger loss gradients may require smaller learning rates to prevent overshooting, while loss averaging across batches affects gradient stability and thus optimal batch size. The loss landscape's curvature shapes which optimization algorithms work best, and the loss value's trajectory determines when training has converged.

Loss functions quantify prediction error, but the error signal alone does not tell the system how to correct it. With 109,386 parameters in the MNIST network, determining which weights should change, by *how much*, and in *what* direction is intractable without an efficient credit-assignment algorithm. This is the credit assignment problem: determining which of thousands of connections contributed to the error. The next section introduces backpropagation, which solves this problem through the chain rule of calculus, systematically computing each weight's responsibility for the final prediction error.

1.4.4 Gradient computation and backpropagation

The credit-assignment problem is the missing step between measuring an error and improving the model: the system must determine which weights caused the error and how strongly each should change. Backpropagation solves that problem for neural-network training.

Definition 1.2: Backpropagation

Backpropagation is the gradient-computation algorithm training systems use to apply the chain rule to a computational graph, the recorded operations and saved intermediate values from the forward pass, computing the gradient of the loss with respect to every parameter in a single backward traversal to solve the credit assignment problem.

1. **Significance**: The backward pass costs approximately $2\times$ the FLOPs of the forward pass and requires storing all intermediate activations for the chain rule traversal. For a model with N_L layers and batch size B, activation memory scales as $\mathcal{O}(N_L \cdot B)$, making backpropagation the primary driver of the memory gap between training and inference: a model that fits on one accelerator for inference often requires multiple accelerators for training, not because of additional compute, but because of the activation storage the backward pass demands.
2. **Distinction**: Unlike numerical differentiation (which requires P perturbed forward passes for P parameters, making it $\mathcal{O}(P)$ times more expensive), backpropagation computes all P gradients in a single backward pass at $\mathcal{O}(1)\times$ the forward-pass cost, regardless of model size.
3. **Common pitfall**: A frequent misconception is that backpropagation is learning. It is a gradient computation algorithm; gradient descent performs the actual parameter update. Confusing the two obscures the systems-level separation: backpropagation determines

memory requirements (activation storage), while the optimizer determines additional state requirements (momentum, variance buffers).

A car factory gives a concrete version of the same credit-assignment problem. Vehicles pass through four stations: frame installation (A), engine mounting (B), wheel attachment (C), and final assembly (D). When inspectors find a defective car, they must determine which station caused the problem.

The solution works backward. Starting from the defect, inspectors trace responsibility through each station: how much D's assembly contributed vs. what it received from C, and how much C's work contributed vs. what came from B. Each station receives adjustment feedback proportional to its contribution. If Station B's engine mounting was the primary cause, it receives the strongest signal to change.

Backpropagation solves this credit assignment problem identically. The output layer receives direct feedback about what went wrong, calculates how its inputs contributed, and sends adjustment signals backward. Each layer receives guidance proportional to its contribution and adjusts weights accordingly—the most responsible connections making the largest adjustments.

In neural networks, each layer acts like a station on the assembly line, and backpropagation determines how much each connection contributed to the final prediction error. Translating this intuition into mathematics requires the chain rule of calculus, which provides the precise mechanism for computing each layer's contribution. In the factory analogy, "Station D's adjustment signal" corresponds to the gradient at the output layer, "proportion of contribution" maps to partial derivatives, and "sending feedback backward" describes the chain rule multiplication that propagates error signals through the network.

1.4.4.1 Backpropagation algorithm steps

While forward propagation computes predictions, backward propagation determines how to adjust weights to improve those predictions. Consider the running example where the network predicts a "three" for an image of "seven". Backward propagation provides a systematic way to adjust weights throughout the network by calculating how each weight contributed to the error.

The process begins at the network's output, where we compare predicted digit probabilities with the true label. This error then flows backward through the network, with each layer's weights receiving an update signal based on their contribution to the final prediction. The computation follows the chain rule of calculus, breaking down the complex relationship between weights and final error into manageable steps.

The mathematical foundations of backpropagation provide the theoretical basis for training neural networks, but practical implementation requires software support. Modern frameworks implement automatic differentiation systems that handle gradient computation automatically, eliminating manual derivative implementation (Wengert 1964). **?@sec-appdx-algorithm-foundations-chain-rule-automatic-differentiation-e0eb** derives the chain rule formally and shows why reverse-mode automatic differentiation computes all parameter gradients in a single backward pass, and **?@sec-appdx-algorithm-foundations-backpropagation-algorithm-93f7** walks through the backward-pass algorithm step by step and explains why it costs roughly twice the FLOPs of the forward pass. **?@sec-ml-frameworks-framework-implementation-automatic-differentiation-1407** later examines the systems engineering aspects of these frameworks. The core implementation contract is shown in algorithm 1.1: the forward pass must save the values that the backward pass will need.

Algorithm 1.1 Backpropagation training step

Require: mini-batch inputs $\mathbf{X}$, labels $\mathbf{y}$; N_L layers with parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{N_L}$; loss $\mathcal{L}$
Ensure: parameter gradients $\{\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}\}$ for the optimizer
1: $\mathbf{A}^{(0)} \leftarrow \mathbf{X}$
2: **for** $\ell = 1$ to N_L **do** ▷ *forward pass*
3: compute $\mathbf{Z}^{(\ell)}, \mathbf{A}^{(\ell)}$; save the values its derivative needs
4: **end for**
5: compute loss $\mathcal{L}(\mathbf{A}^{(N_L)}, \mathbf{y})$; seed the output gradient $\nabla_{\mathbf{A}^{(N_L)}}\mathcal{L}$
6: **for** $\ell = N_L$ down to 1 **do** ▷ *same local rule repeats*
7: use the saved forward values to compute $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}$
8: propagate $\nabla_{\mathbf{A}^{(\ell-1)}}\mathcal{L}$ to the previous layer
9: **end for**
10: **return** the accumulated parameter gradients

Saving activations in the forward loop of algorithm 1.1 is where the systems cost enters: each activation stays live until the backward pass reaches its layer, so **activation memory** scales with batch size, layer count, and activation width. This is why training can exceed inference memory even when the parameter count is unchanged.

Systems Perspective 1.5: The memory cost of backprop

In forward inference, we can discard the activations of layer ℓ as soon as layer $\ell+1$ is computed. Training is different. Because the gradient at layer ℓ depends on the activation at layer ℓ (via the chain rule), we must store every intermediate activation until the backward pass reaches that layer. Equation 1.21 captures this memory decomposition:

$$\text{Training Memory} \approx \text{Model Weights} + \text{Optimizer States} + \text{Activations} \tag{1.21}$$

For deep networks, activations dominate. Storing a batch of high-resolution images across 100 layers can consume gigabytes of accelerator memory. This capacity wall drives the need for later systems techniques that reduce, recompute, or partition training state. **?@sec-appdx-algorithm-foundations-true-cost-training-memory-e54e** provides the complete training memory equation and a worked analysis of weights, gradients, optimizer state, and activation costs.

1.4.4.2 Error signal propagation

The flow of gradients through a neural network follows a path opposite to the forward propagation. Starting from the loss at the output layer, gradients propagate backwards, computing how each layer, and ultimately each weight, influenced the final prediction error.

Consider what happens when the digit classifier misclassifies a "seven" as a "three". The loss function generates an initial error signal at the output layer, essentially indicating that the probability for "seven" should increase while the probability for "three" should decrease. This error signal then propagates backward through the network layers.

For a network with N_L layers, the gradient flow can be expressed mathematically. At each layer ℓ, we compute how the layer's output affected the final loss using the chain rule[37] in equation 1.22:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell+1)}} \frac{\partial \mathbf{A}^{(\ell+1)}}{\partial \mathbf{A}^{(\ell)}} \tag{1.22}$$

This computation cascades backward through the network, with each layer's gradients depending on the gradients from the layer above it. The process reveals how each layer's transformation contributed to the final prediction error. If certain weights in an early layer strongly influenced a misclassification, they receive larger gradient values, indicating a need for more substantial adjustment.

37 **Chain Rule**: The calculus identity $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdots \frac{\partial a_1}{\partial w}$ becomes a *product* of n terms for an n-layer network. If each partial derivative is slightly less than one, the product vanishes exponentially; if slightly greater, it explodes. This multiplicative structure is why depth is a systems constraint, not just a design choice: it dictates the numerical precision requirements and initialization strategies (for example, Glorot, He) needed to keep training stable.

This process faces challenges in deep networks. As gradients flow backward through many layers, they can either vanish or explode. When gradients are repeatedly multiplied through many layers, they can become exponentially small, particularly with sigmoid or tanh activation functions. This causes early layers to learn at negligible rates or not at all, as they receive negligible updates. Conversely, if gradient values are consistently greater than one, they can grow exponentially, leading to unstable training and destructive weight updates.

1.4.4.3 Derivative calculation process

Computing gradients involves calculating several partial derivatives at each layer: how changes in weights, biases, and activations affect the final loss. These computations follow directly from the chain rule of calculus but must be implemented efficiently for practical training.

At each layer ℓ, we compute three main gradient components. Each serves a distinct purpose in the learning process.

Weight gradients measure how changing each weight affects the final loss. These gradients tell us precisely how to adjust the connection strengths between neurons to reduce prediction errors, as shown in equation 1.23:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \mathbf{A}^{(\ell-1)^T} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{1.23}$$

Bias gradients measure how changing each bias term affects the loss. Since biases shift the activation threshold of neurons, these gradients indicate whether neurons should become more or less easily activated, as expressed in equation 1.24:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \mathbf{1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{1.24}$$

Input gradients propagate the error signal backward to the previous layer. Rather than directly updating parameters, these gradients serve as the "adjustment signals" that allow earlier layers to learn from the final prediction error, as shown in equation 1.25:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \mathbf{W}^{(\ell)^T} \tag{1.25}$$

These three gradient types interact with a practical systems constraint: the batch size that produces the most stable parameter updates is often larger than what fits in accelerator memory at once. Training a large model with a batch size of 2,048 examples is often statistically optimal for gradient stability, but storing 2,048 examples' worth of activations simultaneously exceeds available VRAM at large model scales. Engineers resolve this through **gradient accumulation**: the full batch is split into smaller *micro-batches* that each fit in memory, and the weight gradients from successive micro-batches are summed before the optimizer step fires. From the optimizer's perspective the effective batch size equals the sum across all micro-batches; from the hardware's perspective each micro-batch is an independent forward-backward pass that stays within the memory budget. This decoupling of statistical batch size from hardware memory capacity is a direct consequence of gradients being additive: because $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ summed over a batch equals the sum of per-example contributions, accumulating partial gradients and firing the update once produces numerically identical results to processing the entire batch at once.

Consider the final layer where the network outputs digit probabilities. If the network predicted $[0.1, 0.2, 0.5, \ldots, 0.05]$ for an image of "seven", the gradient flows backward through three steps:

1. Start with the error in these probabilities
2. Compute how weight adjustments would affect this error
3. Propagate these gradients backward to help adjust earlier layer weights

A minimal network makes the gradient arithmetic concrete by tracing actual values through backpropagation.

Example 1.4: Tracing gradients: A worked backpropagation example

Setup: Consider a network with two inputs, a hidden layer of two neurons (ReLU activation), and one output neuron (no activation, for simplicity). Suppose the current weights and biases are:

- **Hidden layer**: $\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$, $\mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- **Output layer**: $\mathbf{W}^{(2)} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix}$, $\mathbf{b}^{(2)} = 0$

Given input $\mathbf{x} = [1.0,\ 0.5]$ and target $y = 1.0$, we use mean squared error: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.
Forward pass (to establish the values backpropagation needs):

- Hidden preactivation:

$$\mathbf{z}^{(1)} = \mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)} = [1.0 \cdot 0.5 + 0.5 \cdot 0.8,\ 1.0 \cdot (-0.3) + 0.5 \cdot 0.2] = [0.9,\ -0.2]$$

- Hidden activation (ReLU): $\mathbf{a}^{(1)} = [\max(0, 0.9),\ \max(0, -0.2)] = [0.9,\ 0.0]$
- Output: $\hat{y} = \mathbf{a}^{(1)}\mathbf{W}^{(2)} + b^{(2)} = 0.9 \cdot 0.6 + 0.0 \cdot (-0.4) = 0.54$
- Loss: $\mathcal{L} = \frac{1}{2}(0.54 - 1.0)^2 = 0.1058$

Backward pass (applying the chain rule layer by layer):
Step 1: Output layer gradient. The loss gradient with respect to the output is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.54 - 1.0 = -0.46.$$

Step 2: Output weight gradients (applying equation 1.23). Since the output layer has no activation function

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(2)}} = -0.46, \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = {\mathbf{a}^{(1)}}^T \cdot (-0.46) = \begin{bmatrix} 0.9 \\ 0.0 \end{bmatrix} \cdot (-0.46) = \begin{bmatrix} -0.414 \\ 0.0 \end{bmatrix}$$

Step 3: Propagate to hidden layer (applying equation 1.25). The error signal sent backward is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = (-0.46) \cdot {\mathbf{W}^{(2)}}^T = (-0.46) \cdot [0.6,\ -0.4] = [-0.276,\ 0.184]$$

Step 4: Pass through ReLU. The ReLU derivative is one where $z > 0$ and 0 otherwise, so

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = [-0.276 \cdot 1,\ 0.184 \cdot 0] = [-0.276,\ 0.0].$$

The second neuron's gradient is zeroed because ReLU blocked its forward signal.
Step 5: Hidden weight gradients (applying equation 1.23):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \cdot [-0.276,\ 0.0] = \begin{bmatrix} -0.276 & 0.0 \\ -0.138 & 0.0 \end{bmatrix}$$

Weight updates (with learning rate $\eta = 0.1$): Each weight moves opposite to its gradient. For example, $W_{11}^{(1)}$ updates from 0.5 to $0.5 - 0.1 \cdot (-0.276) = 0.5276$, nudging the network toward the correct output. The second hidden neuron's weights receive zero updates for this example

because ReLU blocked its activation, illustrating the mechanism that can lead to dead neurons if it happens persistently across the training data.

Systems insight: Backpropagation is not only a calculus procedure; it is also a data-dependency graph. Training systems must preserve the forward activations needed by this backward pass, which is why activation memory becomes a first-order systems cost.

While understanding these mathematical details is essential for debugging and optimization, modern practitioners rarely implement gradients manually. The systems breakthrough lies in how frameworks automatically implement these calculations. Consider a simple operation like matrix multiplication followed by ReLU activation: `output = relu(input @ weight)`. The mathematical gradient involves computing the derivative of ReLU (0 for negative inputs, 1 for positive) and applying the chain rule for matrix multiplication. The framework records the operation in a computation graph during the forward pass, stores the pre-ReLU activations needed for gradient computation, attaches the backward rule for each operation, and schedules the reverse traversal to balance correctness, memory usage, and hardware utilization. This automation transforms gradient computation from a manual, error-prone process requiring deep mathematical expertise into a reliable system capability that enables rapid experimentation and deployment.

1.4.4.4 Computational implementation details

Activation storage is only the first backward-pass cost. As model size scales, training also adds gradient storage, optimizer-state traffic, and scheduling pressure that the forward-pass memory estimate does not capture.

Consider a larger variant of our MNIST network ($784 \to 512 \to 256 \to 10$) with a batch size of 32. Each layer's activations must be maintained until the backward pass reaches that layer:

- Input layer: 32×784 values (~100 KB using 32-bit numbers)
- Hidden layer 1: 32×512 values (~66 KB)
- Hidden layer 2: 32×256 values (~33 KB)
- Output layer: 32×10 values (~1 KB)

Beyond activations, we must store gradients for each parameter. For this larger network with approximately 535,818 parameters, gradient storage requires several megabytes. Advanced optimizers like Adam[38] roughly double this by maintaining momentum and velocity terms for every parameter.

Memory bandwidth compounds these capacity requirements. Each training step requires loading all parameters, storing gradients, and accessing activations—creating substantial memory traffic that scales with both model size and batch size. For modest networks like our MNIST example, this traffic remains manageable, but as models grow, memory bandwidth becomes the primary bottleneck, requiring specialized high-bandwidth memory systems.

The computational pattern of backward propagation follows a strict sequence: compute gradients at the current layer, update stored gradients, propagate the error signal to the previous layer, and repeat until the input layer is reached. For batch processing, these computations are performed simultaneously across all examples in the batch, enabling efficient use of matrix operations and parallel processing capabilities.

Modern frameworks handle these computations through sophisticated autograd[39] engines. Dynamic computation graphs record operations as they execute, while static computation graphs defer execution to expose more optimization opportunities. When a training script asks for gradients, the framework automatically manages memory allocation, operation scheduling, and gradient accumulation across the computation graph. The system tracks which tensors require gradients and schedules operations so that the backward pass follows the dependencies created during the forward pass. This automated management allows practitioners to focus on model design rather than the intricate details of gradient computation implementation.

38 **Adam (Adaptive Moment Estimation)**: Maintains per-parameter first and second moment estimates (momentum and velocity), requiring 2× additional memory beyond the parameters themselves (Kingma and Ba 2014). For a 100K-parameter MNIST model this overhead is negligible, but for a 7-billion parameter model it adds ~56 GB in FP32, often the difference between fitting on one GPU or needing two. Adam became the default optimizer despite this cost because it converges with minimal hyperparameter tuning.

39 **Autograd (Automatic Differentiation)**: Reverse-mode automatic differentiation traces back to Linnainmaa's work on differentiating computer programs (1970). Modern autograd engines record forward-pass operations into a directed acyclic graph (DAG), then traverse it backward using the chain rule to compute gradients automatically. The key systems trade-off is that more flexible execution captures exactly what happened in each iteration, while more fixed execution exposes more opportunities for ahead-of-time optimization. **?@sec-ml-frameworks** develops this framework design choice in detail.

Checkpoint 1.4: Backpropagation

The credit assignment problem asks which weight caused a given error. Backpropagation answers it via the chain rule; verify that the mechanism is clear:

The Mechanism

- ☐ **Chain rule**: Can you explain how equation 1.22 decomposes a complex error signal into local gradients for each layer?
- ☐ **Gradient decay**: Why do gradients often vanish in deep networks?

Training vs. Inference

- ☐ **Computational cost**: If the forward pass requires N operations, why does training require roughly $3N$ total operations?
- ☐ **Memory cost**: Why must training store every intermediate activation, and how does this scale with batch size and network depth?
- ☐ **Layer cost**: For a dense layer mapping 4,096 inputs to 4,096 outputs at batch size 32, estimate its forward FLOPs and its FP16 activation bytes to one significant figure, then say which of the two doubling the batch grows.

Backpropagation produces gradients; optimization decides how to apply them as parameter updates.

1.4.5 Weight update and optimization

Backpropagation computes *what* each weight should change, but not *how much*. The step size, the direction refinement, and the momentum across iterations are all governed by the optimizer—the algorithm that converts raw gradients into weight updates. No optimizer is universally best across all possible problems (Wolpert and Macready 1997), so neural-network training depends on choosing update rules and hyperparameters that match the model, data, and hardware constraints.

1.4.5.1 Parameter update algorithms

The optimization process adjusts network weights through gradient descent, a systematic method that uses the error signal from backpropagation to determine the direction and magnitude of each weight update.

Definition 1.3: Gradient descent

Gradient Descent is the iterative optimization algorithm that updates model parameters in the direction of the negative gradient, $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, trading computational steps (O) for loss reduction.

1. **Significance**: The optimizer choice determines the memory overhead of training. In plain FP32 terms, vanilla SGD needs each weight plus its gradient (8 bytes per parameter), while adaptive optimizers like Adam add two moment buffers per parameter, bringing training state to 16 bytes per parameter—a $4\times$ multiplier over the 4-byte inference weight that is a structural constant of the algorithm, independent of model size. Mixed-precision training rearranges these bytes across number formats rather than escaping the multiplier; **?@sec-model-training** develops that accounting. The optimizer-state overhead is the primary reason training requires more accelerators than inference for the same model.
2. **Distinction**: Unlike backpropagation, which computes the gradient (the "what to change" signal), gradient descent applies the update (the "change it" action). Backpropagation determines the memory footprint; the optimizer determines the additional state overhead.
3. **Common pitfall**: A frequent misconception is that gradient descent finds the global minimum. Neural network loss landscapes are nonconvex with many local minima, saddle points, and plateaus. In practice, stochastic gradient descent (SGD) and its variants

converge to regions of low loss that generalize well, but the path depends on the learning rate schedule, batch size, and initialization.

This iterative process calculates how each weight contributes to the error and updates parameters to reduce loss, gradually refining the network's predictive ability. The fundamental update rule combines backpropagation's gradient computation with parameter adjustment, as defined in equation 1.26:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L} \tag{1.26}$$

where θ represents any network parameter (weights or biases), η is the learning rate, and $\nabla_\theta \mathcal{L}$ is the gradient computed through backpropagation.

For the digit classifier, this means adjusting weights to improve classification accuracy. If the network frequently confuses "seven"s with "one"s, gradient descent will modify weights to better distinguish between these digits. The learning rate η[40] controls adjustment magnitude: too large values cause overshooting optimal parameters, while too small values result in slow convergence.

Despite neural network loss landscapes being highly nonconvex with multiple local minima, gradient descent reliably finds effective solutions in practice. The theoretical reasons, involving concepts like the lottery ticket hypothesis (Frankle and Carbin 2019), implicit bias (Neyshabur et al. 2017), and overparameterization benefits (Nakkiran et al. 2019), remain active research areas. For practical ML systems engineering, the key insight is that gradient descent with appropriate learning rates, initialization, and regularization consistently trains neural networks to high performance.

40 **Learning Rate**: This single scalar has an outsized impact on training infrastructure because it couples directly to batch size. Doubling the batch size (to better saturate GPU parallelism) typically requires scaling the learning rate proportionally, a relationship formalized by the linear scaling rule (Goyal et al. 2017). Misjudging this coupling is a common cause of training divergence when teams scale from single-GPU to multi-GPU setups, often misdiagnosed as a hardware or data issue rather than a hyperparameter mismatch.

1.4.5.2 Mini-batch gradient updates

Neural networks typically process multiple examples simultaneously during training, an approach known as mini-batch gradient descent. Rather than updating weights after each individual image, we compute the average gradient over a batch of examples before performing the update.

For a batch of size B, the loss gradient becomes equation 1.27:

$$\nabla_\theta \mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}_i \tag{1.27}$$

With a typical batch size of 32, the system performs one forward pass over 32 images, computes 32 per-example losses, averages their gradients, and applies a single weight update. That average smooths noisy per-example gradients, but it also means the accelerator must keep the batch's activations available until the backward pass consumes them. The choice of batch size therefore directly determines how much hardware parallelism the system can use.

Systems Perspective 1.6: Batch size and hardware utilization

Larger batches improve hardware efficiency because matrix operations can process multiple examples with similar computational cost to processing one. However, each example in the batch requires memory to store its activations, creating a fundamental trade-off: larger batches use hardware more efficiently but demand more memory. Available memory thus becomes a hard constraint on batch size, which in turn affects how efficiently the hardware can be used. This relationship between algorithm design (batch size) and hardware capability (memory) exemplifies why ML systems engineering requires thinking about both simultaneously.

1.4.5.3 Iterative learning process

The complete training process combines forward propagation, backward propagation, and weight updates into a systematic training loop. This loop repeats until the network achieves satisfactory performance or reaches a predetermined number of iterations.

A single pass through the entire training dataset is called an epoch[41]. For MNIST, with 60,000 training images and a batch size of 32, each epoch consists of 1,875 batch iterations. Algorithm 1.2 lays out the mini-batch SGD loop: each epoch shuffles the data, steps through it one mini-batch at a time with a forward and backward pass, and applies a single parameter update per batch.

41 **Epoch**: One complete pass through all training data. The number of epochs is a direct multiplier on total compute cost: a 100-epoch MNIST run executes 100× more forward and backward passes than a single epoch. At frontier scale, this multiplier becomes the binding constraint: GPT-3 trained for only ~1 epoch over 300 billion tokens because the per-epoch cost already consumed thousands of GPU-weeks.

Algorithm 1.2 Mini-batch stochastic gradient descent

Require: training set $\{(x_i, y_i)\}_{i=1}^{D}$; rate η; batch size B; epochs E; initial θ_0
Ensure: trained parameters θ

$\theta \leftarrow \theta_0$
for $e = 1$ to E **do**
  shuffle the training set ▷ *reduce order-dependent patterns*
  for each mini-batch $\mathcal{M}$ of size B **do**
    $\hat{y}_i \leftarrow f_\theta(x_i)$ for all $(x_i, y_i) \in \mathcal{M}$
    $\mathcal{L}_\mathcal{M} \leftarrow \frac{1}{B} \sum_{(x_i, y_i) \in \mathcal{M}} \mathcal{L}(\hat{y}_i, y_i)$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_\mathcal{M}$ ▷ *backpropagate, then update*
  end for
  evaluate on validation; stop if the convergence criterion is met
end for
return θ

The mini-batch loop makes the batch size B the hardware unit of work: a larger B improves matrix utilization and accelerator occupancy but raises activation memory, and because the update fires once per batch rather than once per example, B also sets the gradient-update cadence, until capacity, bandwidth, or convergence becomes the limit. The inner loop in algorithm 1.2 is the unit that the hardware repeatedly executes. Forward propagation creates activations for every example in the current mini-batch, backward propagation consumes those activations to compute gradients, and the update mutates the parameters once per batch rather than once per example. During training, we monitor several key metrics: training loss tracks the average loss over recent batches, validation accuracy measures performance on held-out test data, and learning progress indicates how quickly the network improves. For our digit recognition task, we might observe accuracy climb from 10 percent (random guessing) to over 95 percent through multiple epochs of training.

1.4.5.4 Convergence and stability considerations

A network that achieves 99.5 percent accuracy on training data but only 85 percent on new data has not learned the underlying patterns—it has memorized the training set. This failure mode, overfitting, is the central risk in practical training.

Definition 1.4: Overfitting

Overfitting is an ML-system generalization failure caused by memorizing Noise instead of Signal.

1. **Significance**: The diagnostic signature is measurable: training accuracy climbs toward 99+ percent while validation accuracy plateaus or declines, creating a widening train-test gap. A model with P parameters can memorize a dataset of D examples when $P \gg D$: the model has enough capacity to assign a unique internal representation to each training sample rather than learning the underlying distribution. The gap between training and validation error is the quantitative measure of overfitting severity.
2. **Distinction**: Unlike underfitting (where the model lacks the capacity to capture the target function and both training and validation error remain high), overfitting produces low training error but high validation error, indicating that the model has specialized to the training sample rather than learning the underlying distribution.
3. **Common pitfall**: A frequent misconception is that overfitting is "solved" by more data. Adding data helps only when the model's capacity is appropriate for the expanded dataset; without regularization (dropout, weight decay, early stopping), larger models will memorize even large datasets, achieving near-zero training loss while validation performance stagnates.

Learning rate selection is the single most consequential hyperparameter in training. For our MNIST network, the choice of learning rate dramatically influences the training dynamics. A large learning rate of 0.1 might cause unstable training where the loss oscillates or explodes as weight updates overshoot optimal values. Conversely, a learning rate of 0.0001 might result in extremely slow convergence, requiring many more epochs to achieve good performance. A moderate learning rate of 0.01 often provides a good balance between training speed and stability, allowing the network to make steady progress while maintaining stable learning.

Convergence monitoring provides essential feedback during training and continues into production deployment, as covered in **?@sec-ml-operations**. As training progresses, the loss value typically stabilizes around a particular value, indicating the network is approaching a local optimum. Validation accuracy often plateaus as well, suggesting the network has extracted most learnable patterns from the data. The gap between training and validation performance reveals whether the network is overfitting or generalizing well to new examples. The interplay between batch size, available memory, and computational resources requires careful balancing to achieve efficient training within hardware constraints, the same memory-computation trade-offs established in the preceding backpropagation section.

Detecting the optimal stopping point requires periodically evaluating the model on the validation set and saving its weights to durable storage, a practice called **checkpointing**. For a small model like the MNIST network, writing a checkpoint costs a negligible amount of time. For a model whose parameters occupy tens to hundreds of gigabytes, a checkpoint write serializes all of those bytes to disk or object storage, stalling the training loop for seconds to minutes while the I/O completes. Running validation itself requires a full forward pass over the held-out dataset, which at large scale consumes meaningful accelerator time that would otherwise go to training. Checkpointing and validation are therefore not free statistical observations: they impose a recurring I/O and compute tax whose frequency must be tuned against the risk of missing the generalization peak. Saving too infrequently risks losing the best weights to a subsequent degradation epoch; saving too frequently stresses storage bandwidth and adds overhead. This is why convergence monitoring, while conceptually straightforward, becomes a systems infrastructure problem at scale.

Checkpoint 1.5: Neural network learning process

You have now covered the complete training cycle, the mathematical machinery that enables neural networks to learn from data. Before moving to inference and deployment, verify your understanding:

Use the MNIST network ($784 \to 128 \to 64 \to 10$) and a batch size of 32 as the running check.

- ☐ **Forward pass**: Can you trace the forward pass using $\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$ and $\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$?
- ☐ **Cross-entropy loss**: Can you explain what cross-entropy loss measures and why batch loss is averaged across examples?
- ☐ **Backward pass**: Can you trace how gradients flow backward through the network and why stored activations are needed?
- ☐ **Update rule**: Can you write the gradient descent update rule and explain how batch size trades memory, throughput, and gradient stability?
- ☐ **Training memory**: Can you explain why the full training loop requires more memory than inference?

If any concepts feel unclear, review the earlier sections on Forward Propagation, Loss Functions, Backward Propagation, or the Optimization Process. These mechanisms form the foundation for understanding the training-vs.-inference distinction we explore next.

The complete training pipeline, from forward propagation through loss computation to gradient-based weight updates, is now established. Training, however, is preparation, not the end goal. The following checkpoint consolidates that learning process before we examine what happens when a trained model must answer queries in production.

1.5 Inference Pipeline

Training transforms randomly initialized weights into parameters that encode meaningful patterns, but training is preparation, not the end goal. The inference[42] phase renegotiates the silicon contract: the same mathematical operators now face different hardware constraints—latency budgets instead of throughput targets, milliwatt power envelopes instead of kilowatt racks, and edge devices instead of GPU clusters. Understanding how the contract changes between training and inference is essential for practical systems design.

1.5.1 Production deployment and prediction pipeline

A model that achieved 99 percent accuracy on the test set produces nonsensical outputs three months after deployment, yet no code has changed. The weights are frozen, the architecture is identical, and the inference pipeline runs without error. The problem is that the world moved while the model stood still.

The transition from training to inference introduces a constraint on model adaptability that fundamentally shapes system design. Trained models generalize to unseen inputs through learned statistical patterns, but parameters remain fixed throughout deployment. Once training concludes, the model applies its learned probability distributions without modification. When operational data distribution diverges from training distributions, the model continues executing its fixed computational pathways regardless of this shift. Consider an autonomous vehicle perception system: if construction zone frequency increases substantially or novel vehicle configurations appear in deployment, the model's responses reflect statistical patterns learned during training rather than adapting to the evolved operational context. Adaptation in ML systems emerges not from runtime model modification but from systematic retraining with updated data, a deliberate engineering process detailed in **?@sec-model-training**.

1.5.1.1 Operational phase differences

Neural network operation divides into two distinct phases with markedly different computational requirements. Figure 1.18 contrasts these phases visually. Inference performs only the forward pass, processing inputs through the learned weights with batch sizes that vary according to demand. Training adds the backward pass for gradient computation and parameter updates, requires larger fixed batches to stabilize gradient estimates, and must store activations, gradients, and optimizer state simultaneously, consuming significantly more memory. The network architecture is identical in both phases; the difference lies entirely in computational and memory orchestration.

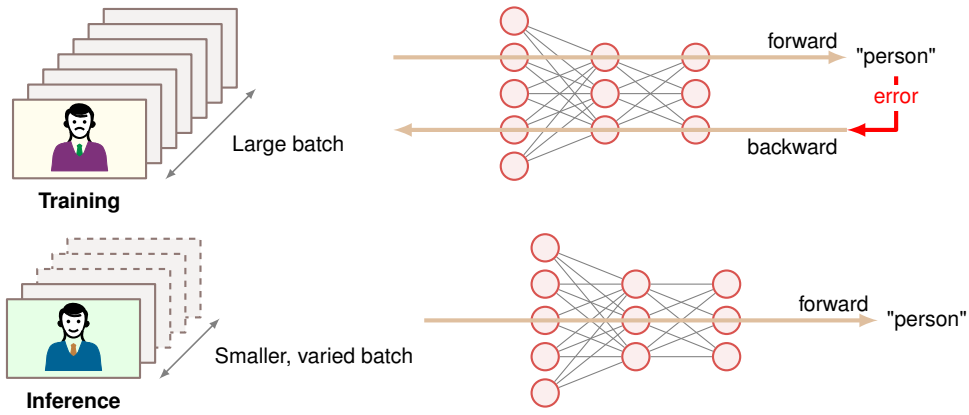


Figure 1.18: Inference vs. Training Flow: During inference, neural networks use learned weights for forward pass computation only, simplifying the data flow and reducing computational cost compared to training, which requires both forward and backward passes for weight updates. This streamlined process enables efficient deployment of trained models for real-time predictions.

These computational differences manifest directly in hardware requirements and deployment strategies. Training environments typically employ high-memory accelerators[43] with substantial

42 **Inference**: From Latin *inferre* ("to bring in, to conclude"), borrowed from logic where it means deriving conclusions from premises. The ML usage marks a sharp systems boundary: training optimizes weights using forward and backward passes with gradient storage, while inference executes only the forward pass with frozen parameters. This distinction halves or quarters memory requirements and eliminates the need for gradient computation, fundamentally changing which hardware is viable, from 400 W data center GPUs to 2 W edge accelerators.

43 **Training GPU Power Budget**: The "high-memory" requirement is driven by the need to hold parameters, gradients, optimizer state, and activations simultaneously. The corresponding power draw dictates the "substantial cooling infrastructure," as a single high-end training GPU consumes 400 W–700 W. Even compared with a 4 W mobile inference chip, that is at least 100× the power budget.

cooling infrastructure. Inference deployments on constrained hardware prioritize latency and energy efficiency across diverse platforms: mobile devices use low-power neural processors (typically 2–4 W), edge servers use specialized inference accelerators[44], and cloud services often use reduced numerical precision for increased throughput[45]. Production inference systems serving millions of requests daily require infrastructure concerns, such as request routing and failure handling, that are usually absent from a single training run.

Training preserves activations for backpropagation, while inference releases layer buffers as soon as possible; table 1.12 turns that difference into a resource profile.

Table 1.12: Training vs. Inference Forward Pass: Although both phases execute identical mathematical operations layer-by-layer, they differ fundamentally in memory management. Training must preserve all intermediate activations for gradient computation during the backward pass; inference can discard each layer's outputs immediately after computing the next layer, enabling aggressive memory optimization. This distinction explains why training typically requires several times more memory than inference for the same model; the MNIST worked example below is about 4.3×.

Characteristic	Training Forward Pass	Inference Forward Pass
Activation Storage	Maintains complete activation history for backprop	Retains only current layer activations
Memory Pattern	Preserves intermediate states throughout forward pass	Releases memory after layer computation completes
Computational Flow	Structured for gradient computation preparation	Optimized for direct output generation
Resource Profile	Higher memory requirements for training operations	Minimized memory footprint for efficient execution

1.5.1.2 Memory and computational resources

Neural networks consume computational resources differently during inference than during training. Inference has two memory obligations. The first is persistent: the trained weights and biases must remain available for every request. The second is transient: each layer produces an activation buffer that is needed only until the next layer consumes it. This distinction is the reason inference can be much leaner than training, even though the arithmetic in the forward pass is the same.

For our canonical MNIST network (784 → 128 → 64 → 10), the persistent parameter block contains 109,386 parameters, or about 438 KB at 32-bit floating point precision[46]. The layer-level counts in table 1.13 show why: each fully connected layer performs one multiply-add for each weight, so parameter memory and arithmetic scale together.

Table 1.13: MNIST Inference Resources by Layer: The persistent parameter block and multiply-add count for the canonical MNIST network. In fully connected layers, the same weights that occupy memory also determine the arithmetic work per inference.

Layer	Weights	Biases	Multiply-Adds
Layer 1	$784 \times 128 = 100{,}352$	128	100,352
Layer 2	$128 \times 64 = 8{,}192$	64	8,192
Output	$64 \times 10 = 640$	10	640
Total	109,386 parameters	Included in total	109,184

The transient side is smaller. A single image starts as 784 input values, then produces layer outputs of 128, 64, and 10 values. If counted naively, that is 986 activation values, or about 4 KB at FP32. In a real inference engine, those values do not all need to persist at once. Once Layer 2 has consumed Layer 1's output, the Layer 1 buffer can be reused; once the output layer has consumed Layer 2's activations, that buffer can be reused as well. Training cannot discard those intermediates because backpropagation needs them later. The training memory estimate therefore has distinct terms: parameters, gradients, optimizer state, and batch-scaled saved activations, all multiplied by their

44 **Edge Inference Accelerators**: The Edge TPU (Google Coral) operates in the mobile/embedded tier at about 2 W, delivering 4 TOPS through an INT8 datapath. Edge servers sit in a different tier: Jetson AGX Orin reaches 275 TOPS at 15 W–60 W, about 68.8× more throughput but with wired-power assumptions.

45 **Quantization**: Reducing numerical precision from 32-bit values to INT8 yields 4× less memory per parameter and up to 4× higher throughput on hardware with INT8 datapaths. Trained models often tolerate some precision loss during inference because inference does not accumulate rounding errors across gradient updates the way training does. The trade-off is not free: overly aggressive quantization, especially below four bits, can degrade accuracy on tasks sensitive to fine-grained distinctions. See **?@sec-model-compression** for quantization techniques and **?@sec-hardware-acceleration** for hardware support.

46 **FP32 (Single Precision)**: The IEEE 754 standard (1985) format using 32 bits (1 sign, 8 exponent, 23 mantissa) for each parameter. FP32 is the default for training because its dynamic range accommodates gradient magnitudes spanning many orders of magnitude. Halving to FP16 or BF16 ("brain floating point," developed at Google Brain) saves 2× memory and doubles throughput on hardware with 16-bit datapaths; further reduction to INT8 yields 4× savings but requires posttraining calibration. See **?@sec-appdx-machine-foundations-numerical-representations-c889** for a detailed comparison of numerical formats and their precision-throughput trade-offs.

numerical precision. That is why gradient storage and backpropagation overhead multiply training resource demands by 4.3× or more for this worked example (see section 1.3.4.3).

This persistent-plus-rolling-buffer model also explains the deployment optimizations that follow. Batching increases arithmetic reuse and hardware occupancy but expands activation storage. Lower numerical precision can shrink the persistent parameter block and activation buffers, but only if accuracy survives the smaller representation. Hardware-specific layouts improve cache reuse by keeping the rolling buffer close to the compute units. The predictable, streamlined nature of inference enables these optimizations precisely because parameters are fixed and activations have short lifetimes.

1.5.1.3 Performance enhancement techniques

The fixed nature of inference computation presents optimization opportunities unavailable during training. Once parameters are frozen, the predictable computation pattern allows systematic improvements in both memory usage and computational efficiency.

Batch size selection represents a key inference trade-off. During training, large batches stabilized gradient computation, but inference offers more flexibility. Processing single inputs minimizes latency, making it ideal for real-time applications requiring immediate responses. Batch processing, however, improves throughput by using parallel computing capabilities more effectively. For our MNIST network, processing a single image requires storing 202 layer-output activation values (986 values including the input buffer), while a batch of 32 requires 6,464 layer-output activation values but can process more images per unit time on parallel hardware.

Memory management during inference is far more efficient than during training. Since intermediate values serve only forward computation, memory buffers can be reused aggressively. Activation values from each layer need only exist until the next layer's computation completes, enabling in-place operations that reduce the total memory footprint. The fixed nature of inference allows precise memory alignment and access patterns optimized for the underlying hardware architecture.

Hardware-specific optimizations become particularly important during inference. On CPUs, computations can be organized to maximize cache utilization and exploit SIMD parallelism. Accelerator deployments benefit from optimized matrix multiplication routines and efficient memory transfer patterns. These optimizations extend beyond computational efficiency to reduce power consumption and improve hardware utilization, critical factors in real-world deployments.

The predictable nature of inference also enables optimizations like reduced numerical precision. While training typically requires enough floating-point precision to maintain stable learning, inference can often operate with reduced precision while maintaining acceptable accuracy. For our MNIST network, such optimizations could halve the memory footprint with corresponding improvements in computational efficiency.

These optimization principles, while illustrated through our simple MNIST feedforward network, represent only the foundation of neural network optimization. More sophisticated architectures introduce additional considerations and opportunities, including specialized designs for spatial data, sequences, and context-dependent computation. These architectural variations and their optimizations are explored in **?@sec-network-architectures** and **?@sec-model-compression**. Production deployment considerations, including batching strategies and runtime optimization, are covered in **?@sec-model-serving-throughput-optimization-18d1** and **?@sec-ml-operations**.

1.5.2 Output interpretation and decision making

Neural network outputs become useful only after they are converted back into decisions a conventional system can act on. Preprocessing bridges real-world data into tensor form; postprocessing maps neural outputs back into labels, confidence thresholds, validation logic, error handling, and downstream messages. In the MNIST running example, logits are not enough: a digit-recognition system needs the most likely digit, a confidence score, and a route for uncertain cases to human review or a secondary recognizer.

Those steps have a different performance shape from the forward pass. Inference benefits from batched matrix operations on accelerators, while thresholding, formatting, validation, and exception handling often run as sequential CPU logic. If that surrounding code is ignored, preprocessing and postprocessing can dominate end-to-end latency even when the neural network itself is fast.

The complete neural network lifecycle, from architecture design through training to inference deployment, now sits in the toolkit as a set of mathematical operations with quantifiable resource costs. These operations have so far lived in the controlled environment of our MNIST running example, where data is clean, latency is unconstrained, and hardware is unchallenged. Real production systems face all of these pressures simultaneously. Before the historical case study shows how the pieces fit together in one of the earliest large-scale neural network deployments, pause to consolidate how the components integrate.

Checkpoint 1.6: Complete neural network system

Before examining how these concepts integrate in a real-world deployment, verify your understanding of the complete neural network lifecycle:

Use the MNIST classifier ($784 \to 128 \to 64 \to 10$) as the running production system.

- ☐ **System lifecycle**: Can you trace the lifecycle from architecture design to training loop to trained model to inference deployment?
- ☐ **Training vs. inference memory**: Can you explain why training requires roughly 4.3× more memory than inference, naming the gradients, optimizer state, and activation terms?
- ☐ **End-to-end trace**: Can you trace one digit image through preprocessing, forward propagation, output probabilities, postprocessing, and final prediction?
- ☐ **Bottleneck spotting**: Can you identify where bottlenecks might appear in a real-time system processing 100 images per second?
- ☐ **Human-in-the-loop**: Can you explain how confidence thresholds, validation, and monitoring determine when human intervention is needed?

The USPS case study shows architectural choices, training strategies, and deployment constraints combining into a working ML system.

1.6 USPS Digit Recognition

In the early 1990s, postal and financial institutions needed to read handwritten digits at industrial scale. LeCun and colleagues demonstrated backpropagation-based recognition of USPS ZIP-code digits (LeCun et al. 1989), and later described LeNet-style convolutional document recognition, deployed check-reading systems, and the MNIST benchmark used throughout this chapter (LeCun et al. 1998). This case study gives concrete form to every operation from this chapter: preprocessing normalizes varying handwriting, the neural network performs forward propagation through learned weights, confidence thresholds implement postprocessing logic, and the complete pipeline must coordinate with downstream sorting decisions. The engineering principles it illustrates (robust preprocessing, confidence-based routing, and end-to-end pipeline optimization) remain the template for production ML systems three decades later.

1.6.1 The mail sorting challenge

The United States Postal Service (USPS) operated at national mail scale, with large daily volumes requiring accurate routing based on handwritten ZIP codes. In the early 1990s, human operators still handled many hard-to-read cases, making automation of handwritten digit recognition an important operational target. Automating this process through neural networks represented an early, successful large-scale deployment path for applied machine learning (LeCun et al. 1998).

The complexity of this task becomes evident: a ZIP code recognition system must process images of handwritten digits captured under varying conditions. The samples in figure 1.19 show the wide variation in writing styles, pen types, stroke thickness, and character formation that the system must handle. The system must make accurate predictions quickly enough to maintain mail processing speeds, yet errors in recognition can lead to significant delays and costs from misrouted mail. This real-world constraint meant the system needed both high accuracy and reliable measures of prediction confidence to identify when human intervention was necessary.

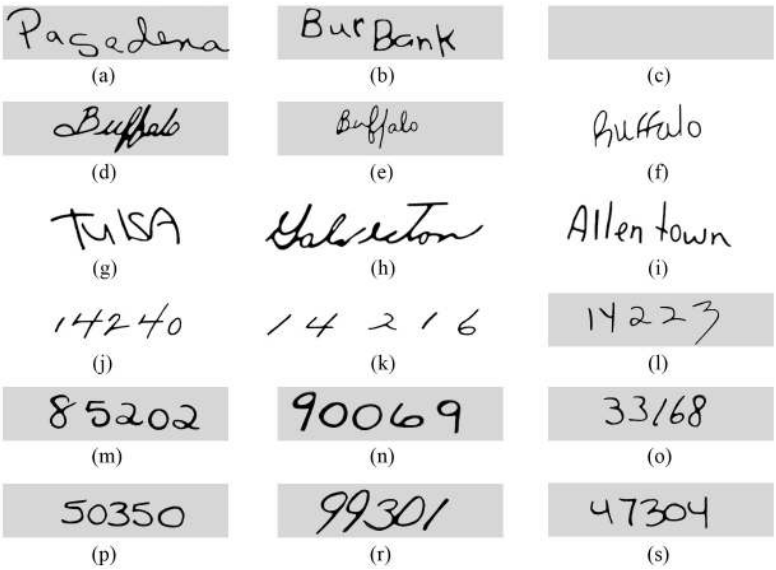


Figure 1.19: Handwritten Address Variability: Real-world USPS mail samples, including handwritten city names (Pasadena, BurBank, Buffalo variants, TULSA, Galveston, Allentown) and five-digit ZIP codes, exhibit significant variations in stroke width, slant, and character formation. These examples demonstrate the need for effective feature extraction and model generalization to achieve high accuracy in optical character recognition (OCR) tasks.

The challenging environment imposed requirements spanning every aspect of neural network implementation discussed in this chapter. Success depended on the entire pipeline from image capture through final sorting decisions, with the neural network's accuracy as only one factor among many.

1.6.2 Engineering process and design decisions

Recognizing a handwritten "seven" on a white envelope is straightforward. Recognizing it on a crumpled package with coffee stains, ballpoint smudges, and overlapping address lines requires engineering decisions at every stage from data collection to deployment.

Data collection presented the first major challenge—and a concrete instance of the data pipeline principles covered in **?@sec-data-engineering**. Unlike controlled laboratory environments, postal facilities processed mail with tremendous variety. The training dataset had to capture this diversity: digits written by people of different ages, educational backgrounds, and writing styles; envelopes in varying colors and textures; and images captured under different lighting conditions and orientations. The data quality, labeling consistency, and distribution coverage that **?@sec-data-engineering** emphasizes were not abstract concerns here; they directly determined whether the system could handle a hurried scrawl as reliably as a carefully printed digit. This extensive data collection effort later contributed to the creation of the MNIST database (LeCun et al. 1998) used throughout our examples.

Network architecture design required balancing multiple constraints. Deeper networks achieve higher accuracy but also increase processing time and computational requirements. Processing 28×28 pixel images of individual digits had to complete within strict time constraints while running reliably on available hardware, maintaining consistent accuracy from well-written digits to hurried scrawls.

Training introduced additional complexity. The system needed high accuracy across real-world handwriting styles, not merely on a curated test dataset. Careful preprocessing normalized input images for variations in size and orientation. Data augmentation techniques, a form of the data transformation strategies discussed in **?@sec-data-engineering**, increased training sample variety. The team validated performance across different demographic groups and tested under actual operating conditions, following the kind of systematic evaluation workflow described in **?@sec-ml-workflow**.

The engineering team faced a critical decision regarding confidence thresholds. Setting these thresholds too high would route too many pieces to human operators, defeating the purpose of automation. Setting them too low would risk delivery errors. The operating rule is an expected-cost decision: choose threshold t to minimize manual_rate$(t)C_{\text{manual}}$ + error_rate$(t)C_{\text{misroute}}$ subject to throughput and latency constraints. The solution emerged from analyzing the confidence distributions of correct vs. incorrect predictions, establishing thresholds that balanced automation rate against error rate while maintaining acceptable accuracy.

1.6.3 Production system architecture

Following a single piece of mail through the USPS recognition system illustrates how the concepts in this chapter integrate into a complete solution. The journey from physical mail to sorted letter demonstrates the interplay between traditional computing, neural network inference, and physical machinery. Trace the data flow in figure 1.20 to see this hybrid architecture in action, with the neural network operating as one component within a broader pipeline of conventional preprocessing and postprocessing stages.

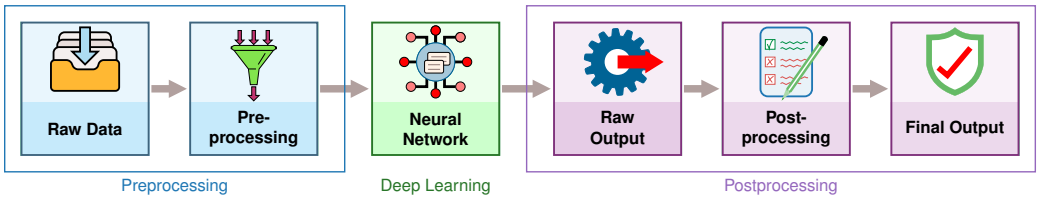


Figure 1.20: USPS Inference Pipeline: The mail sorting pipeline combines traditional preprocessing (blue) with neural network inference (green) and traditional postprocessing (purple). Raw envelope images undergo preprocessing, including thresholding, segmentation, and normalization, before the neural network classifies individual digits. Postprocessing applies confidence thresholds and formats sorting instructions for the physical sorting machinery.

The process begins when an envelope reaches the imaging station. High-speed cameras capture the ZIP code region at rates exceeding ten pieces per second—a pace that leaves no room for manual intervention. This image acquisition must adapt to varying envelope colors, handwriting styles, and lighting conditions while maintaining consistent quality despite motion blur.

Once captured, the raw images are far from ready for neural network processing. Preprocessing transforms these camera images into a standardized format. The system must locate the ZIP code region, segment individual digits, and normalize each digit image. This stage employs traditional computer vision techniques: image thresholding adapts to envelope background color, connected component analysis identifies individual digits, and size normalization produces standard 28×28 pixel images. Speed remains critical; these operations must complete within milliseconds to maintain throughput.

The neural network then processes each normalized digit image. The original 1989 system used an early LeNet variant (LeCun et al. 1989) with approximately 10,000 parameters—remarkably compact compared to our running example's 109.4K. The network processes each digit through multiple layers, ultimately producing ten output values representing digit probabilities. This inference process, while computationally intensive by 1990s standards, benefits from the optimization principles we discussed in the previous section.

Postprocessing converts these neural network outputs into sorting decisions. The system applies confidence thresholds to each digit prediction. A complete ZIP code requires high confidence in all five digits; a single uncertain digit flags the entire piece for human review. When confidence meets thresholds, the system transmits sorting instructions to mechanical systems that physically direct the mail to its appropriate bin.

The entire pipeline operates under strict timing constraints. From image capture to sorting decision, processing must complete before the mail piece reaches its sorting point. The system maintains multiple pieces in various pipeline stages simultaneously, requiring careful synchronization between computing and mechanical systems. This real-time operation illustrates why the optimizations we discussed in inference and postprocessing become essential in practical applications.

1.6.4 Performance outcomes and operational impact

Neural-network-based digit recognition reached about 1 percent digit error, processed 10–30 digits per second, and rejected uncertain cases for human handling in the LeNet deployment literature. Those concrete deployment metrics changed how document and mail-processing systems handled handwritten numerals while also exposing the limits of neural networks in high-volume applications. Table 1.14 summarizes representative performance metrics from that literature (LeCun et al. 1998).

Table 1.14: USPS LeNet deployment results: Representative LeNet-style deployment metrics reported in the document-recognition literature. The key systems point is not the exact percentage in isolation, but the operating trade-off among error rate, rejection rate, throughput, and human review (LeCun et al. 1998).

Metric	Neural Network	Human Operators
Error rate	1%	2.5%
Rejection rate	9%	N/A
Throughput	10–30 digits/sec	~1 digit/sec
Model parameters	~10,000	N/A
Training time	3 days (Sun-4/260)	N/A
Training epochs	23	N/A

The numbers tell a deployment story rather than an accuracy story. The neural network lowered automated recognition error while preserving a rejection path for uncertain digits, and by the late 1990s LeNet-based systems were reading millions of checks per day at financial institutions, with related ZIP-code recognizers proving neural networks viable for mission-critical, high-volume postal automation.

Systems lesson: The rejection path, not the raw error rate, is what made the deployment work. Uncertain cases went to human operators rather than forcing every example through the model, so the system bought reliability by choosing where to stop trusting the network.

Performance metrics validated many of the principles developed earlier in the chapter. The system achieved its highest accuracy on clearly written digits similar to those in the training data, but performance varied with real-world factors: lighting conditions affected preprocessing effectiveness, unusual writing styles occasionally confused the neural network, and environmental vibrations degraded image quality. These challenges led to continuous refinements in both the physical system and the neural network pipeline.

The economic impact came from shifting the operating point, not from eliminating people entirely. Automation handled high-confidence digits, while human operators handled uncertain cases and maintained system performance. This hybrid approach, combining artificial and human intelligence, became a model for subsequent automation projects.

The system also revealed important lessons about deploying neural networks in production. Training data quality proved essential: the network performed best on digit styles well-represented in its training set—a direct validation of the data quality principles established in **?@sec-data-engineering**. Regular retraining helped adapt to evolving handwriting styles, embodying the iterative lifecycle that **?@sec-ml-workflow** formalized. Maintenance required both hardware specialists and deep learning experts, introducing new operational considerations. These insights influenced subsequent neural network deployments across industrial applications.

1.6.5 Key engineering lessons and design principles

The USPS ZIP code recognition system's central lesson is that neural computation succeeds only when the surrounding pipeline shares the same operating constraint. Preprocessing, inference, postprocessing, and physical sorting all had to support national-scale throughput with bounded error.

The system's development shows why understanding both theoretical foundations and practical considerations matters. While the biological visual system processes handwritten digits effortlessly, translating this capability into an artificial system required careful consideration of network architecture, training procedures, and system integration.

The success of this early large-scale neural network deployment helped establish many practices we now consider standard: the importance of thorough training data, the need for confidence metrics, the role of pre- and postprocessing, and the critical nature of system-level optimization. These operational considerations are formalized in **?@sec-ml-operations**, which covers production ML system maintenance and monitoring. The comparison with a modern embedded implementation separates what changed in the machine from what stayed fixed in the algorithm.

Example 1.5: Then vs. now: USPS on modern hardware

The same neural network computation that required industrial-scale infrastructure in 1990 runs on pocket-sized devices today. Table 1.15 quantifies four decades of progress, separating what changed in the surrounding machine from what stayed fixed in the algorithm. The unchanged model-storage row matters as much as the dramatic hardware rows.

Table 1.15: Hardware Progress for Neural Network Computation: The same LeNet computation that required a workstation-class system in 1990 can now run on inexpensive embedded hardware—about 1,000× cheaper, 1,000× faster, and 20,000× more energy-efficient under the illustrative assumptions in the table. Crucially, the algorithm is unchanged; the main improvement came from hardware and systems implementation. This validates the systems principle that algorithm-hardware co-design multiplies gains across both dimensions.

Aspect	1990s USPS System	Modern Equivalent	Improvement
Hardware cost	about $50,000	about $50	1,000×
Inference latency	~100 ms/digit	~0.1 ms/digit	1,000×
Power consumption	50 W–100 W	5 W	10–20×
Training time	3 days	~30 seconds	8,640×
Model storage	~40 KB	~40 KB (unchanged)	1× (same model)
Energy/inference	~10 J	~0.5 mJ	20,000×
Cost/inference	~$0.001	~$0.000001	1,000×

Hardware improved dramatically across the table, ranging from 10–20× lower power to 20,000× lower energy per inference, while LeNet's architecture and model storage remain essentially unchanged. This is **algorithm-machine co-design** at work: improvements in either dimension multiply together. What the table does not change is the engineering problem. Modern smartphone OCR still requires preprocessing for lighting variation, confidence thresholds for uncertain predictions, and fallback to human review for edge cases, and the USPS system's architecture (capture, preprocess, inference, postprocess, action) remains the template for every production ML pipeline.

Systems lesson: Today's smartphones run real-time neural networks for face recognition, language translation, and voice assistants, tasks that would have required far larger infrastructure in the 1990s. Forty years of hardware progress did not change the algorithm or the pipeline shape; it moved the same computation from one postal facility to billions of devices.

While hardware efficiency improved by orders of magnitude, modern edge AI systems face even tighter constraints than the USPS deployment: milliwatt power budgets vs. watts, millisecond latency requirements vs. tens of milliseconds, and deployment on battery-powered devices vs. dedicated infrastructure. Yet the same engineering principles apply—preprocessing for real-world variation, confidence-based routing to human review, and end-to-end pipeline optimization. This historical case study provides a reusable template for reasoning about ML systems deployment across the entire spectrum from cloud to edge to tiny devices. The operational considerations demonstrated here are formalized in **?@sec-ml-operations**.

The USPS case is not unique; the alignment pattern it exhibits is a recurring property of every successful deep learning deployment. The next section formalizes it.

1.7 D·A·M Taxonomy

The USPS system succeeded because three dimensions aligned: LeNet's architecture matched the digit recognition task (Algorithm), diverse handwriting samples captured real-world variation (Data), and specialized hardware met latency constraints (Machine). This alignment was not coincidental; it reflects the D·A·M taxonomy. **?@sec-appdx-dam-taxonomy** formalizes how each component constrains and enables the others.

Forward propagation, activation functions, backpropagation, and gradient descent define the algorithmic core of deep learning systems. The architecture choices we make (layer depths, neuron counts, connection patterns) directly determine the computational complexity, memory requirements, and training dynamics. Each activation function selection, from ReLU's computational efficiency to sigmoid's saturating gradients, represents an algorithmic decision with profound systems implications. The hierarchical feature learning that distinguishes neural networks from classical approaches emerges from these algorithmic building blocks, but success depends critically on the other two triangle components.

Learning depends entirely on labeled data to calculate loss functions and guide weight updates through backpropagation. Our MNIST example demonstrated how data quality, distribution, and scale directly determine network performance: the algorithms remain identical, but data characteristics govern whether learning succeeds or fails. The shift from manual feature engineering to automatic representation learning does not eliminate data dependency; it transforms the challenge from designing features to curating datasets that capture the full complexity of real-world patterns. Preprocessing, augmentation, and validation strategies become algorithmic design decisions that shape the entire learning process.

The machine component manages the massive number of matrix multiplications required for forward and backward propagation, revealing why specialized hardware became essential for deep learning success. Memory bandwidth limitations, parallel computation patterns that favor GPU architectures, and the different computational demands of training vs. inference all stem from the mathematical operations at the core of neural networks. The evolution from CPUs to GPUs to specialized AI accelerators directly responds to the computational patterns inherent in neural network algorithms. Understanding these mathematical foundations enables engineers to make informed decisions about hardware selection, memory hierarchy design, and distributed training strategies.

The interdependence of these three components is the central lesson: algorithms define what computations are necessary, data determines whether those computations can learn meaningful patterns, and machines determine whether the system can execute efficiently at scale. Neural networks succeeded not because any single component improved, but because advances in all three areas aligned. More sophisticated algorithms, larger datasets, and specialized hardware created a synergistic effect that transformed artificial intelligence.

The D·A·M perspective explains why deep learning engineering requires systems thinking that extends well beyond traditional software development. Optimizing any single axis without considering the others leads to suboptimal outcomes: the most elegant algorithms fail without quality data, the best datasets remain unusable without adequate machines, and machines with the largest memory and compute budgets achieve nothing without algorithms that can learn from data. When performance stalls, the diagnostic question is *where* the flow is blocked—check the D·A·M.

These foundations equip engineers to reason about neural networks from first principles. Yet conceptual understanding alone is insufficient: practitioners must also recognize the recurring misconceptions that derail real-world projects.

1.8 Fallacies and Pitfalls

Intuitions from traditional software (that bugs are deterministic, that more resources always help, that code inspection reveals problems) fail when applied to statistical learning systems. The following fallacies and pitfalls cause teams to misallocate effort, deploy inappropriate solutions, or encounter production failures that could have been avoided.

Fallacy: *Neural networks are "black boxes" that cannot be understood or debugged.*

Engineers assume neural networks lack the transparency of traditional code. In practice, networks are interpretable through statistical methods: activation visualization reveals learned patterns, gradi-

ent analysis quantifies input sensitivity (saliency maps identify which of 784 pixels most influenced a digit classification), and ablation studies isolate component contributions. For the MNIST classifier in section 1.3.2, visualizing first-layer weights can show edge-like detectors emerging automatically, a pattern also visible in early convolutional vision models (Krizhevsky et al. 2012). Teams expecting line-by-line debugging waste time searching for "bugs" in correctly functioning statistical systems. The perceived opacity stems from applying wrong analysis paradigms to probabilistic pattern recognition.

Pitfall: *Discarding domain expertise because a deep model is available.*

Teams assume automatic feature learning removes the need for domain knowledge. Successful systems require domain expertise at every stage: architecture selection, training objective design, dataset curation, and output interpretation. The USPS-style system in section 1.6 succeeded because engineers specified confidence thresholds based on operating economics, routing uncertain cases to human operators. Without domain knowledge, teams can deploy networks that look strong on a test set but fail in production because their thresholds, fallback paths, or error costs are wrong.

Fallacy: *Deeper networks are always more accurate than wider ones.*

Engineers assume that stacking more layers is the primary path to higher accuracy, since depth enables hierarchical feature extraction. In practice, depth alone encounters diminishing returns. ResNet showed that very deep residual networks can train effectively, but its ImageNet results also make clear that more layers must be weighed against added training and inference cost (He et al. 2016). EfficientNet later demonstrated that compound scaling of width, depth, and input resolution can outperform depth-only scaling at a given resource budget (Tan and Le 2019). The lesson is not that depth is bad; it is that teams should profile capacity utilization and scale the architecture dimension that gives the best accuracy per FLOP.

Pitfall: *Using neural networks for problems solvable with simpler methods.*

Teams assume deep learning always performs better. Logistic regression training in 10 ms often outperforms a neural network requiring two hours when data contains fewer than 1,000 examples or relationships are approximately linear. If logistic regression achieves 94 percent accuracy, a neural network achieving 95 percent rarely justifies the cost: 100–1,000× longer training, 10–50× more memory, and ongoing maintenance burden. As shown in section 1.2.3, neural networks excel at hierarchical pattern discovery but impose substantial overhead. Reserve them for problems with spatial locality, temporal dependencies, or high-dimensional nonlinear interactions that simpler models cannot capture.

Fallacy: *Training data distribution issues can be fixed after model design.*

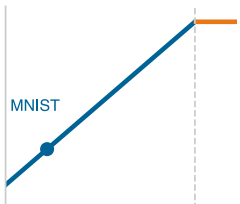
Teams treat training as mechanically feeding data through architectures. Networks on imbalanced datasets exhibit catastrophic minority-class performance: a fraud detector with 99:1 imbalance achieves 99 percent accuracy by always predicting "not fraud" while catching zero fraud cases. The loss functions in section 1.4.3 optimize for average-case performance, causing networks to ignore rare but critical classes. Teams that skip exploratory data analysis deploy models achieving strong metrics on balanced holdout sets but failing on production data with 10:1 or 100:1 imbalances, requiring expensive retraining.

Pitfall: *Deploying research models to production without addressing system constraints.*

Data scientists develop models with unlimited time budgets, assuming deployment is straightforward. Production imposes constraints absent from research: latency budgets (50–100 ms end-to-end), memory limits (2–4 GB for edge devices), and concurrent loads (100–1,000 requests per second (RPS)). As shown in section 1.5, the complete pipeline includes preprocessing, inference, and postprocessing. A model achieving 20 ms inference fails its 50 ms budget when preprocessing adds 25 ms and postprocessing adds 10 ms (55 ms total). Teams separating model development from system design waste months optimizing accuracy while ignoring constraints that determine deployment feasibility.

Fallacy: *More compute automatically means faster training.*

Teams purchase expensive GPUs expecting proportional speedups, then discover workloads are memory bound. Arithmetic intensity determines which resource constrains performance. The MNIST forward-pass analysis in table 1.11 and the roofline model in **?@sec-appdx-machine-foundations-roofline-model-2529** show why small networks like MNIST (784 to 128 to 64 to 10) have arithmetic intensity of approximately 0.5 FLOP/byte, far below the hundreds of FLOP/byte



Small neural workloads stay memory-bound even on large GPUs.

often required to keep high-throughput accelerators compute-bound. For memory-bound workloads, a commodity CPU can match an expensive accelerator; for compute-bound GPT-scale models, accelerators provide the large speedups they were built for. This mismatch explains why teams report widely varying utilization depending on model architecture.

Pitfall: *Extrapolating accuracy improvements without considering diminishing returns.*

Teams observe that scaling from 10K to 100K parameters improves accuracy by 5 percentage points, then assume scaling to 1M parameters yields another 5 points. Neural network accuracy follows logarithmic scaling: each order of magnitude in compute yields diminishing returns. Within table 1.5, the ImageNet rows show error falling from AlexNet's 15.3 percent to ResNet-152's 3.6 percent while training FLOPs rise from 5×10^{17} to roughly 10^{19}, about one to two orders of magnitude. The lesson is not a fixed percentage-point-per-order rule; it is that later accuracy gains become progressively more expensive. Achieving 99 percent accuracy might cost $10\times$ more than 98 percent, and 99.9 percent might cost $100\times$ more than 99 percent. Teams that fail to model this relationship overpromise accuracy and underestimate resources.

These fallacies and pitfalls share a common root: applying intuitions from deterministic software engineering to probabilistic learning systems. Recognizing them early saves weeks of misdirected effort and prevents production failures that are expensive to diagnose after deployment.

1.9 Summary

Deep learning systems engineers need mathematical understanding precisely because neural networks cannot be treated as black-box components. When a production model fails, the problem lies not in the code but in the mathematics: a misconfigured learning rate causes gradients to explode during backpropagation, an activation function saturates and blocks learning in deep layers, or memory requirements during training exceed GPU capacity because of stored activations and optimizer states. Engineers who understand forward propagation can trace which layer produces anomalous activations. Engineers who understand backpropagation can diagnose vanishing gradients. Engineers who understand the distinction between training and inference can predict memory consumption before deployment surprises them.

Neural networks transform computational approaches by replacing rule-based programming with adaptive systems that learn patterns from data. The biological-to-artificial neuron mapping (weighted sums, nonlinear activations, and gradient-based learning) provides the atomic operations from which all modern architectures are composed.

Neural network architecture demonstrates hierarchical processing, where each layer extracts progressively more abstract patterns from raw data. Training adjusts connection weights through iterative optimization to minimize prediction errors, while inference applies learned knowledge to make predictions on new data. This separation between learning and application phases creates distinct system requirements for computational resources, memory usage, and processing latency that shape system design and deployment strategies. Training requires ~4.3× more memory than inference because gradients, optimizer state, and activations must be stored and updated. The USPS digit recognition case study demonstrated that these mathematical principles combine into production systems where the complete pipeline (preprocessing, neural inference, and postprocessing) must operate within real-world latency and reliability constraints.

The running MNIST example made this escalation tangible: the same 28 by 28 digit that required about 100 comparisons demanded 109,184 MACs in even a modest three-layer network—a 1,091.8× increase that generalizes across the systems dimensions captured in table 1.3. These fundamentals primarily develop the algorithm axis of the D·A·M taxonomy while revealing how algorithmic choices propagate into machine constraints.

The mathematical and systems implications emerge through fully connected architectures. The multilayer perceptrons explored here demonstrate universal function approximation: with enough neurons and appropriate weights, such networks can theoretically learn any continuous function. This mathematical generality comes with computational costs. Consider our MNIST example: a 28 by 28 pixel image contains 784 input values, and a fully connected network treats each pixel independently, learning over 100,352 weights in the first layer alone by connecting 784 inputs to 128 neurons. Neighboring pixels are highly correlated while distant pixels rarely interact. Fully connected architectures expend computational resources learning irrelevant long-range relationships.

These foundations establish the mathematical and systems vocabulary for reasoning about neural network behavior. The forward-backward propagation cycle, activation function choices, and memory-computation trade-offs recur throughout every subsequent chapter, whether analyzing why certain architectures train faster, why lower-precision approximations preserve accuracy in some layers but not others, or why multi-machine training requires careful coordination. Understanding these fundamentals enables engineers to move beyond treating neural networks as black boxes toward principled system design.

Key Takeaways: The math behind the model

- **Each paradigm shift buys representation power at exponential systems cost**: Classifying the same 28×28 digit escalates from ~100 comparisons (rule-based) through ~8,000 operations (classical ML) to 109,184 MACs (deep learning)—a 1,091.8× increase that reshapes hardware requirements at every level.
- **Neural networks learn patterns, not rules**: These networks replace hand-coded features with hierarchical representations discovered from data. The system adapts to the problem rather than requiring manual engineering.
- **Training and inference have opposite priorities**: Training optimizes throughput (large batches, hours of compute); inference optimizes latency (single samples, milliseconds). Batch size is the systems lever that links utilization, memory, throughput, and statistical stability across both phases.
- **Activations are math and hardware**: ReLU dominates because $\max(0, x)$ is orders of magnitude cheaper than $\exp(x)$, and its constant gradient for positive inputs prevents the vanishing gradient problem that plagues sigmoid and tanh in deep networks.
- **Forward propagation is matrix work**: Dense matrix kernels account for over 90 percent of neural network FLOPs, which is why hardware optimized for dense matrix operations can outperform general-purpose CPUs by orders of magnitude.
- **Backpropagation stores the path**: Solving credit assignment requires saving intermediate activations, so memory cost often determines whether a model can be trained on a given device and motivates techniques that reduce, recompute, or partition training state.
- **The complete ML pipeline determines end-to-end performance**: Preprocessing, neural computation, and postprocessing all contribute to latency and reliability. The USPS deployment demonstrated that production success depends on the entire pipeline operating within real-world constraints, not on model accuracy on a test set alone.

Reading the code reveals what a network is asked to do; reading the math reveals what it will cost to do it. That is why this chapter dwells on the arithmetic. A neural network is the point where the algorithm meets the machine: every weight is a number that must be stored and moved, every layer a matrix multiply that must land on silicon built for dense arithmetic, every saved activation a claim on memory that decides whether the model trains at all. The math is where an algorithm signs its contract with the hardware, committing in advance to the operations the machine runs efficiently and paying in latency for any it does not. To read that contract is to know, before a single line of code runs, where the network will be fast and where it will stall.

What's Next: From universal to specialized

Real-world problems exhibit structure that generic fully-connected networks cannot efficiently exploit: images have spatial locality, text has sequential dependencies, and time-series data has temporal dynamics. **?@sec-network-architectures** addresses this structural blindness through specialized architectures that encode problem structure directly into network design. Each architecture trades the universal flexibility of fully-connected networks for inductive biases that match problem structure, achieving dramatic efficiency gains while creating new

systems engineering trade-offs in memory access patterns, parallelization constraints, and computational bottlenecks.

Belkin, Mikhail, Daniel Hsu, Siyuan Ma, and Soumik Mandal. 2019. "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off." *Proceedings of the National Academy of Sciences* 116 (32): 15849–54. https://doi.org/10.1073/pnas.1903070116.

Dalal, Navneet, and Bill Triggs. 2005. "Histograms of Oriented Gradients for Human Detection." *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* 1: 886–93. https://doi.org/10.1109/cvpr.2005.177.

European Space Agency. 1996. *Ariane 501: Presentation of Inquiry Board Report*. European Space Agency press release.

Frankle, Jonathan, and Michael Carbin. 2019. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks." *International Conference on Learning Representations*.

Gabor, D. 1946. "Theory of Communication. Part 1: The Analysis of Information." *Journal of the Institution of Electrical Engineers - Part III: Radio and Communication Engineering* 93 (26): 429–41. https://doi.org/10.1049/ji-3-2.1946.0074.

Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the Difficulty of Training Deep Feedforward Neural Networks." *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 9: 249–56.

Goyal, Priya, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour." *arXiv Preprint arXiv:1706.02677* abs/1706.02677.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." *2015 IEEE International Conference on Computer Vision (ICCV)*, 1026–34. https://doi.org/10.1109/iccv.2015.123.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. "Deep Residual Learning for Image Recognition." *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–78. https://doi.org/10.1109/cvpr.2016.90.

Horowitz, Mark. 2014. "1.1 Computing's Energy Problem (and What We Can Do about It)." *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. https://doi.org/10.1109/isscc.2014.6757323.

Ioffe, Sergey, and Christian Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *Proceedings of the 32nd International Conference on Machine Learning (ICML)* 37: 448–56.

Jouppi, Norm, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, et al. 2023. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 1–14. https://doi.org/10.1145/3579371.3589350.

Kingma, Diederik P., and Jimmy Ba. 2014. "Adam: A Method for Stochastic Optimization." *ICLR* in press.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems* 25.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. "Deep Learning." *Nature* 521 (7553): 436–44. https://doi.org/10.1038/nature14539.

LeCun, Yann, Bernhard Boser, John S. Denker, Donald Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. "Backpropagation Applied to Handwritten Zip Code Recognition." *Neural Computation* 1 (4): 541–51. https://doi.org/10.1162/neco.1989.1.4.541.

LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86 (11): 2278–324. https://doi.org/10.1109/5.726791.

Linnainmaa, Seppo. 1970. "The Representation of the Cumulative Rounding Error of an Algorithm as a Taylor Expansion of the Local Rounding Errors." Master's thesis, University of Helsinki.

Lowe, David G. 1999. "Object Recognition from Local Scale-Invariant Features." *Proceedings of the Seventh IEEE International Conference on Computer Vision* 2: 1150–1157 vol.2. https://doi.org/10.1109/iccv.1999.790410.

McCulloch, Warren S., and Walter Pitts. 1943. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *The Bulletin of Mathematical Biophysics* 5 (4): 115–33. https://doi.org/10.1007/bf02478259.

Minsky, Marvin, and Seymour A. Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.

Mitchell, Tom M. 1980. *The Need for Biases in Learning Generalizations*. CBM-TR-117. Rutgers University, Department of Computer Science.

Nair, Vinod, and Geoffrey E. Hinton. 2010. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807–14.

Nakkiran, Preetum, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. 2019. "Deep Double Descent: Where Bigger Models and More Data Hurt." *arXiv Preprint arXiv:1912.02292*.

Neyshabur, Behnam, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. 2017. "Exploring Generalization in Deep Learning." *Advances in Neural Information Processing Systems* 30.

Raichle, Marcus E., and Debra A. Gusnard. 2002. "Appraising the Brain's Energy Budget." *Proceedings of the National Academy of Sciences* 99 (16): 10237–39. https://doi.org/10.1073/pnas.172399499.

Rosenblatt, Frank. 1958. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65 (6): 386–408. https://doi.org/10.1037/h0042519.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323 (6088): 533–36. https://doi.org/10.1038/323533a0.

Shannon, Claude E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27 (3): 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Srivastava, Nitish, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15 (1): 1929–58.

Tan, Mingxing, and Quoc V Le. 2019. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *International Conference on Machine Learning (ICML)*, 6105–14.

Telgarsky, Matus. 2016. "Benefits of Depth in Neural Networks." *Proceedings of the 29th Annual Conference on Learning Theory*, 1517–39.

Wengert, Ralph E. 1964. "A Simple Automatic Derivative Evaluation Program." *Communications of the ACM* 7 (8): 463–64. https://doi.org/10.1145/355586.364791.

Werbos, Paul. 1974. "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences." PhD thesis, Harvard University.

Wolpert, D. H., and W. G. Macready. 1997. "No Free Lunch Theorems for Optimization." *IEEE Transactions on Evolutionary Computation* 1 (1): 67–82. https://doi.org/10.1109/4235.585893.